

データベース

レクチャー	レクチャーで学ぶ内容
データベースの基礎	最適なデータベースの選択を実施するために様々なデータベースのタイプとその活用方法を概要的に理解します。
DynamoDBの概要	DynamoDBの基本的な機能や仕組みについて理解します。
DynamoDBの構築 (ハンズオン)	実際にDynamoDBを構築して、その設定や操作方法を学習します。
Auroraの概要	Auroraの基本的な機能や仕組みについて理解します。
Auroraの構築 (ハンズオン)	実際にAuroraを構築して、その設定や操作方法を学習します。

データベース

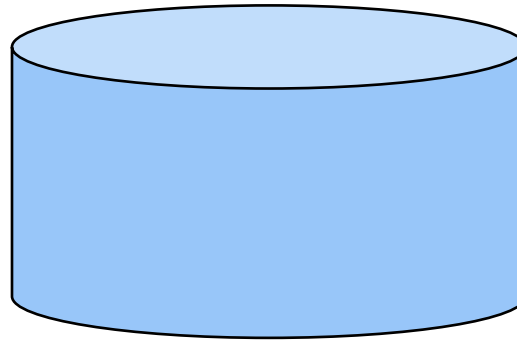
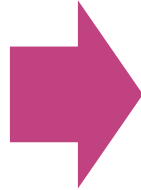
レクチャー	レクチャーで学ぶ内容
Auroraサーバレスの設定	Auroraのサーバレス構成であるAuroraサーバレスの基本を学習した上で実際にハンズオンで操作方法を学習します。
Redshiftの概要	Redshiftの基本的な機能や仕組みについて理解します。
Redshiftの構築 (ハンズオン)	Redshiftを実際に構築して、その設定や操作方法を学習します。
ElastiCacheの概要	ElastiCache RedisとMemcachedの基本的な機能や仕組みについて理解します。
ElastiCacheの構築 (ハンズオン)	ElastiCache RedisとMemcachedを実際に構築して、その設定や操作方法を学習します。

データベースの基礎

データベース

データベースは関連したデータを形式を揃えて収集・整理して、検索などの操作やデータ管理を実行するシステム

新規にデータを
保存する。

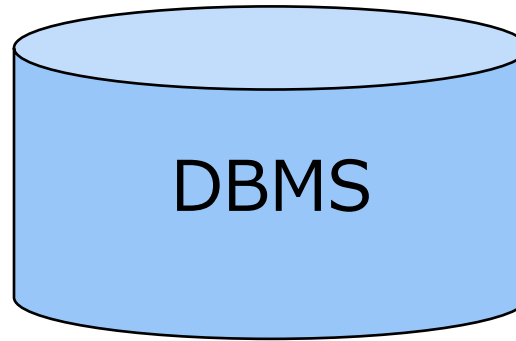
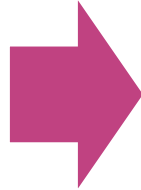


DBからデータを
抽出する。

データベース

データベースを実現したシステムをDBMS（データベースマネジメントシステムという）

新規にデータを
保存する。



DBからデータを
抽出する。

データベース

基本的なデータベースはテーブルという形式でデータを格納している。

社員名簿テーブル

ID	名前	部署	生年月日
1001	佐藤周大	営業	19870621
1002	小林隆文	開発	19830821
1003	元木博夫	経理	19820524

データベース

データベースは追加・参照・更新・削除などのデータ操作を容易に実行するソフトウェアやデータモデルと一体

- 追加：データを整理して保存したい。
- 参照：必要なデータを参照または抽出したい。
- 更新：データの変化に応じて好きなときに更新したい。
- 削除：必要ないデータを削除したい。

データベース

追加・参照・更新・削除を総称してCRUDと呼ぶ。このCRUD操作を実現するのがデータベースとなる。

- Create 追加：データを整理して保存したい。
- Read 参照：必要なデータを参照または抽出したい。
- Update 更新：データの変化に応じて好きなときに更新したい。
- Delete 削除：必要ないデータを削除したい。

データベースとストレージ

ストレージはデータベースの記憶装置を構成する1つの要素

ストレージ

- コンピュータの主要な構成要素の一つで、データを永続的に記憶する装置

データベース

- データベース内のデータを保存する装置はストレージであるが、データベースそのものではない。
- ストレージ+データを管理・操作するデータベースソフトウェア

データベースの役割

データベースはデータ操作を異状なく実行でき、データを安全に保護しつつ、保存・操作ができる仕組みを提供している。

【データ操作にかかる様々な課題】

- システムがクラッシュしたときにデータが消えないか？
- データを誤って削除してしまった場合に対処できないか？
- データ抽出に誤りが発生しないか？
- 2人が同時に同じデータにアクセスした際にどうするか？
- 大量のデータを上手く検索できないか？

データベースの役割

データベースの役割を支える仕組みを理解する。

トランザクション

データベースをある一貫した状態から別の一貫した状態へ変更する1つの処理の束のこと

データモデル

実世界におけるデータの集合を、DBMS上で利用可能な形に落とし込むためのモデル

トランザクション

データベースをある一貫した状態から別の一貫した状態へ変更する1つの処理の束のこと。

- 同時アクセスした場合に上手く処理する。
- データ処理に失敗したら、元に戻してくれる。
- システムがクラッシュしてもデータを保護する。

トランザクション：ACID

ACIDは信頼性のあるトランザクションシステムの持つべき性質のこと

- Atomicity（原子性）

トランザクションが「すべて実行される」か「一つも実行されない」のどちらかの状態になるという性質

- Consistency（整合性）

トランザクションの前後でデータの整合性が保たれ、矛盾の無い状態が継続される性質

- Isolation（独立性）

トランザクション実行中の処理過程が外部から隠蔽され、他の処理などに影響を与えない性質

- Durability（耐久性）

トランザクションが完了したら、その結果は記録され、クラッシュしても失われることがないという性質

トランザクション：耐久性

データを更新する際にCOMMITとする更新が反映されるが、COMMITされないとデータがロールバックして保護する。

データ更新
(COMMIT実施)

氏名データ：高橋を佐藤に更新する。



氏名データの更新がCOMMITされる。



データベースがクラッシュする。



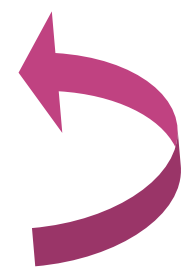
氏名データ：佐藤でデータを保護

データ更新
(COMMIT未実施)

氏名データ：高橋を佐藤に更新する。



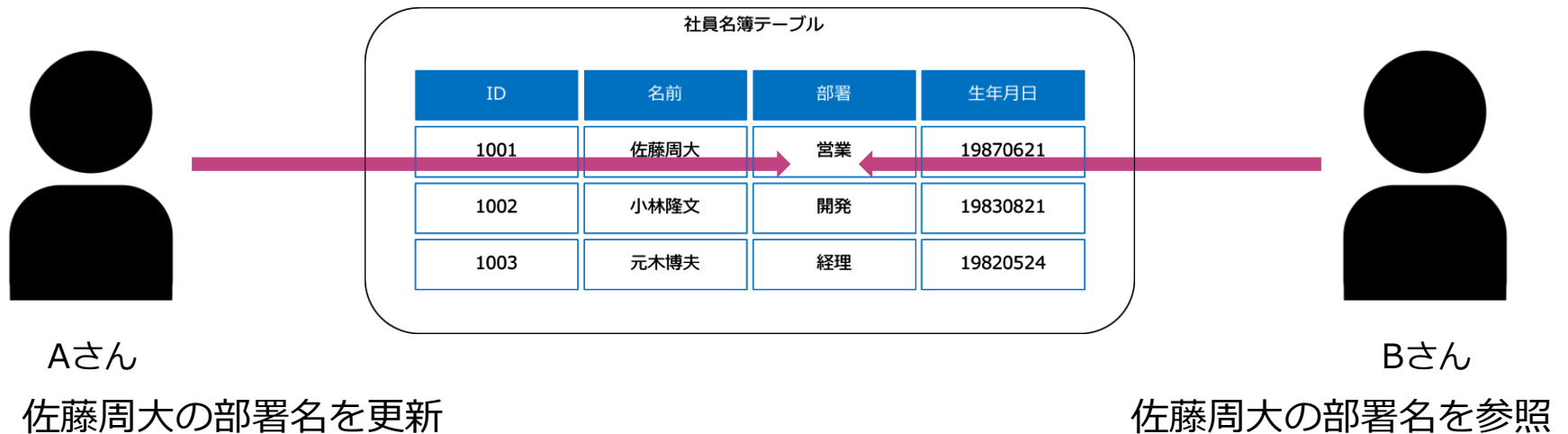
データベースがクラッシュする。



ロールバック
する。

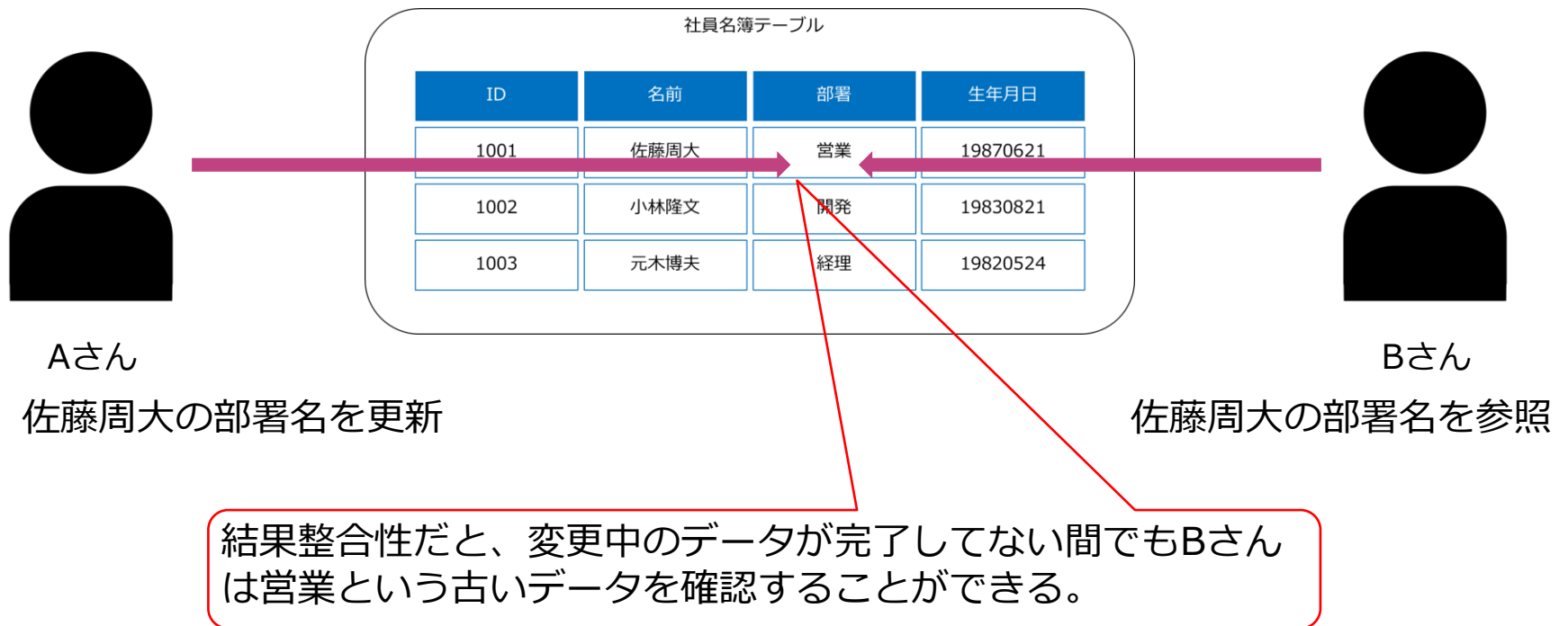
トランザクション：整合性

同時に複数人がアクセスした場合などにデータ整合性を維持する必要がある。



整合性モデル

同時に複数人がアクセスした場合などの、データのデータ整合性を維持するための方式（結果整合性や強い整合性など）



整合性モデル

同時に複数人がアクセスした場合などの、データのデータ整合性を維持するための方式（結果整合性や強い整合性など）



強い整合性だと、変更中のデータが完了していない間はBさんは変更完了までデータを参照できない。

データモデル

データモデルはデータベースのデータの持ち方などの構造や処理を定めるデータの論理的な表現方法

```
graph TD; A[データモデル] --> B[データモデルに応じたトランザクション機能];
```

データモデルに応じたトランザクション機能

データモデル

データモデル

データベースには様々なデータモデルが存在し、利用目的に応じて使い分ける

- リレーショナルモデル
- グラフモデル
- キーバリューストア
- オブジェクト
- ドキュメント
- ワイドカラム
- 階層型

⑥最適なデータベース選択

ワークロードに応じた最適なデータベース技術を利用する

関連する主要サービス

RedShift

RDS

Dynamo
DB

Aurora

Elastic
search

データベース

データベースはリレーショナルDBかそうでないDBかの大きく2つの種類がある

これまでのDB

リレーショナル
DB

ビッグデータ向けDB

NoSQL

リレーショナルDB

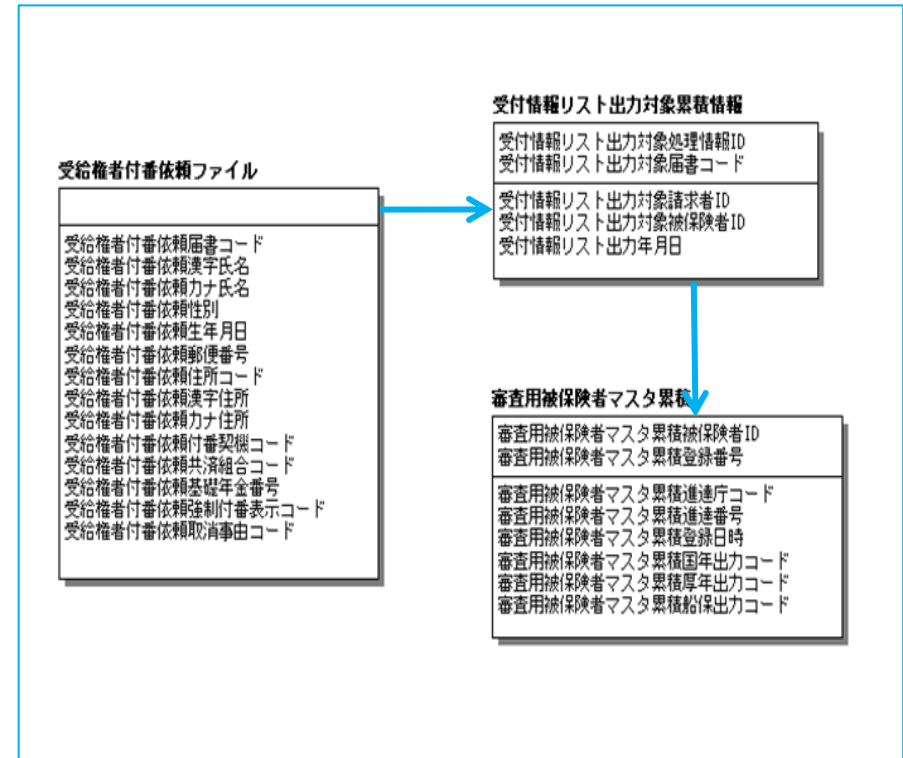
データベースの基本はリレーショナルDBシステム（RDBS）

■ 概要

- データ間の関係性が定義されたデータを取り扱う一般的なDBシステム
- 列と行がいくつかのテーブルで定義されていて、テーブル間のリレーションが設計される
- データ操作にSQLを利用

■ 利用データ

- 会計データ／顧客データといった構造化データ



NoSQL

IoTと同様にビッグデータ解析にはNoSQL DBを利用する

■ 概要

- リレーショナルデータ構造を持たずSQLを利用しないDBの総称
- ただし、現在はSQLやSQLに似た適用したモデルもある

■ 利用データ

- 構造化されていないKeyとValueのみ（ID番号に一行に全データを格納）のKVSデータ
- 動画／画像／ドキュメントなどの非構造化データ
- XML/JSONなどの半構造化データ

非構造化

テキスト

動画

音声

半構造化

XML

JSON

文書

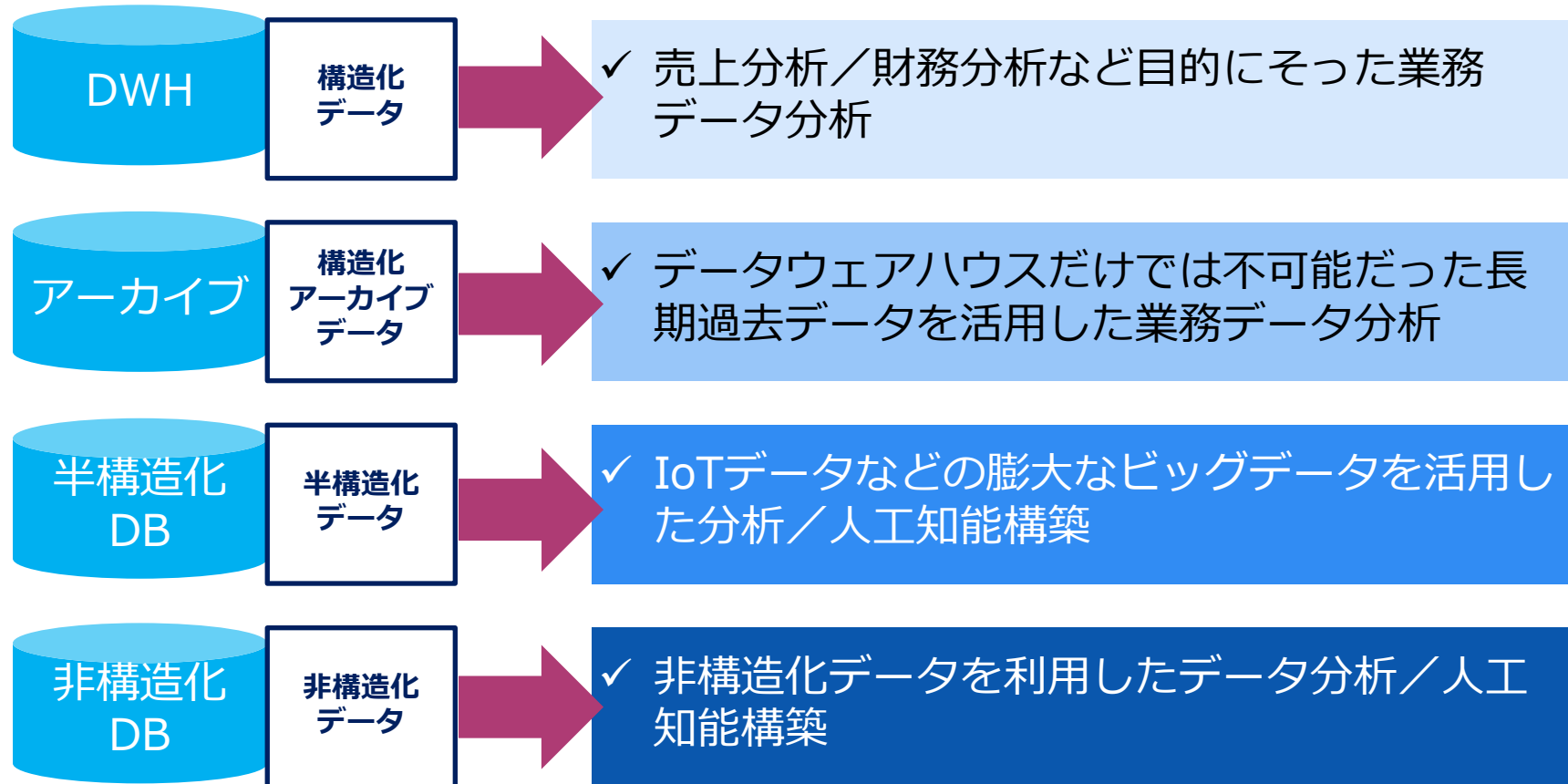
NoSQL

キーに対するデータ形式の格納方式の違いで様々なタイプのNoSQLが存在する

キーバリューストア	ドキュメントデータベース
✓ キーに対してバリュー（値）を入れる単純な構造 ✓ 高速なパフォーマンスと分散型拡張に優れている ✓ データ読み込みが高速	✓ キーに対してバリューではなく、JSONやXMLなどのデータを格納 ✓ 複雑なデータ構造を扱うアプリで生産性高く柔軟に開発する
ワイドカラムストア	グラフデータベース
✓ 列指向とも呼ばれ、キーを利用するがデータはレコード（行）で管理する ✓ 非構造データを大規模に格納することを目的にしている ✓ 行ごとに任意の名前のカラムを無数に格納できる	✓ グラフ理論に基づき、データ同士の関係をグラフで総合に結びつけた要素で構成される ✓ RDBと比較して高速横断検索が可能

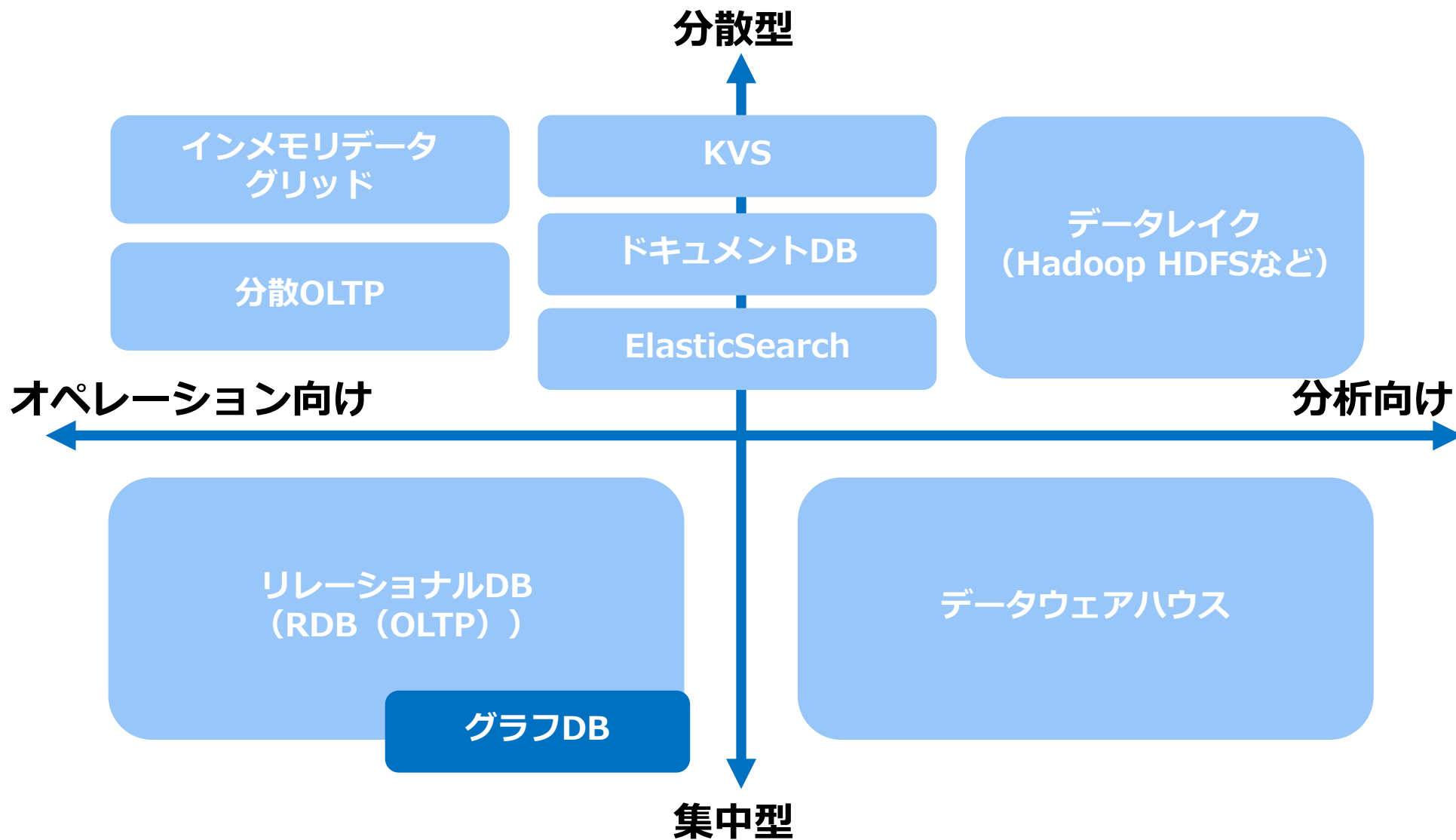
ビッグデータの活用

データの特徴に応じて利用するデータベースを選択する



データベース全体像

データベースを2つの軸で表すと次のように分類される



リレーショナルDB（RDB）

業務システム向けのDBの基本はリレーショナルデータベース

■ 概要

- 業務システムなどで最も頻繁に利用されるオペレーション用のデータベース
- 利用者はSQLなどのクエリ言語でデータ操作を実施

■ アーキテクチャ

- テーブル間のリレーションが定義されたデータモデル
- 行志向で1つの行をデータのかたまりとして扱う

■ 利用データ

- 会計データなどの業務系の構造化データ



データウェアハウス（DWH）

構造化データを利用した経営分析向けのデータベース

■ 概要

- データの抽出・集約に特化したBIデータ分析用のデータベース
- 読み込むデータ構造を予め設計して、加工してから利用分のデータを蓄積
- レスポンス重視でデータ抽出・集計が速いが、更新・トランザクションは遅い

■ アーキテクチャ

- データをパーティショニングして、複数ディスクから読み込む
- 列志向でデータを格納

■ 利用データ

- 会計データなどの業務系の構造化データを分析用に加工し、BIで利用
- KPI測定／競合分析／アクセス分析など

オンプレ／ソフトウェア



ORACLE
Exadata



TERADATA



AWSサービス



Redshift

分散型DB／データレイク

ビッグデータやIoTデータを蓄積して高速処理を可能にするDBとストレージの組合せ

■ 概要

- データ抽出に特化したDB
- 分散してデータを保存しており、ビッグデータの高速処理向け

■ アーキテクチャ

- SQLライクなクエリで操作可能
- INSERT/UPDATE/DELETEはない
- トランザクションはない
- データ書き込みは一括ロードまたは全件削除のみ

■ 利用データ

- ビッグデータ



KVS :キーバリュ型

シンプルなデータ構造にすることで高速処理を可能にしたDB

■ 概要

- 分散して、シンプルなオペレーションを高速に実施できるDB

■ アーキテクチャ

- 強い整合性を犠牲にして、結果的な整合性を採用
- 分散向けのデータモデル／クエリの採用
- トランザクション／集計／JOINなど不可

■ 利用データ

- 大規模WEBサイトのバックエンドデータ（ユーザーセッション／ユーザー属性／事前計算データのキャッシュ）
- メッセージングシステムのデータ
- 大規模書き込みが必要なIOTセンサーデータなど

オンプレ



redis



AWSサービス



ElastiCache

KVS :キーバリュー型

リレーショナルなしにバリュー一行にデータをまとめることで、高速処理を可能にする

リレーショナルDBのテーブル

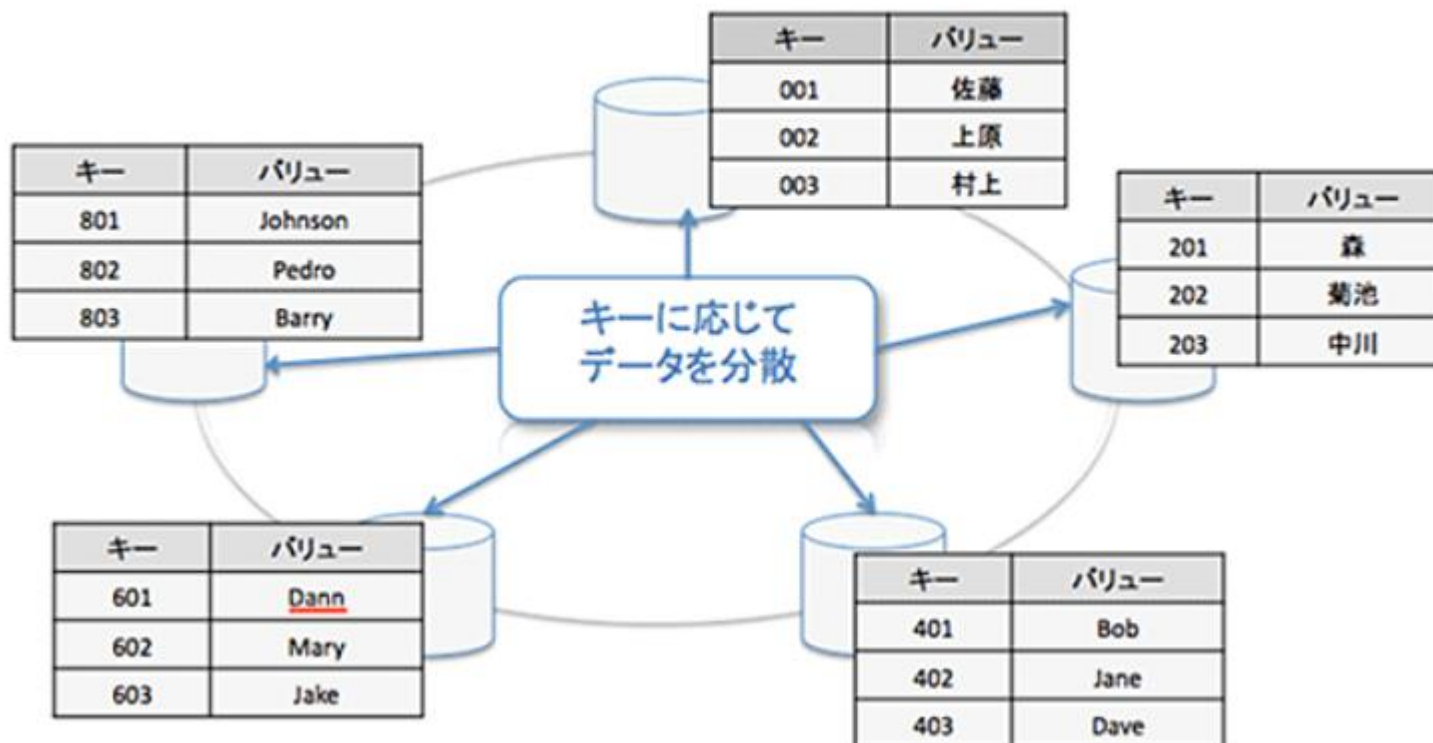
部門	場所	役職	名前
人事	豊洲	部長	佐藤
製造	新宿	課長	上原
総務	三鷹	社員	村上

キーバリュー型DBのテーブル

キー	バリュー
1	名前:佐藤、部門:総務、場所:三鷹、役職:部長
2	名前:上原、部門:人事、場所:新宿、役職:課長
3	名前:村上、部門:製造、場所:新宿、役職:社員

KVS :キーバリュー型

キーに応じてデータを分散して処理することができる



ワイドカラム型

キーに対してカラムを大規模に登録できるのがワイドカラム型

■ 概要

- 分散して、シンプルなオペレーションを高速に実施できるDB
- データ取得する際にデータ結合しなくても済むように、可能な限り多くのデータを同じ行に保持

■ アーキテクチャ

- 結果整合性を採用
- キースペース、カラムファミリ、row、(スーパーカラム)、カラムの入れ子構造
- SQLライクなデータ操作が可能
- データ操作は挿入、削除、参照のみで、データの更新は挿入による上書き

■ 利用データ

- Facebook/Twitterなどソーシャルデータの位置情報データストレージ／リアルタイム分析／データマイニング処理

オンプレ



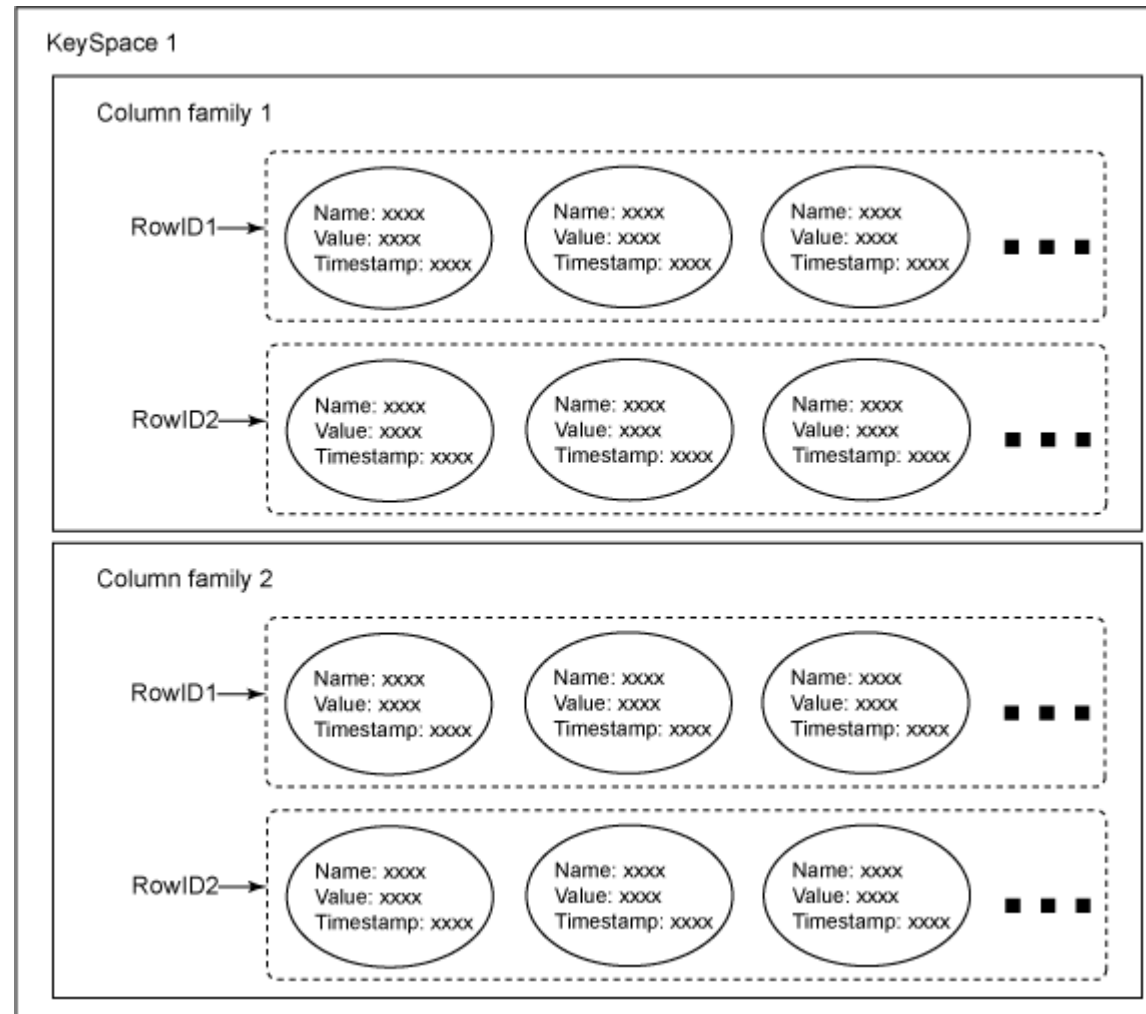
APACHE
HBASE

AWSサービス



ワイドカラム型

キーとカラムが入れ子構造となったデータモデルを利用



ドキュメントDB

キーに対してドキュメント指向でXMLなどのデータを格納する

■ 概要

- ドキュメント指向データベースでは、さまざまなデータ構造のドキュメントを混在して保存することができる

■ アーキテクチャ

- JSON/XMLをデータモデルに利用
- 小規模データの同期集計処理が可能だが、バッチは不向き
- SQLライクなデータ操作が可能で、KVSよりもクエリが豊富なため操作しやすい
- Shardingによるデータベース分散化

■ 利用データ

- 半構造化データ（XML/JSON）
- 大規模WEBのログ保管など
- オンラインゲームデータ
- カタログ管理

オンプレ



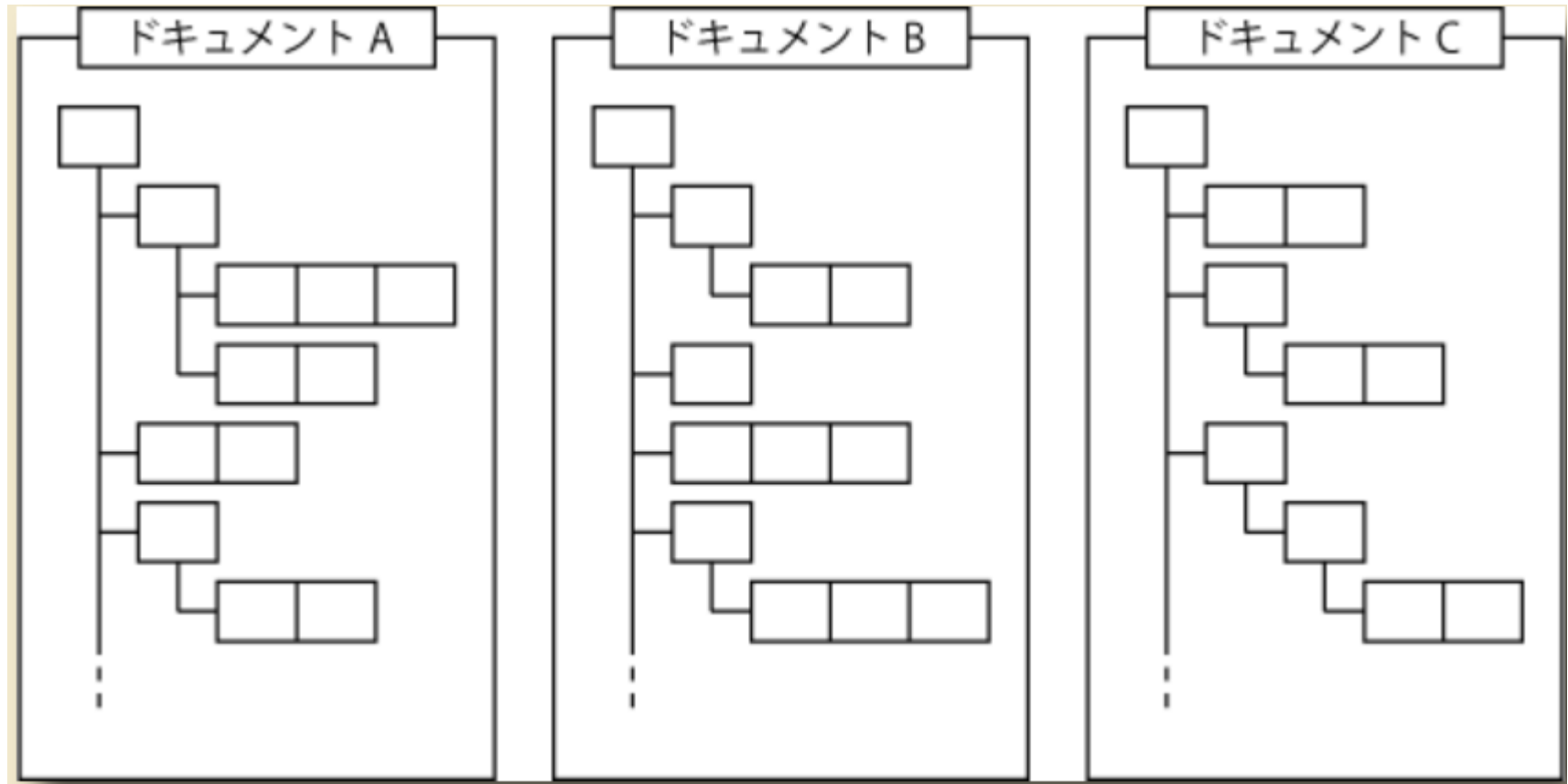
AWSサービス

Amazon DocumentDB



ドキュメントDB

様々な形式のドキュメント構造を一緒に保存できる



インメモリデータグリッド

KVSの仕組みをメモリを利用してより高性能にしたDB

■ 概要

- 大量のデータを多数のサーバのメモリ上で分散して管理する技術
- ミリ秒単位の高速の応答処理が可能

■ アーキテクチャ

- データをメモリ上に置くことで、高速なデータアクセスを実現
- データを多数のサーバで分散して管理

■ 利用データ

- 金融の取引処理データでミリ秒以下の応答時間を実現



全検索型エンジン×分散DB

データの全検索エンジンであるElasticsearchは分散データベースと連携してデータ全検索処理が可能

■ 概要

- 全検索型のデータ検索エンジンで、分散データベースと連携して検索データベースを構築
- 検索条件との関係性/関連性が高いデータを抽出して返す

■ アーキテクチャ

- Elasticsearchは全文検索用のライブラリ Apache Luceneを利用したデータストア
- 分析の柔軟性や速度が高く、分析/蓄積/可視化環境を容易に構築可能

■ 利用データ

- 構造化データ／非構造化データ
- 高可用な全検索エンジン
- サイト内データの検索
- デバイス登録状況・配信状況のリアルタイム可視化などリアルタイムの検索要件／検索行動の可視化



グラフDB

グラフ構造でデータ間のつながりを検索・可視化するDB

■ 概要

- グラフ演算に特化したDBで、データ間のつながり方を検索・可視化に利用

■ アーキテクチャ

- グラフデータ構造を取るため、RDB以上にスケールアウトができない
- レコード数が増えると、検索に掛かる時間と難易度が増大
- ACID特性が担保されており、オブジェクト間の関連づけを簡単に表現できる

■ 利用データ

- 最短経路探索
- 金融取引の詐欺検出
- ソーシャルネットワークによるリレーション計算

オンプレ



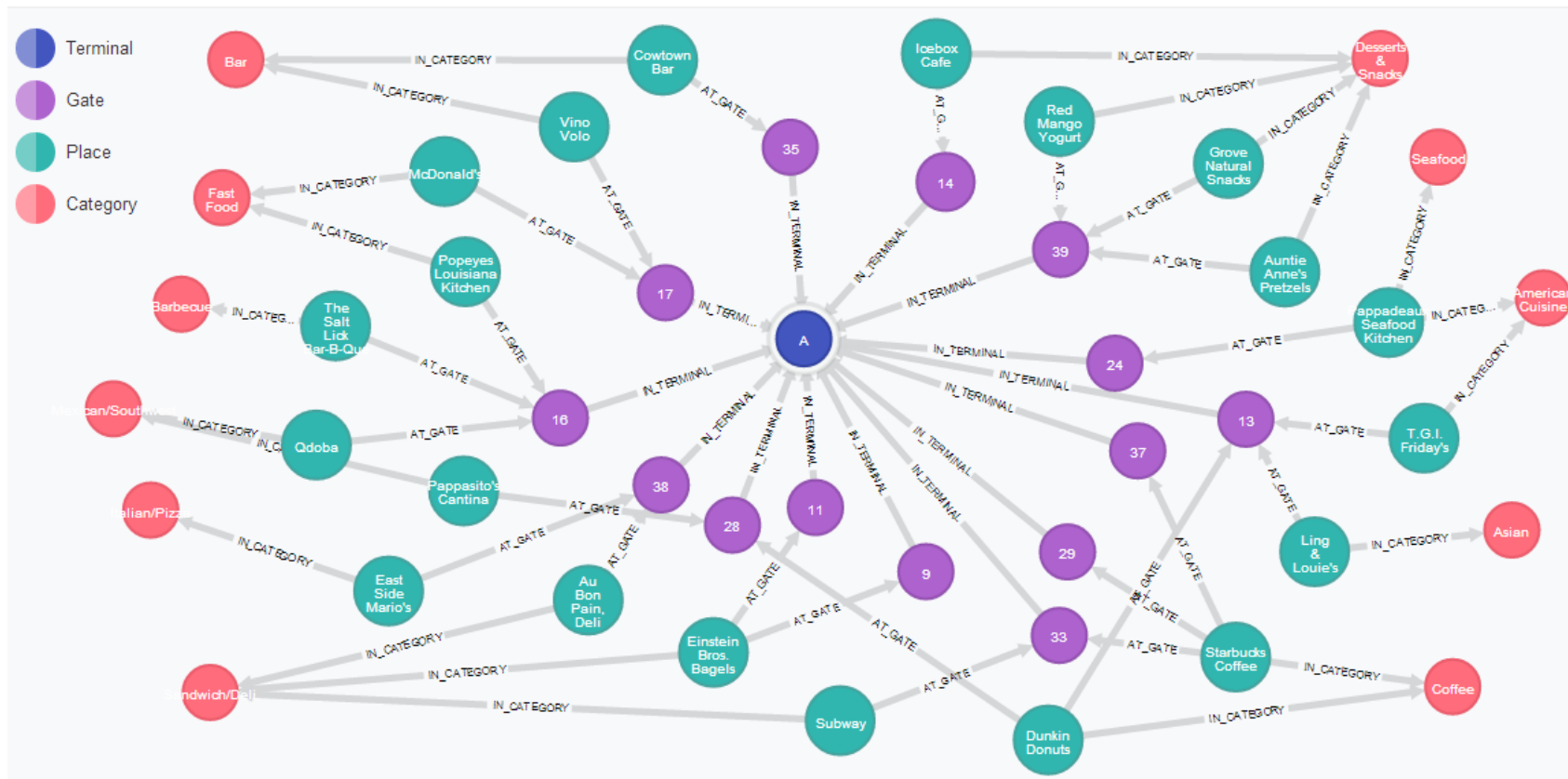
AWSサービス

Amazon Neptune

グラフDB

グラフ構造はマインドマップのようにデータ関連をグラフ表示するデータ構造

```
CYPHER MATCH p = (:Category)<--(:Place)-[*]->(:Terminal {name:'A'}) RETURN p
```



参照: <https://nicolewhite.github.io/2014/06/30/create-shiny-app-neo4j-graphene.html>

分散OLTP (RDB)

オンライントランザクション処理 (Online Transaction Processing) を分散化する次世代DB

■ 概要

- グローバルに分散され、強整合性を備えたデータベース

■ アーキテクチャ

- リレーショナルデータベースの構造と非リレーショナルデータベースの分散スケーラビリティを兼ね備える
- 高い可用性、高性能のトランザクションと強整合性が実現

■ 利用データ

- 大規模な業務データ処理

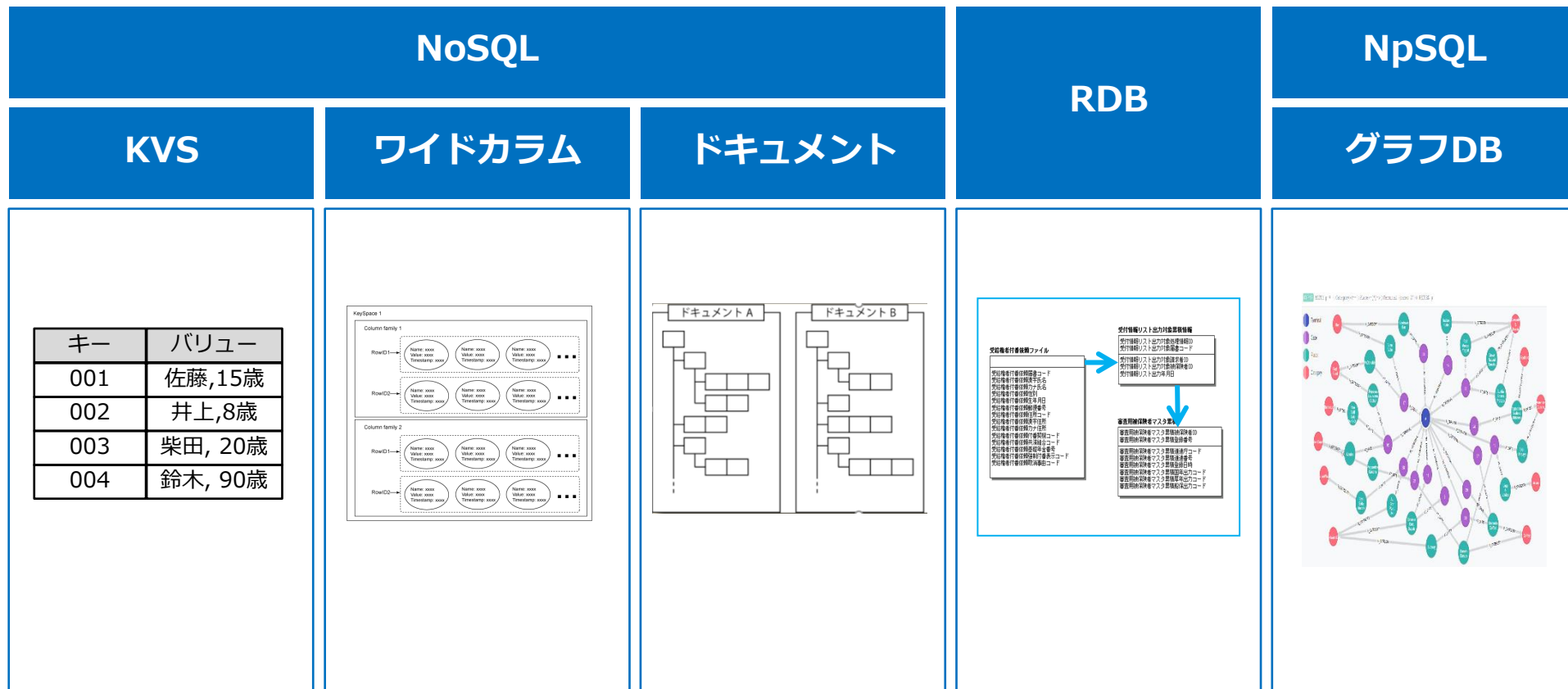
オンプレ

AWSサービス



DB種別とデータモデル

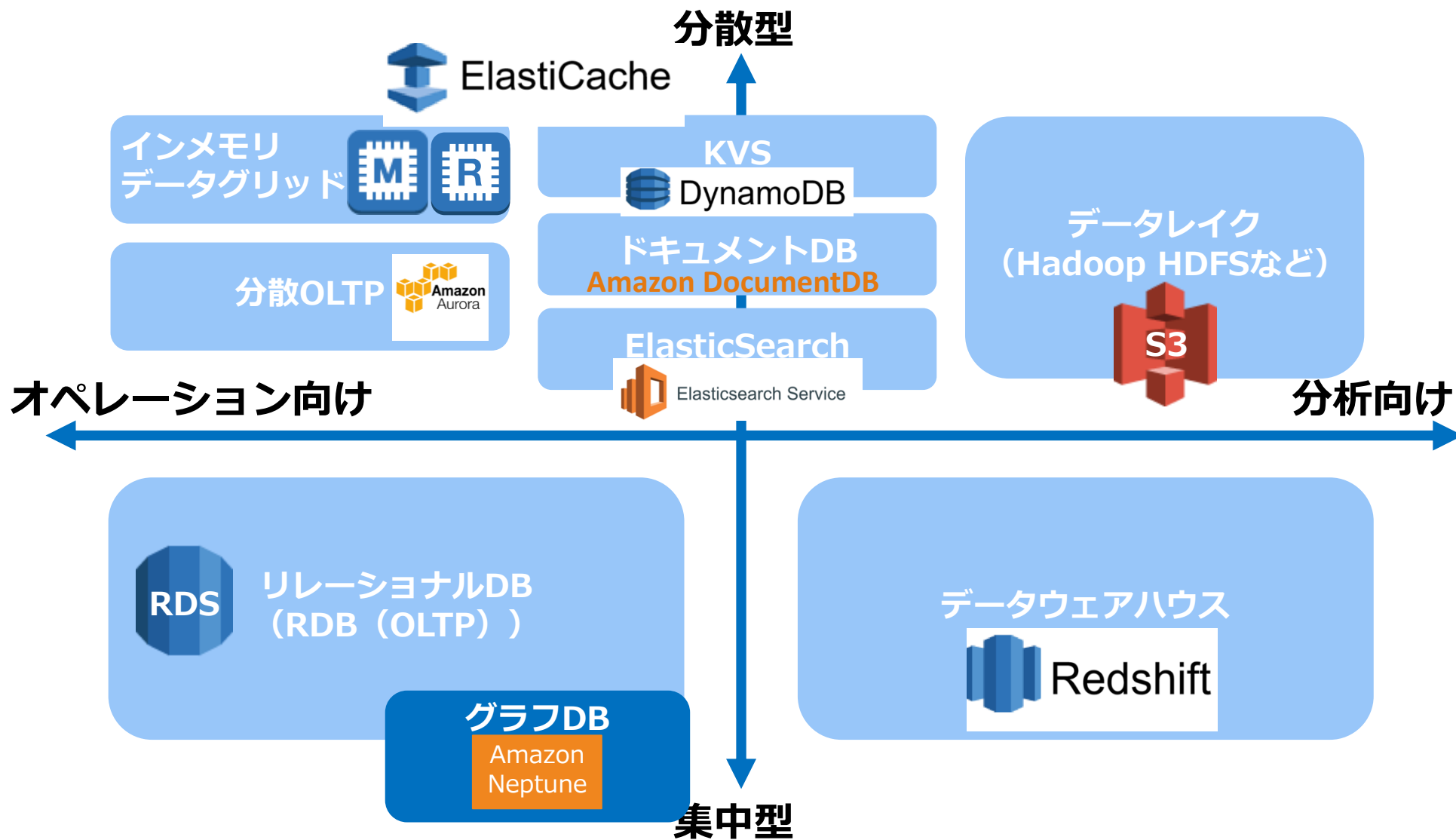
データ構造がシンプルなデータベースが分散しやすく、ビッグデータ処理に向いている



シンプルで分散しやすい

複雑なデータ処理向け

データベース全体像

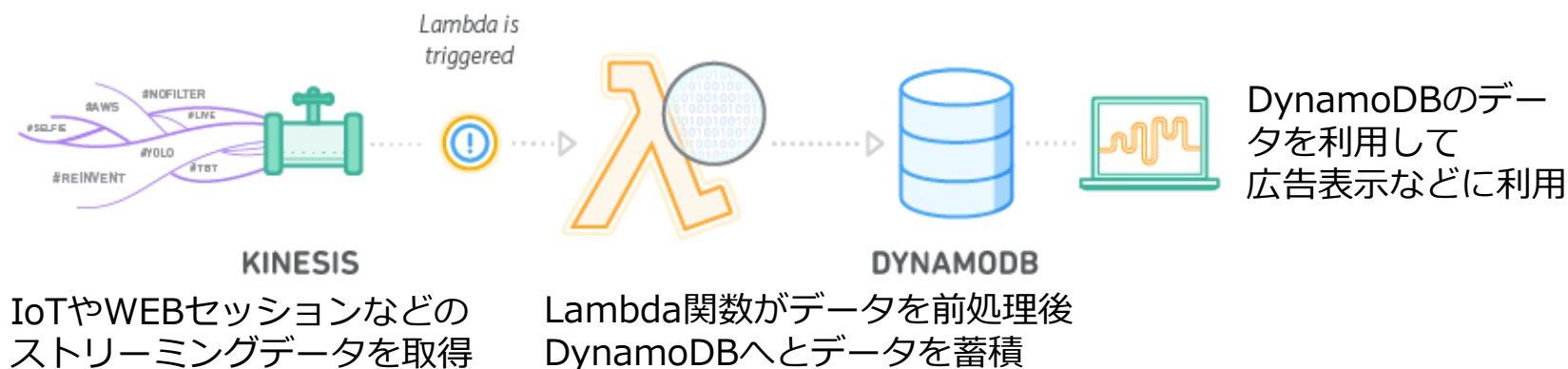


DynamoDBの出題範囲

DynamoDBとは何か？

ストリーミングデータを利用したリアルタイムデータ処理などに最適なデータベースとして利用するNoSQL型データベース

DynamoDBの活用例



Reference: <https://aws.amazon.com/jp/dynamodb/>

DynamoDBの出来る事

キーバリュー（ワイドカラム型）でデータを簡易に操作することができる。

出来る事

- ❑ キーに対するバリュー（値）の CRUD操作
- ❑ 簡易なクエリやオーダー
- ❑ 例えば、数万人以上が同時アクセスして処理が必要になるアプリケーションのセッションデータ処理などが得意

出来ない事／向いていない事

- ❑ JOIN／TRANSACTION／COMMIT／ROLLBACKは不可
- ❑ 詳細なクエリやオーダー（データの検索や結合処理などには向いていない）
- ❑ 大量のデータ読み書きにはコストがかかる

DynamoDBのユースケース

ビッグデータ処理向けか大量データ処理が必要なアプリケーション向けに利用する

ビッグデータ

- ❑ IoTデータなどKey Value型のシーケンシャルなデータを収集・蓄積・分析するのに最適
- ❑ Amazon EMRのHadoop処理と連携してビッグデータ処理が可能

アプリケーション

- ❑ セッションデータやメタデータなどのアプリケーション上でシンプルでデータを蓄積
- ❑ 高パフォーマンスな処理が必要なデータを保存

DynamoDBのユースケース

大量に発生しうるWEB行動データやログデータの保存にはDynamoDBを利用する。

ユーザー行動 データ管理

- ゲームのセッションデータやWEBサイトのユーザー行動データを保存・処理する。
- ユーザー毎の行動履歴管理などに利用する。

バックエンド データ処理

- モバイルアプリのバックエンド／バッチ処理のロック管理／フラッシュマーケティング／ストレージのインデックス

DynamoDBのユースケース

DynamoDBとElastiCacheはNoSQL型であるためユースケースが似ている。

比較項目	RDS	DynamonDB	ElastiCache	Redshift
リレーショナルデータベース	○	×	×	○
データベースキャッシュ	△	○	○	×
メタデータ検索	○	○	○	×
セッションなどの状態管理	△	○	△	×
大容量データ分析	△	×	×	○
リアルタイムデータ分析	△	○	○	×
低レイテンシー	△	○	○	×
モバイルバックエンドデータベース	△	○	△	×

参照 <https://qiita.com/leomaro7/items/e48d9941dab5b5f2a718>

DynamoDBの性能

完全マネージド型のNoSQLデータベースサービスであり、テーブルサイズは無制限だが、1つのデータは400KBに制限

【パフォーマンス】

- ハイスケーラブルで無制限に性能を拡張できる
- 負荷が高くなっても応答速度が低下しない低レイテンシー
- 高可用性（SPOFなしでデータは3箇所のAZに保存）
- マネージド型のためメンテナンスフリー：CloudWatchで運用

【データ容量の制限】

- **ストレージの容量制限がない**
テーブルのサイズには実用的な制限はない。
テーブルは項目数やバイト数について制限がない
- **データ項目には制限あり**
項目のサイズ制限は400 KBであり、大きなデータを格納できない。

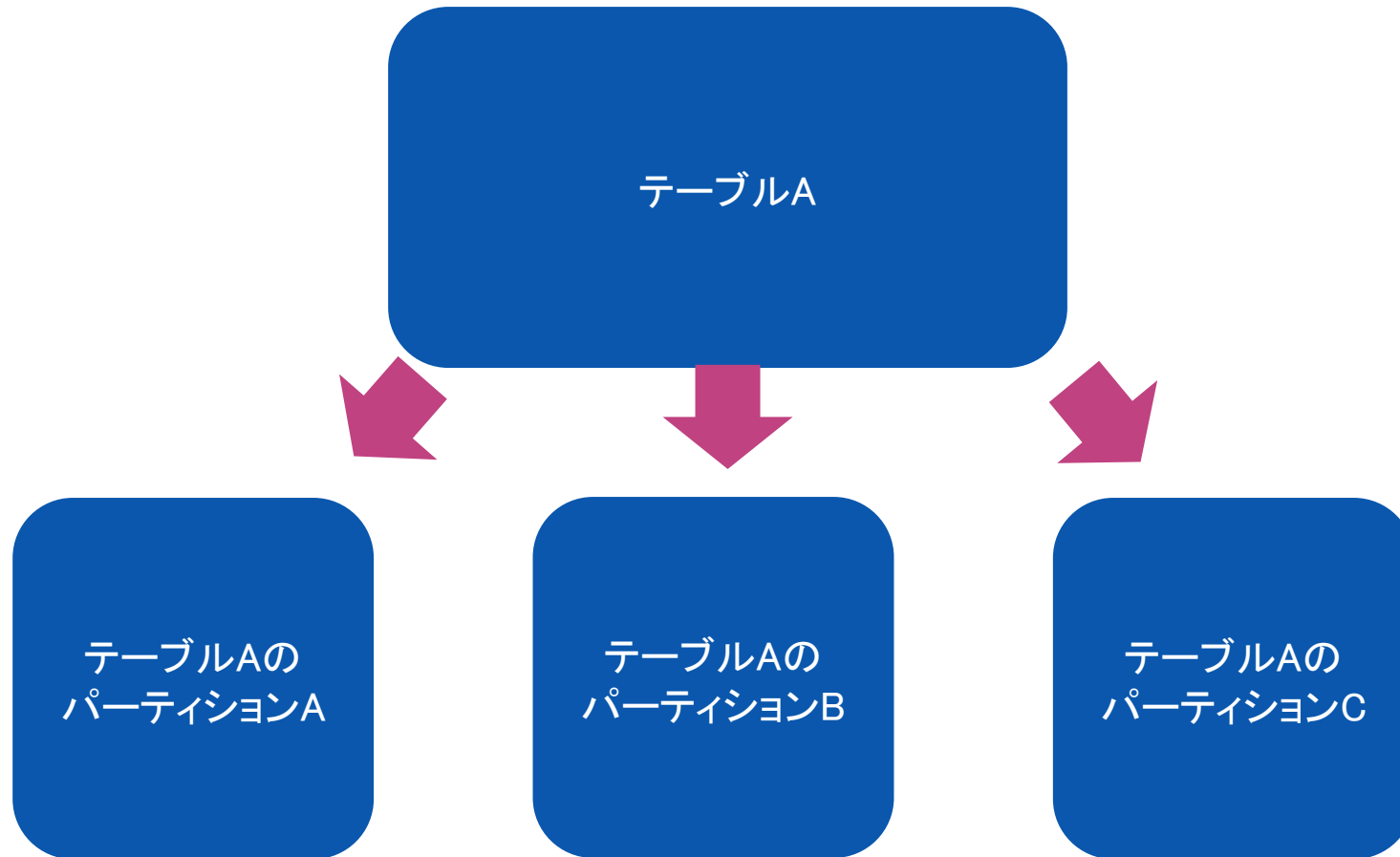
DynamoDBの性能

1桁台のミリ秒レイテンシーを安定して実現しつつ、DAXを利用すればマイクロセカンド単位でのリクエスト処理が可能

DynamoDBテーブル	1 桁のミリ秒レイテンシーを実現
DAX	1 秒あたりのリクエスト数が数百万件になる場合でも、ミリセカンドからマイクロセカンドへ向上

パーティショニング

大量データを高速処理するためにパーティショニングによる分散処理を実施している



DynamoDBの整合性モデル

デフォルトで結果整合性モデルであり、一部処理に強い整合性モデルを利用している

Write

少なくとも2つのAZでの書き込み完了が確認とれた時点で完了

Read

□ デフォルト：結果整合性モデル

最新の書き込み結果が即時読み取り処理に反映されない可能性がある

□ オプション：強い整合性モデル

GetItem／Query／Scanでは強い整合性のある読み込みオプションが指定可能

テーブル設計

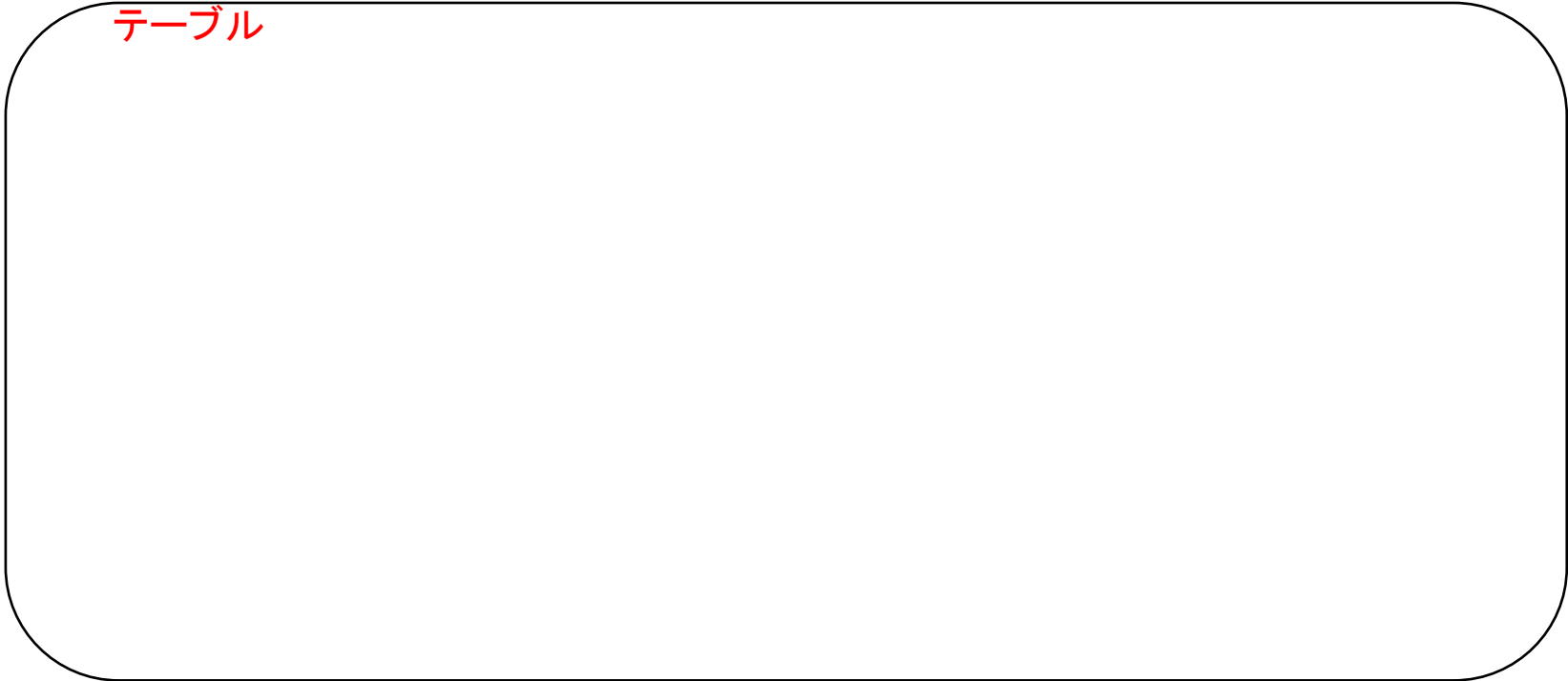
DynamoDBはテーブル単位から利用が開始され、テーブル→項目→属性と設計する

テーブル	DynamoDBはテーブルはデータのコレクションのこと。他のDBと同様にテーブル単位にデータを保存する
項目（アイテム）	各テーブルの中に項目を作ってデータを作成する。項目間で一意に識別可能な属性グループとなる。Personalという項目を作成すれば、名前やIDなどが属性として付属する
属性	各項目は 1 つ以上の属性で構成される。属性はそれ以上分割する必要がない最小のデータ単位。例えばPersonal項目には、姓名といった名前の属性を設定する

テーブル設計

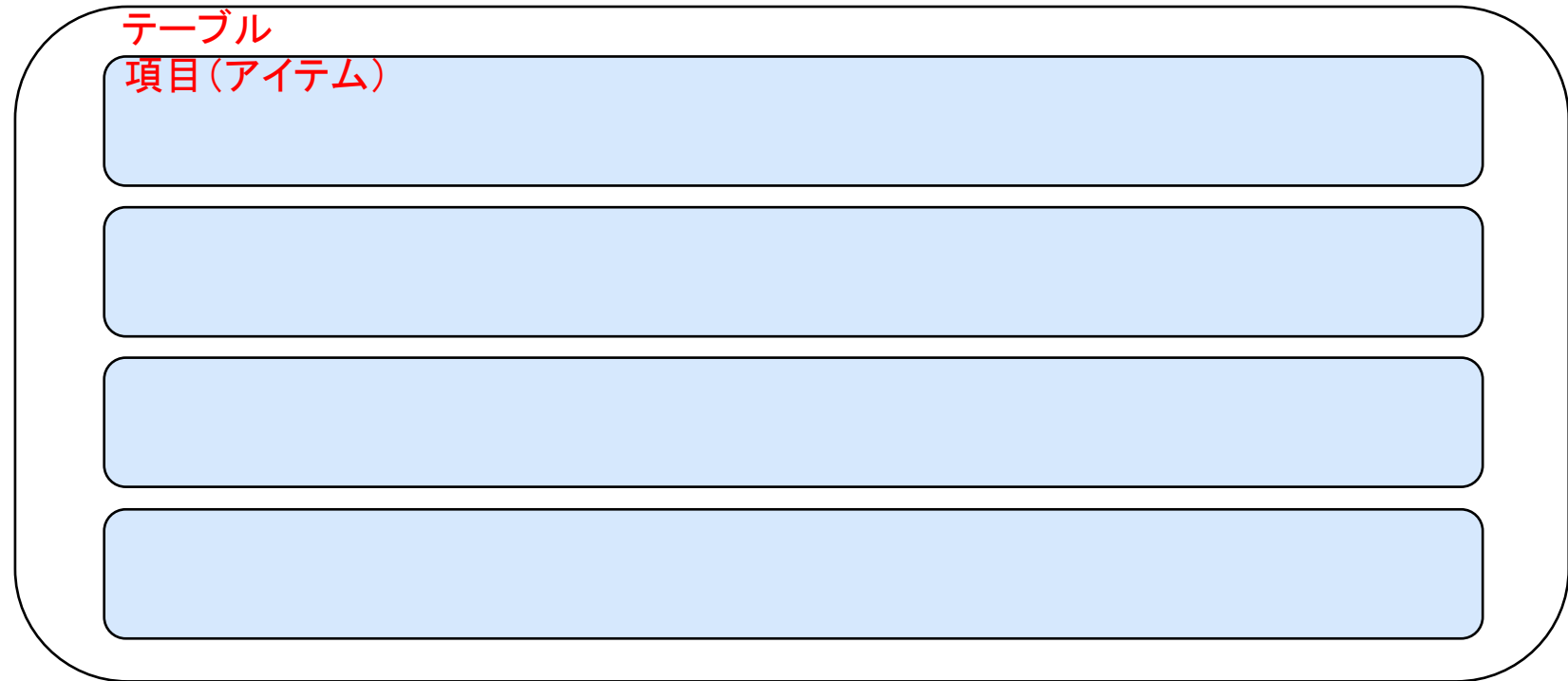
テーブルと項目と属性の関係性を入れ子状にしてテーブルを設計する

テーブル



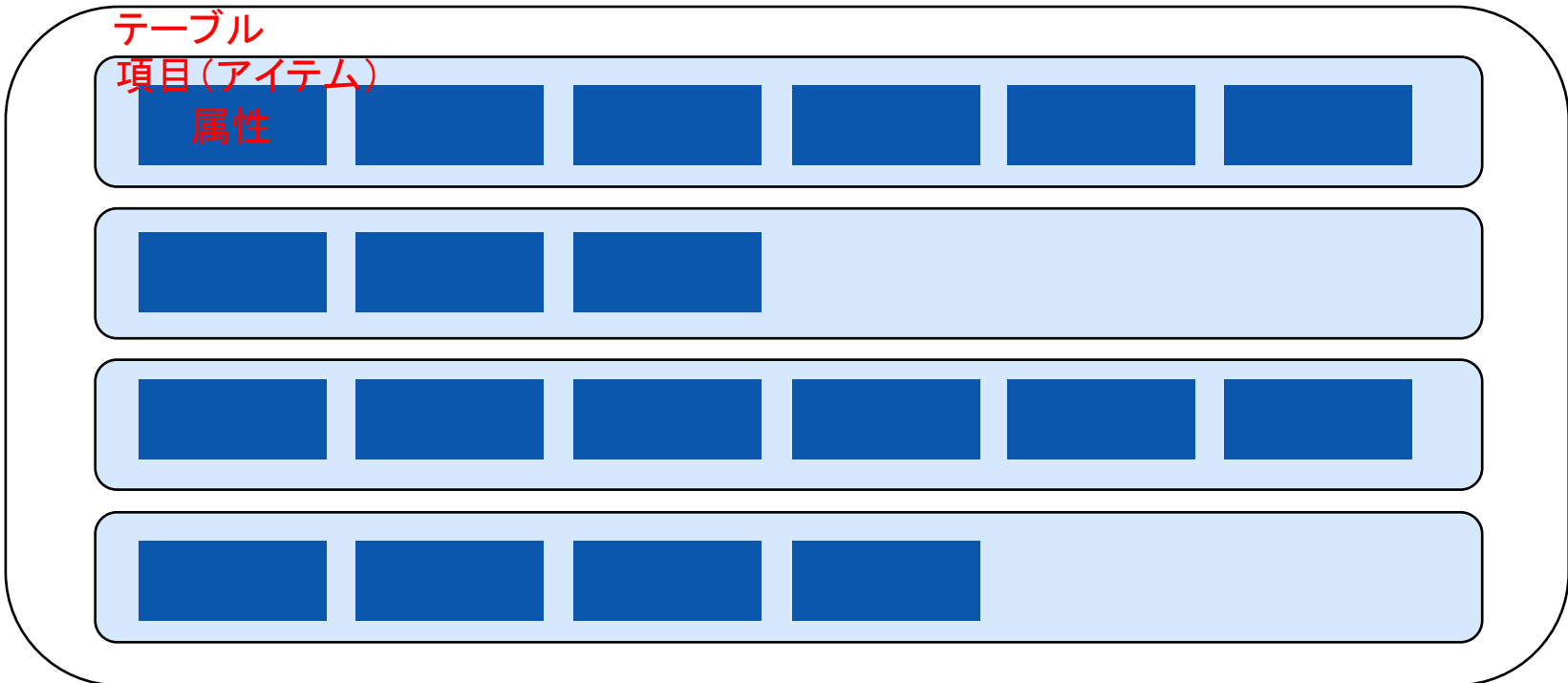
テーブル設計

テーブルと項目と属性の関係性を入れ子状にしてテーブルを設計する



テーブル設計

テーブルと項目と属性の関係性を入れ子状にしてテーブルを設計する



属性はVALUE型やJSON型など不ぞろいであっても構わない

キャパシティモードの設定

利用するキャパシティが予測できるか否かでモードを選択

オンデマンドモード

- 利用するキャパシティが予測できないときに選択するモード
- トラフィック量の予測が困難な場合にリクエストの実績数に応じて課金
- オンデマンドでRead/Write処理に自動スケーリングを実施

プロビジョニング モード

- 利用するキャパシティが事前予測できるときに選択するモード
- 事前に予測した書き込みキャパシティユニット（WCU）と読み込みキャパシティユニット（RCU）を設定する。
- 設定したキャパシティに基づいて課金
- UpdateTable オペレーションを使用して、必要な回数だけ ReadCapacityUnits または WriteCapacityUnits を増やすことができる。
- キャパシティ容量に近づくとHTTP 400コード（不正なリクエスト）とProvisionedThroughputExceededExceptionが発せられる。

24 時間ごとに 1 回、読み込み/書き込みキャパシティーモードを切り替えることができる。

DynamoDBの料金

キャパシティ設定の方式と利用する機能に応じて課金される。

オンデマンド	□ ストレージ容量 (GB単位) □ 書き込み単位 □ 読み込み単位
プロビジョンド	□ ストレージ容量 (GB単位) □ 読み込みキャパシティユニット (RCU) □ 書き込みキャパシティユニット (WCU)
その他	□ グローバルテーブル：レプリケート書き込みキャパシティユニット (rWCU) □ DynamoDB Accelerator (DAX)：ノード時間単位 □ DynamoDB ストリーム：ストリーム読み込みリクエスト単位

インデックス

DynamoDBは暗黙的に設定するKVSにおけるKeyに値するものと、明示的に設定するキーがインデックスとして利用できる

暗黙的なキー

データを一意に特定するために暗黙的にキー（ハッシュキーやレンジキー）として宣言して検索に利用するインデックスで、1テーブルに1つ宣言する

明示的なキー

ローカル・セカンダリ・インデックス（LSI）はプライマリキーのタイプがハッシュキーやレンジキーの場合に追加で別のレンジキーを増やすように利用できる

1テーブルに5つ作成可能／テーブル作成時に作成

グローバル・セカンダリ・インデックス（GSI）は別のハッシュキーを設定することができる。全データに対してグローバルに検索を実施する。

1テーブルに5つ作成可能／テーブル作成後に作成

プライマリーキー

DynamoDBはハッシュキーとレンジキーという2種類のプライマリーキーを利用する

ハッシュキー

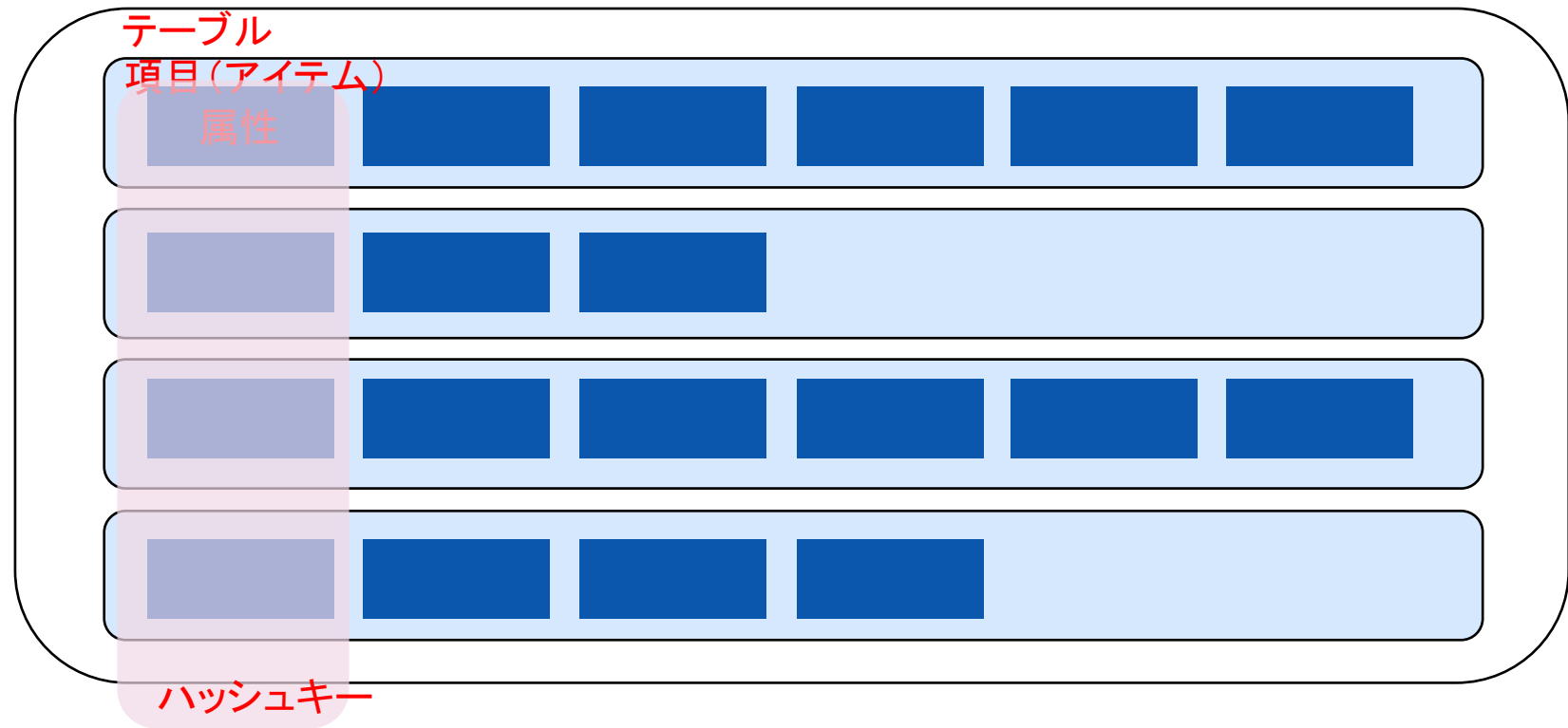
- ❑ KVSにおけるキーに相当するデータを一意に特定するためのIDなどのこと
- ❑ テーブル作成時に1つの属性を選び、ハッシュキーとして宣言
- ❑ ハッシュ関数によってパーティションを決定するためハッシュキーと呼ばれる
- ❑ ハッシュキーは単独での重複を許さない

レンジキー

- ❑ ハッシュキーにレンジを加えたものをレンジキーまたは複合キーと呼ぶ
- ❑ テーブル作成時に2つの属性を選び、1つをハッシュキーとして、もう一つをレンジキーと呼ばれるキーとして宣言
- ❑ 2つの値の組み合わせによって、1つの項目を特定
- ❑ 複合キーは、単独であれば重複が許される

プライマリキー

テーブルと項目と属性の関係性を入れ子状にしてテーブルを設計する



プライマリキー

テーブルと項目と属性の関係性を入れ子状にしてテーブルを設計する



セカンダリインデックス

ハッシュキーやレンジキーだけでは検索要件が満たせない場合にLSIとGSIを追加する。

Local Secondary Index (LSI)

- ソートキー以外にインデックスを作成できる検索方式。
- 複合キーテーブルにのみ設定可
- 複合キーによって整理されている項目に対して、パーティションキーを指定した上で、別の規則のインデックスとなりクエリ検索に利用できる。

Global Secondary Index (GSI)

- GSIはインデックス用に新たにパーティションキーとソートキーを指定する検索方式。
- ハッシュキーテーブル及び複合キーテーブルどちらにでも設定可能
- ハッシュキーの代わりになるため、ハッシュキーをまたいで物理パーティションに囚われない検索が可能

スループットやストレージ容量を追加で必要で書込みも増大するため、多様すべきではない。

テーブル操作

テーブル操作としては以下のようなコマンドを利用する

GetItem ハッシュキーを条件に一定の項目 (アイテム) を取得	Query ハッシュキーとレンジキーにマッ チする項目を取得 (最大 1 MB)
PutItem 1件のアイテムを書き込む	Scan テーブルを全件検索する (最大 1 MB)
Update 1件のアイテムを更新	BatchGetitem 複数のプライマリーキーに対して マッチする項目を取得
Delete 1件のアイテムを削除	

DynamoDBストリーム

DynamoDB テーブルに保存された項目の追加・変更・削除の発生時の履歴をキャプチャできる機能

データの保存

- ❑ 過去24時間以内のデータ変更の履歴を保存し、24時間を経過すると消去される
- ❑ データ容量はマネージド型で自動的に管理

データ保存の順番

- ❑ 操作が実施された順番に応じてデータはシリアライズされる
- ❑ 特定のハッシュキーに基づいた変更は正しい順番で保存されるが、ハッシュキーが異なる場合は受信した順番が前後される可能性がある

DynamoDBストリームのユースケース

データ更新をトリガーとして処理を実行するアプリケーション機能や、DynamoDBテーブルのレプリケーションに活用できる

クロスリージョンレプリケーション

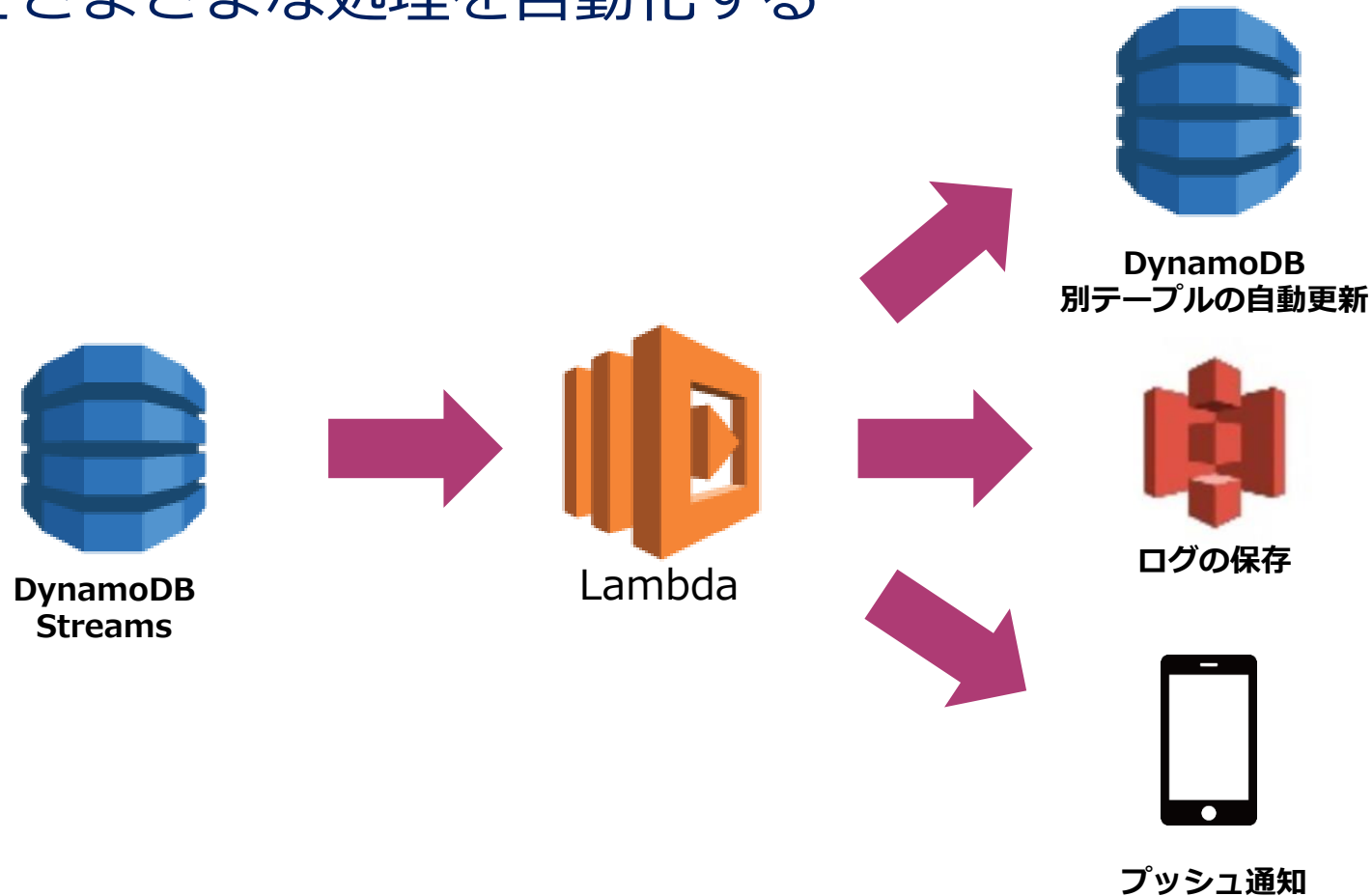
- ストリームによるキャプションをトリガーとしてクロスリージョンレプリケーションを実施することが可能

データ更新をトリガーとしたアプリケーション機能

- データ更新に応じた通知処理などのアプリケーション処理の実行 など

DynamoDBストリームのユースケース

DynamoDBの書き込み処理をトリガーにしてLambda関数によってさまざまな処理を自動化する



DynamoDB Auto Scaling

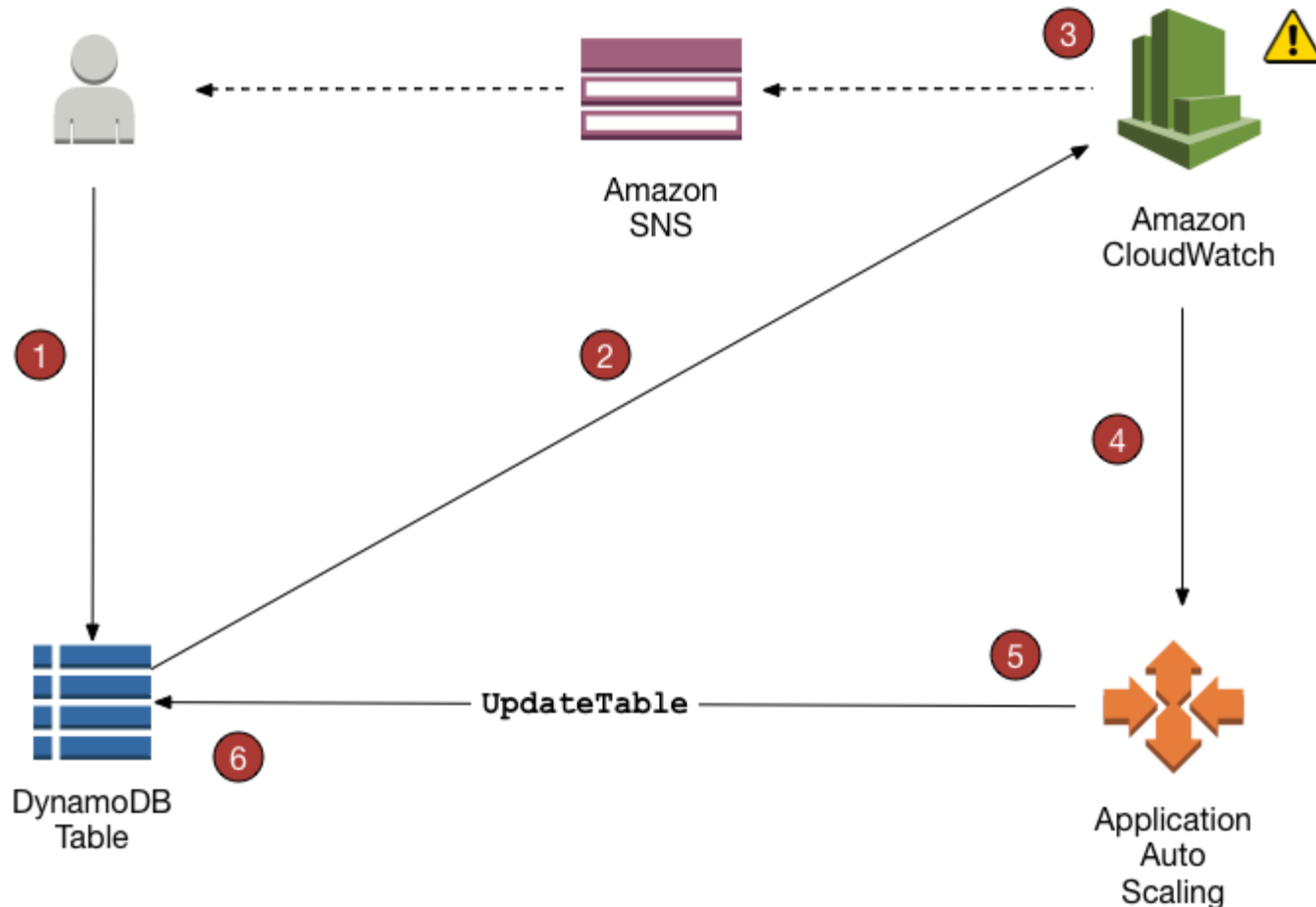
スケーリングポリシーに基づいてテーブルまたはGSIを自動でスケーリングする。

DynamoDB Auto Scaling

- ❑ AWS Application Auto Scaling サービスを使用して Application Auto Scaling ポリシーを設定する。
- ❑ CloudWatchのモニタリングに基づいてトラフィックパターンに応じてプロビジョンドスループット性能をユーザーに代わって動的に調節する。
- ❑ テーブルまたは グローバルセカンダリインデックスはプロビジョニングされた読み込みおよび書き込みキャパシティーを増やし、急激なトラフィック増加をスロットリングなしに処理できる。
- ❑ DynamoDB テーブルを作成すると、Auto Scaling がデフォルトで有効化されている。

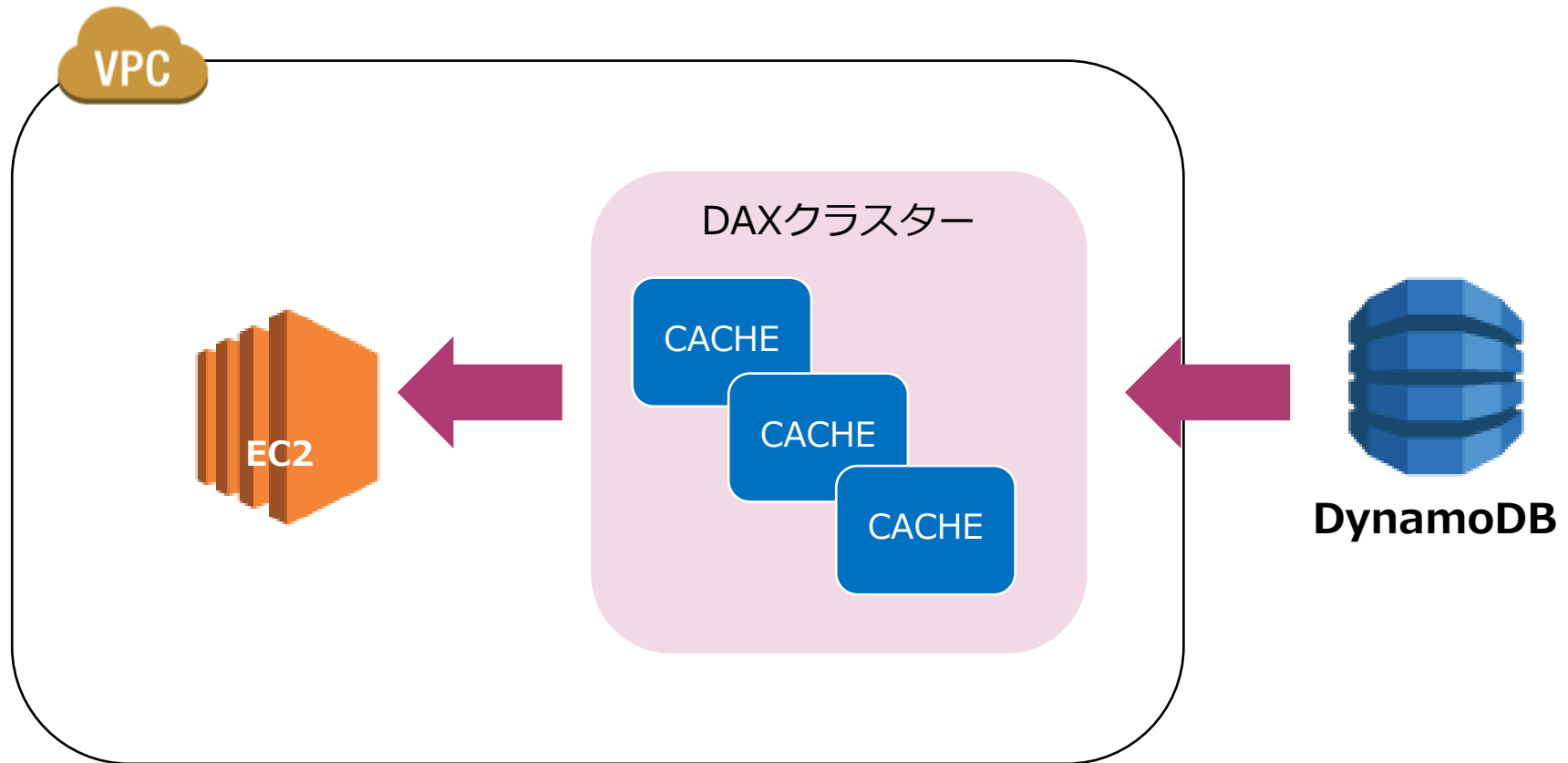
DynamoDBのスケーリング

スケールリングポリシーに基づいてテーブルまたはGSIを自動でスケールリングする。



DynamoDB Accelerator (DAX)

DAXはDynamoDBにインメモリキャッシュ型の機能を付加する



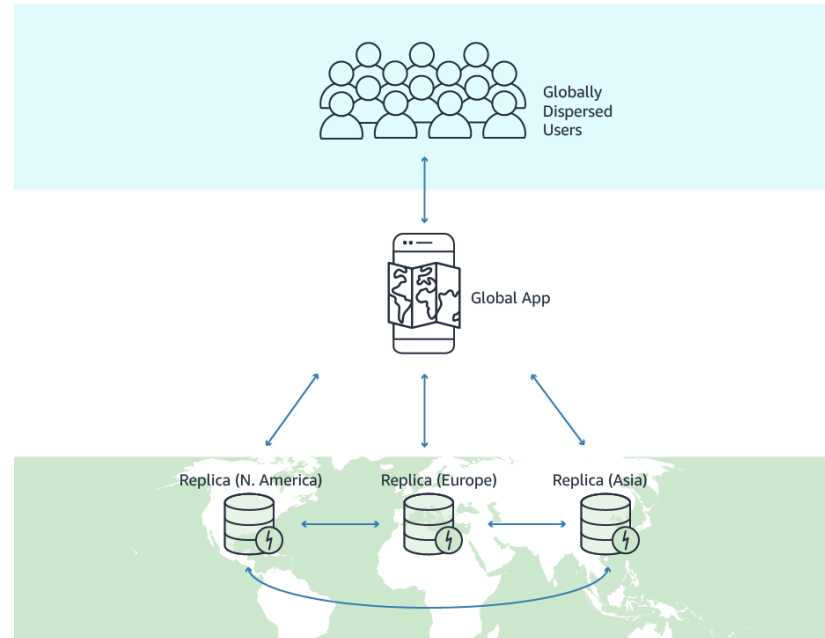
DynamoDB Accelerator (DAX)

DynamoDBにおいて高速なインメモリパフォーマンスを可能に

- インメモリキャッシュとして 1桁台のミリ秒単位からマイクロ秒単位まで結果整合性のある読み込みワークロードの応答時間を短縮。マルチAZ DAXクラスターは、1秒間に数百万件のリクエストを処理できる。
- DAXはDynamoDBを使用するAPIと互換性を持つマネージド型サービスであり、運用上そしてアプリケーションの複雑性を減少させて容易に導入可能
- 読み取りの多いワークロードや急激に増大するワークロードに対して、DAXはスループットを強化したり、読み込みキャパシティユニットを必要以上にプロビジョニングしないよう設計することで運用コストの節約できる

グローバルテーブル

リージョン間で同期されるマルチマスターテーブルを作成可能



- ❑ DynamoDBの性能のまま、世界中で複数のリージョンにエンドポイントを持つことができる
- ❑ 読み書きのキャパシティに加えて、クロスリージョンレプリケーションのデータ転送料金に課金される。
- ❑ オプションで実施できた強い整合性は使用できない。

バックアップとリカバリ

パフォーマンスに影響なく数百TBのバックアップを実行可能

□ オンデマンドバックアップ

- 任意のタイミングでテーブルの完全なバックアップを作成する。
- 長期間の保存とアーカイブを実施

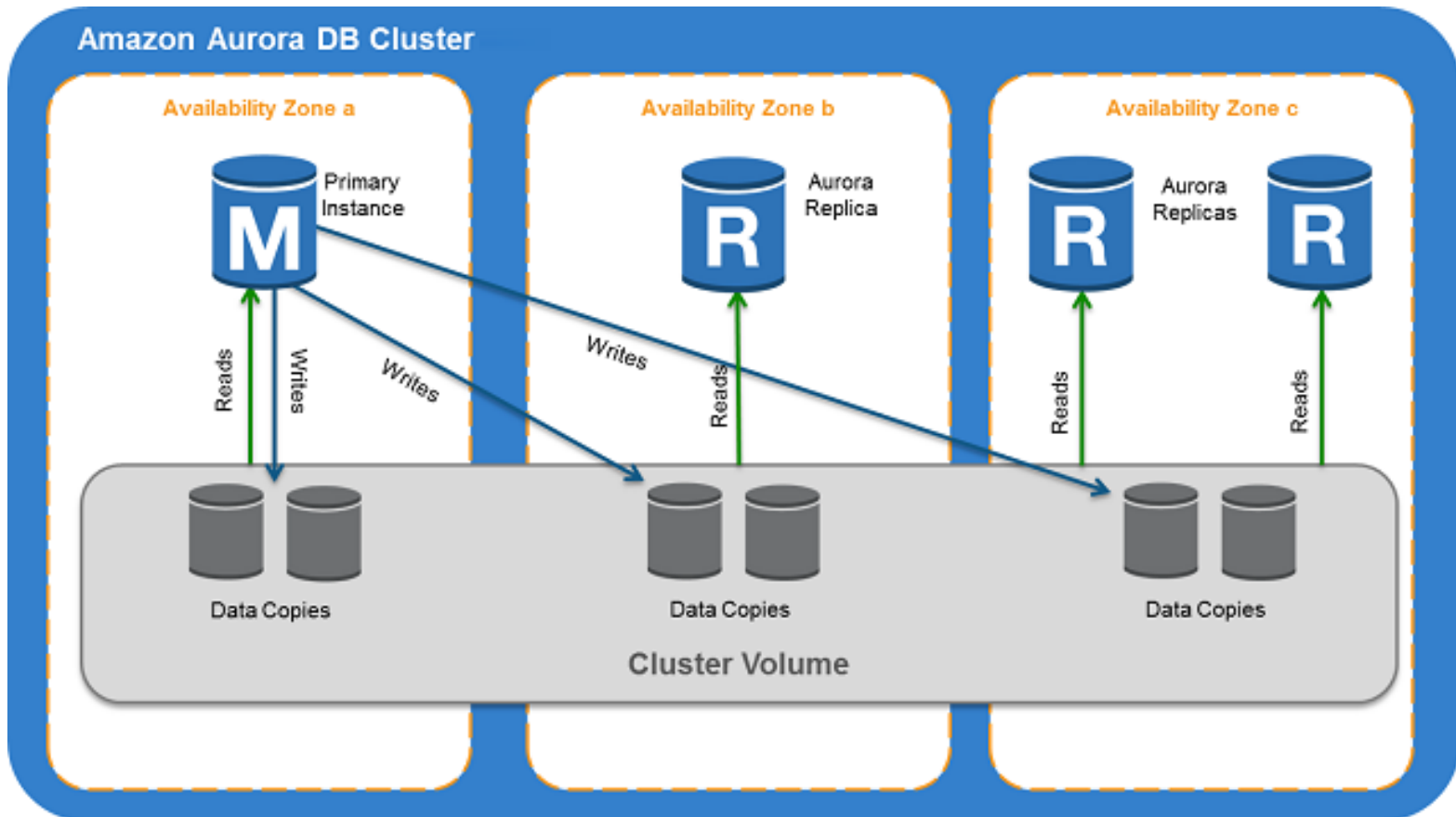
□ ポイントインタイムリカバリ

- 連続バックアップを有効化して、バックアップを継続的に実施する。そのため、任意のタイミングでリカバリできる。
- 過去 35 日間の任意の時点にテーブルを復元することができる。

Auroraの概要

Auroraとは何か？

マルチAZで分散されたクラスター構成により、高速・高性能なリレーショナルデータベース



Aurora

クラウド時代の新しい分散型のリレーショナルデータベースとして誕生

- Amazonがクラウド時代に適したリレーショナルDBを一から考えて構築された新RDB
- その特徴はNoSQL型の分散高速処理とRDBとしてのデータ操作性を両立させたこと

Aurora

MySQLと2.5～5倍の性能を商用データベースの10分の1の価格で提供する高性能・低コストDB

性能5倍

Sysbench4インスタンスと
r3.8xlargeAuroraとの比較

性能約2.5～5倍

TPC-Cをr3.8xlargeのAuroraとの比較

Aurora

RDSにおいてデータベースソフトウェアの一つとして選択

エンジンのオプション

☒ Amazon Aurora

Amazon
Aurora

☐ MySQL



☐ MariaDB



☐ PostgreSQL



☐ Oracle

ORACLE®

☐ Microsoft SQL Server



Auroraの特徴

高い並列処理性能によって大量の読み書きをするのに適したDB

- 高い並列処理によるストレージアクセスによってクエリを高速処理することが可能
- Auroraは大量の書き込みや読み込みを同時に扱うことができる
- データベースの集約やスループット向上が見込まれる
- ただし、すべてが5倍高速というわけではなく、適用すべき領域を見つけて利用する

Auroraの特徴

MySQL／PostgreSQLと互換性があり、同じ操作方法とそのコミュニティを利用することが可能

**MySQL互換
を選択可能**

**PostgreSQL互換
を選択可能**

Auroraの特徴

分散型で耐障害性と自己回復性を備えたスケーラブルな新しいタイプのフルマネージド型RDB

耐障害性／自己回復性

- ❑ 3つのAZに2つのコピーを設置可能で合計6つのコピーを保持
- ❑ 過去のデータがそのままS3に継続的バックアップ
- ❑ リストアも差分適用がなく高速
- ❑ どのタイミングでも安定したリストア時間を実現
- ❑ 99.99%の高可用性・高耐久性

スケーラビリティ

- ❑ 10GBから最大64TBを提供するSSDデータプレーンを利用してシームレスに拡張可能
- ❑ Auto-Scalingなどのクラウド独自のスケーラブルが可能
- ❑ 最大15のリードレプリカを利用した高速読み込みが可能

Auroraのユースケース

大規模なクエリ処理が発生するRDB環境などはAuroraへの移行を検討すべし

大規模なクエリ データ処理

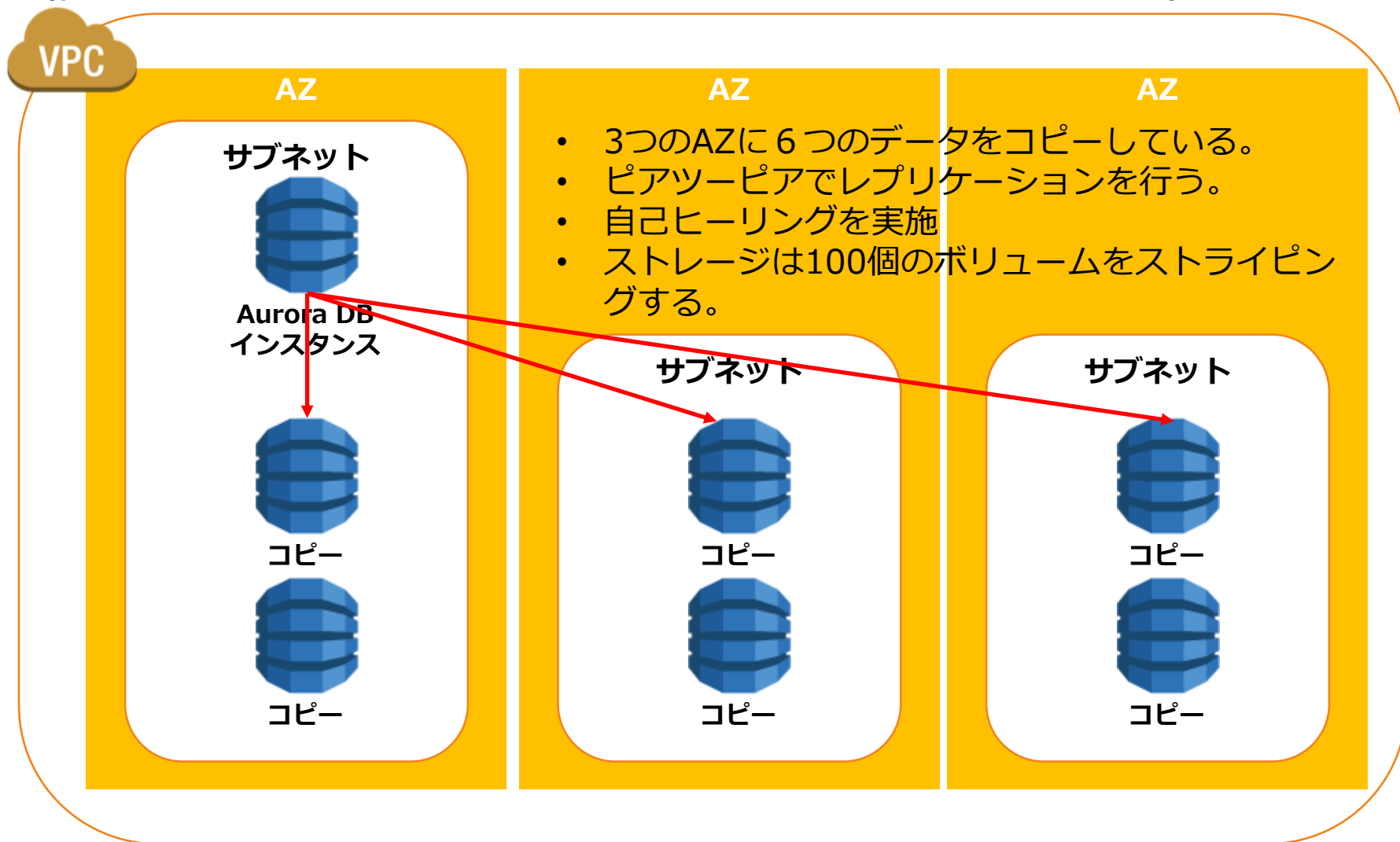
- ❑ 書込み量が多くでトランザクション量が多い
- ❑ クエリ並行度が高い、データサイズが大きいケースで効果を発揮する
- ❑ コネクション数やテーブル数が多いデータベース処理

運用の容易さ を活用する

- ❑ スケーラビリティの高さやデータ容量が無制限に拡張できる
- ❑ レプリケーションなどの性能の高さ

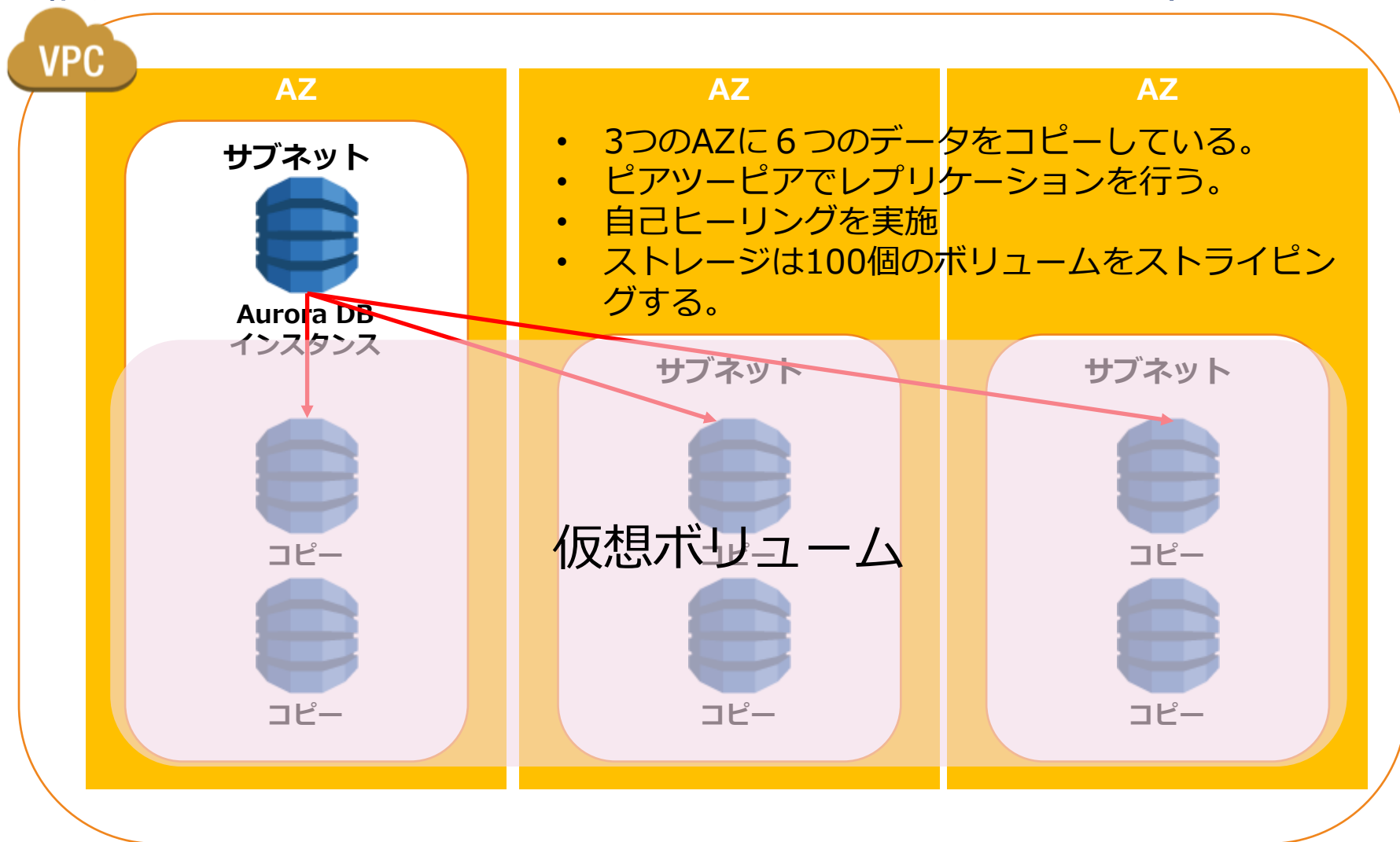
DBクラスタの仮想ボリューム

Auroraは1つのDBインスタンスと1つのDBクラスタボリュームで構成される。3つのAZにコピーされたクラスタを単一と認識



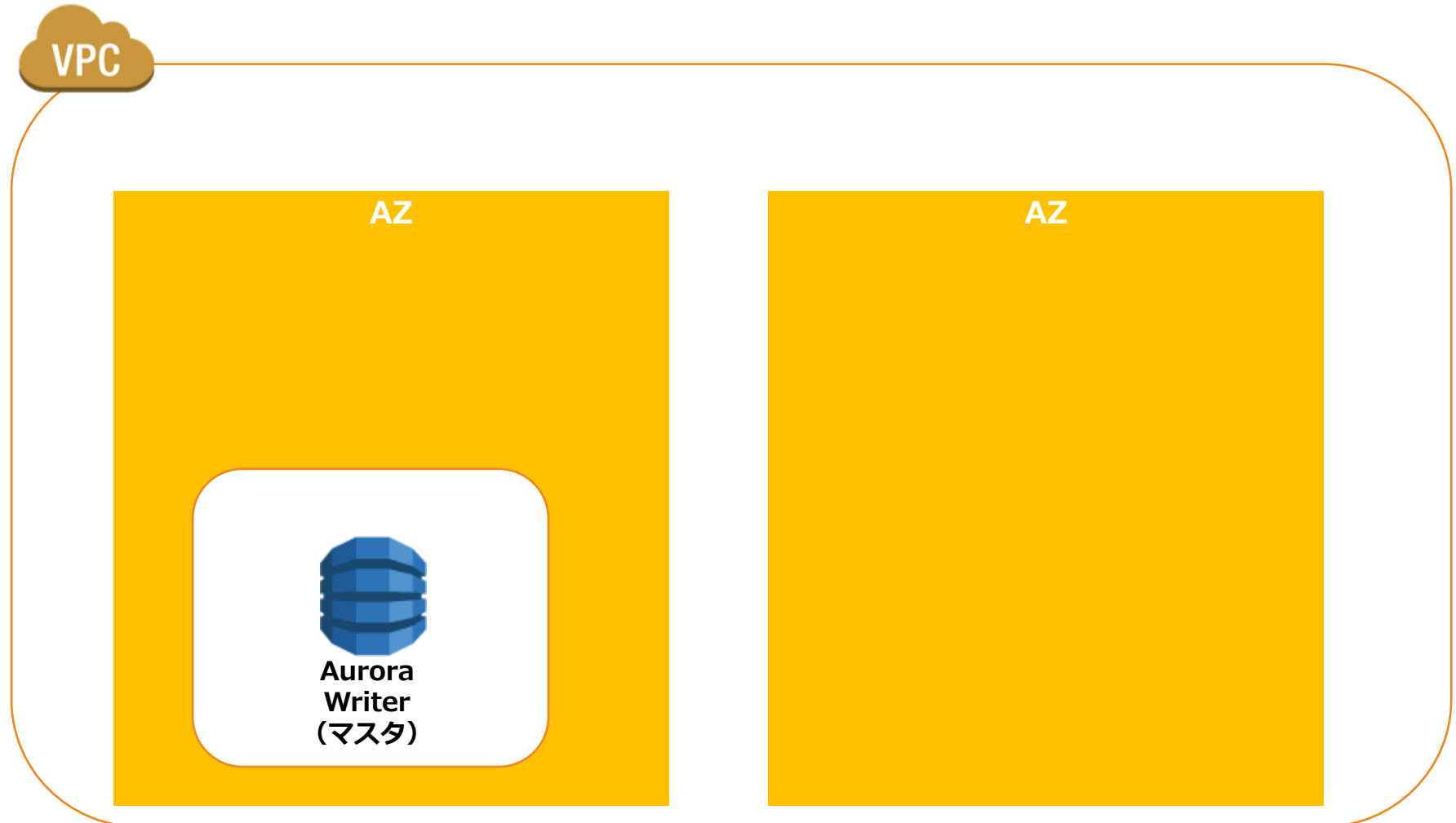
DBクラスタの仮想ボリューム

Auroraは1つのDBインスタンスと1つのDBクラスタボリュームで構成される。3つのAZにコピーされたクラスタを単一と認識



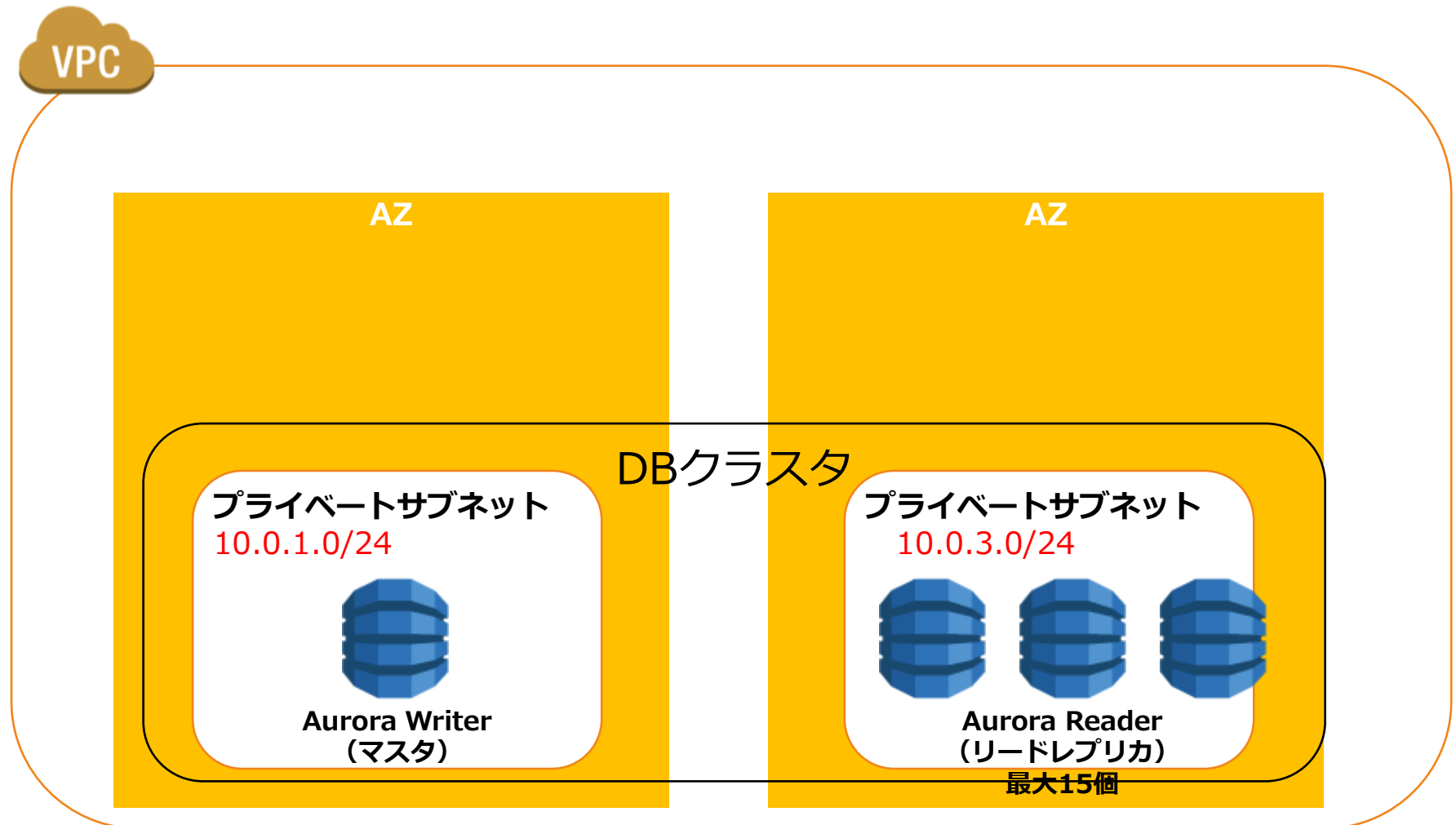
DBクラスタ構成

クラスタ構成を割愛してDBインスタンスのみで次を説明する



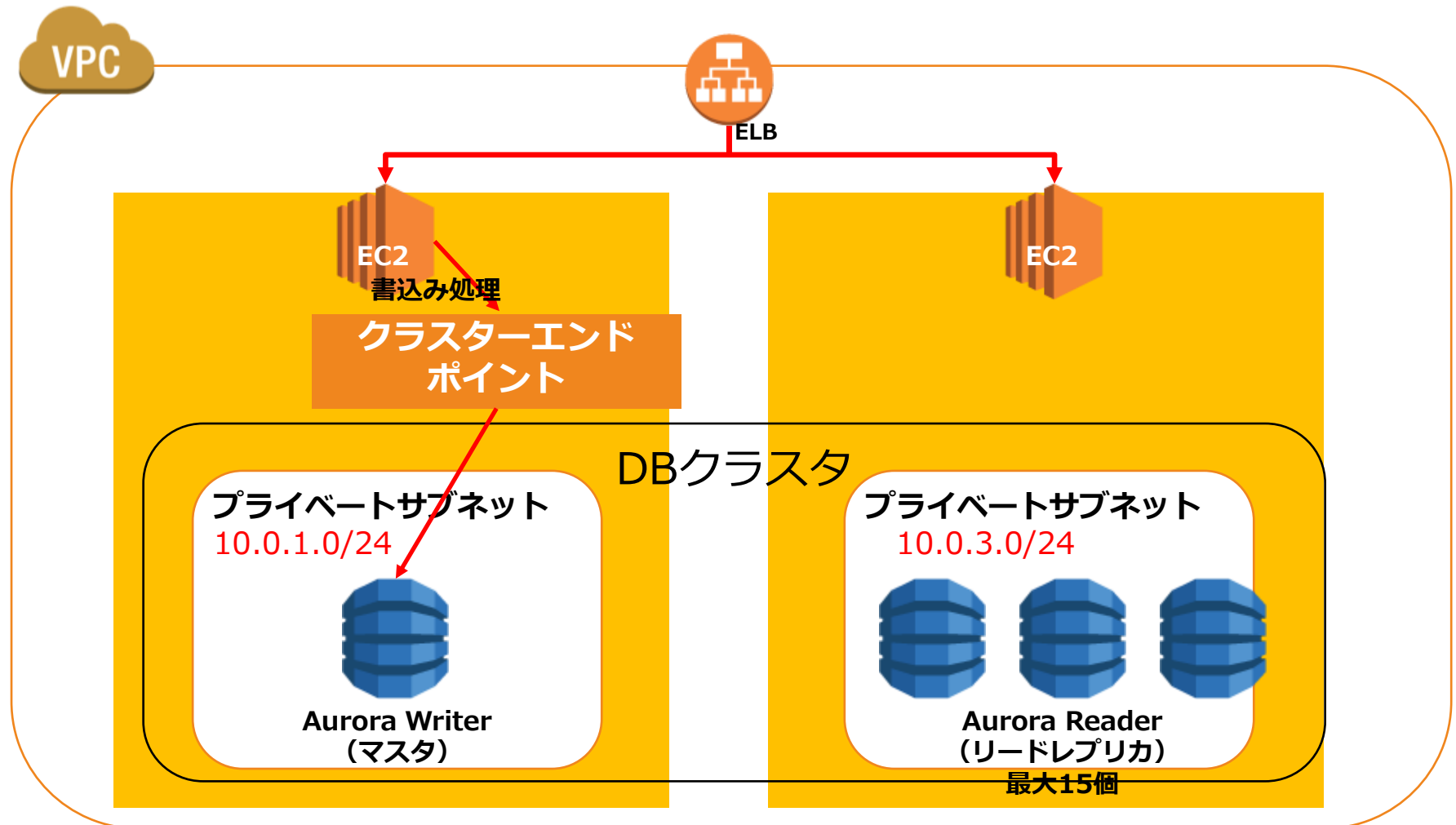
DBクラスタ構成

AuroraはマスタとリードレプリカをまとめたDBクラスタを構成



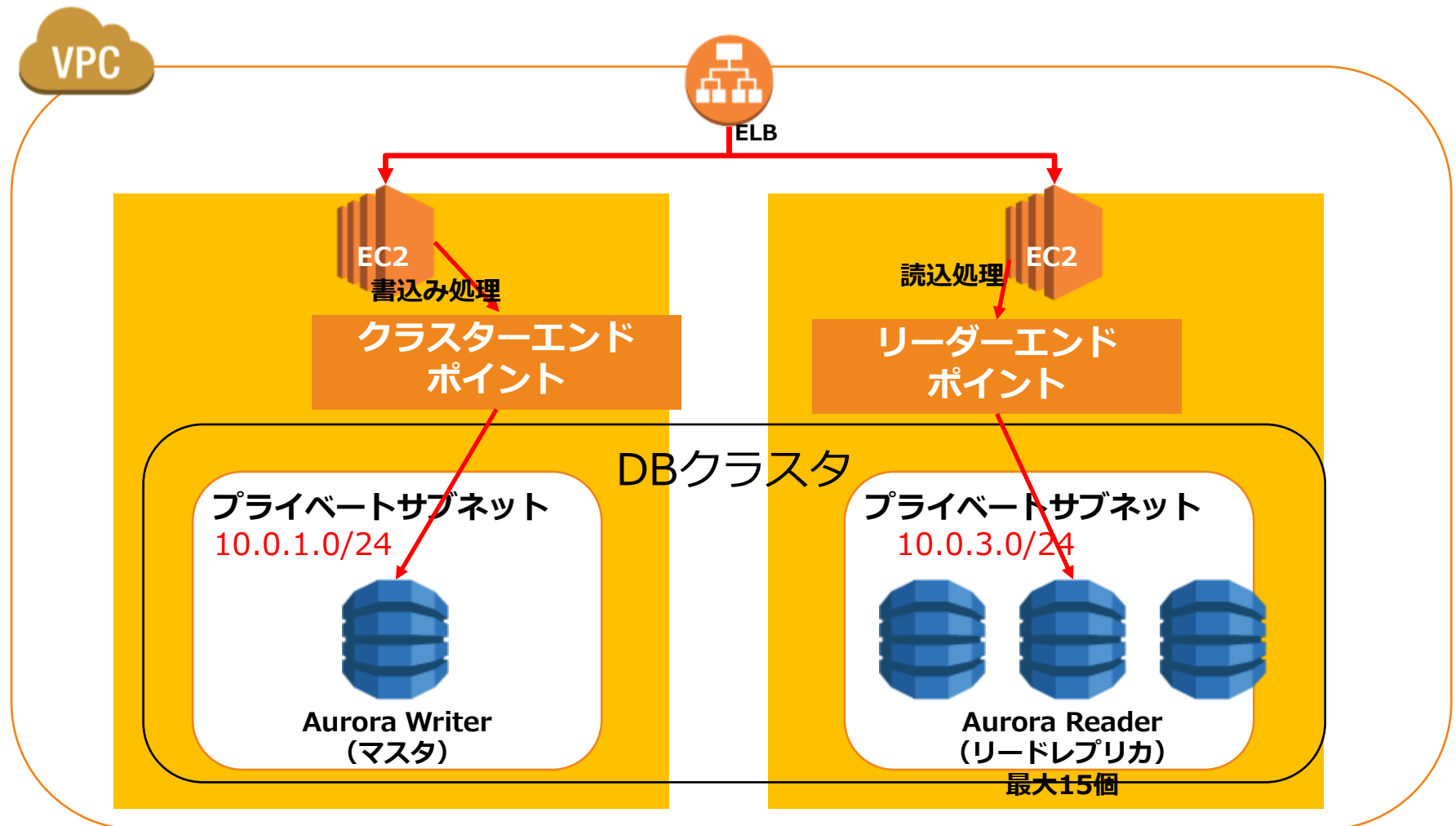
DBクラスタ構成

書込み処理はエンドポイントからWriterが指定される



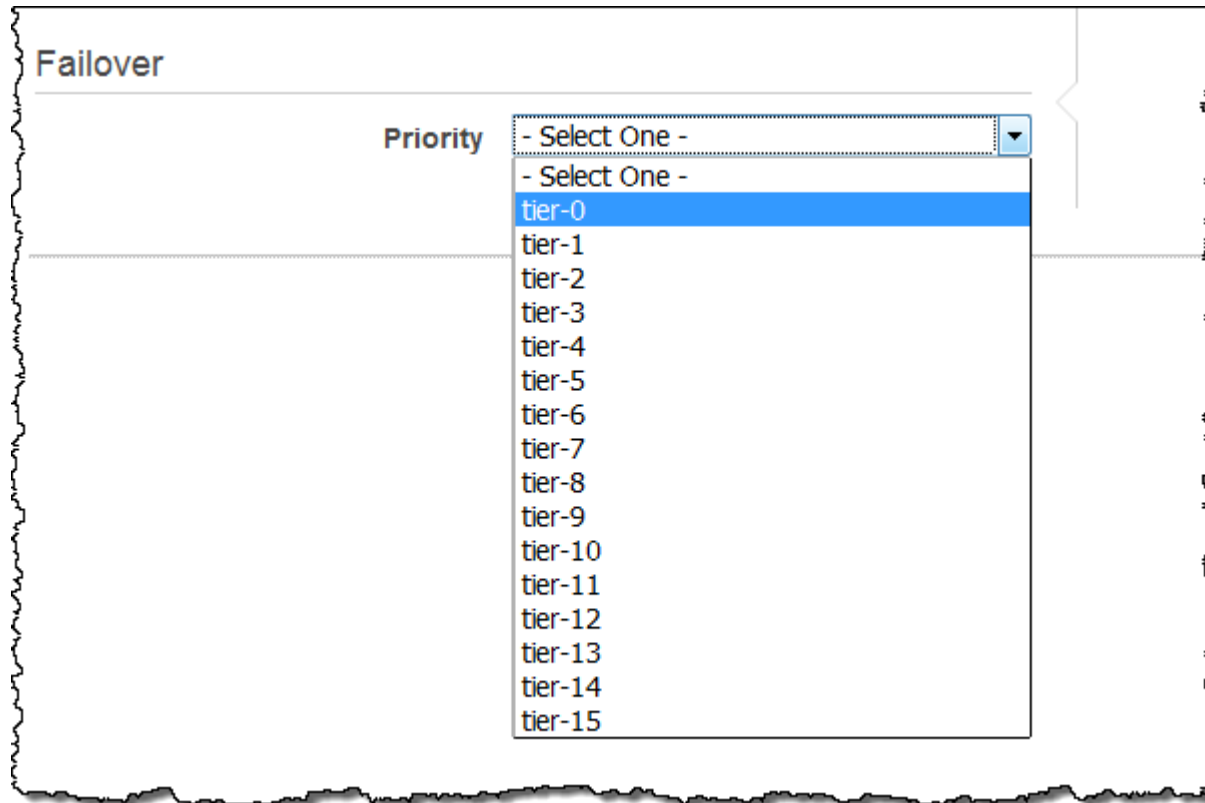
DBクラスタ構成

読み込み処理はリーダーエンドポイントからレプリカへのアクセスが実施される。



フェールオーバー構成

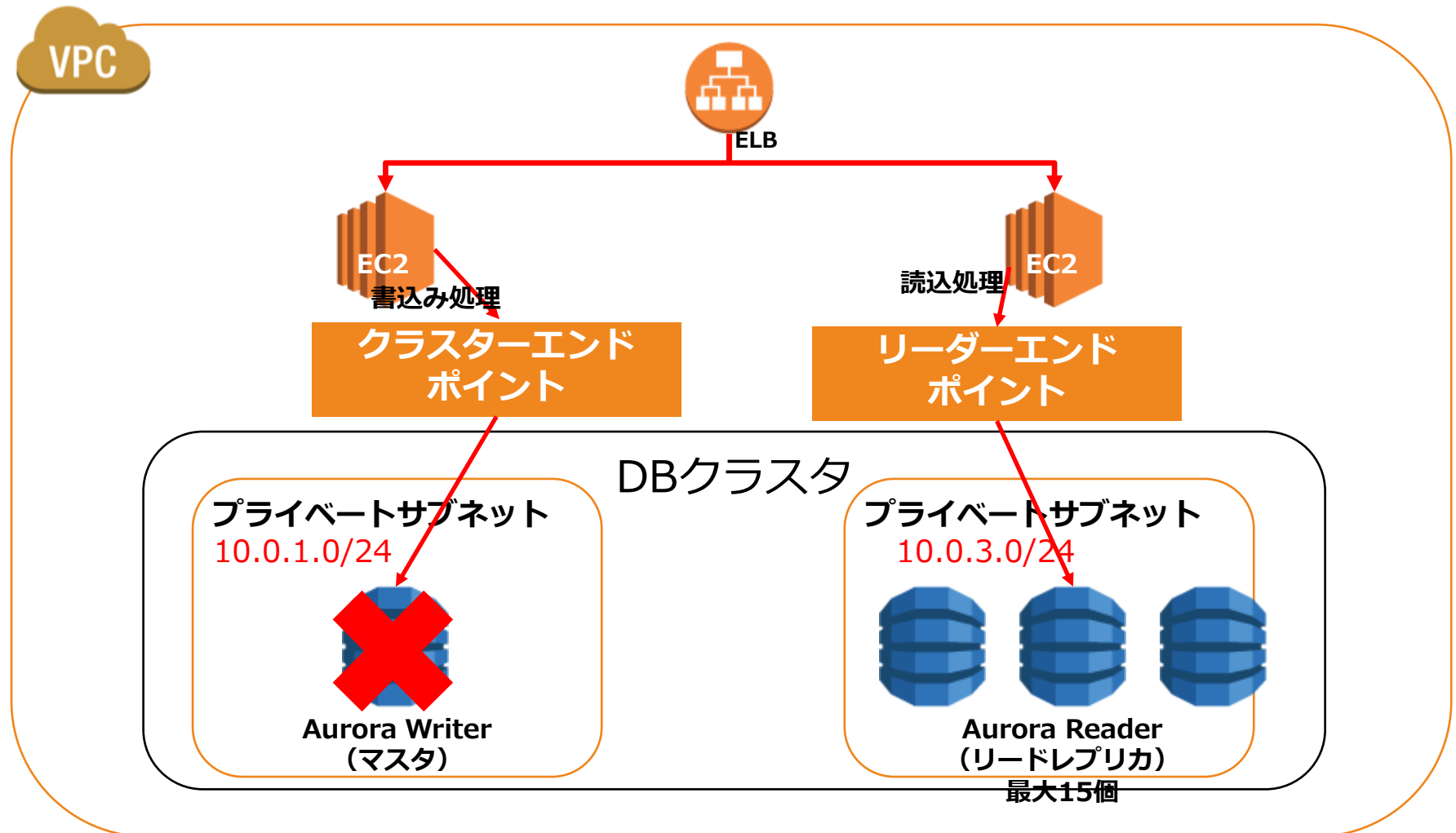
Auroraはティア番号が小さい最大サイズが多しい順番にリードレプリカをプロモートして、フェールオーバーを実施する。



1. Amazon Auroraは、優先度が高い（番号が最も小さいティア）リードレプリカからプロモートする。
2. 2つ以上のAuroraレプリカが同じ優先度を共有している場合、最大サイズのレプリカをプロモートする。

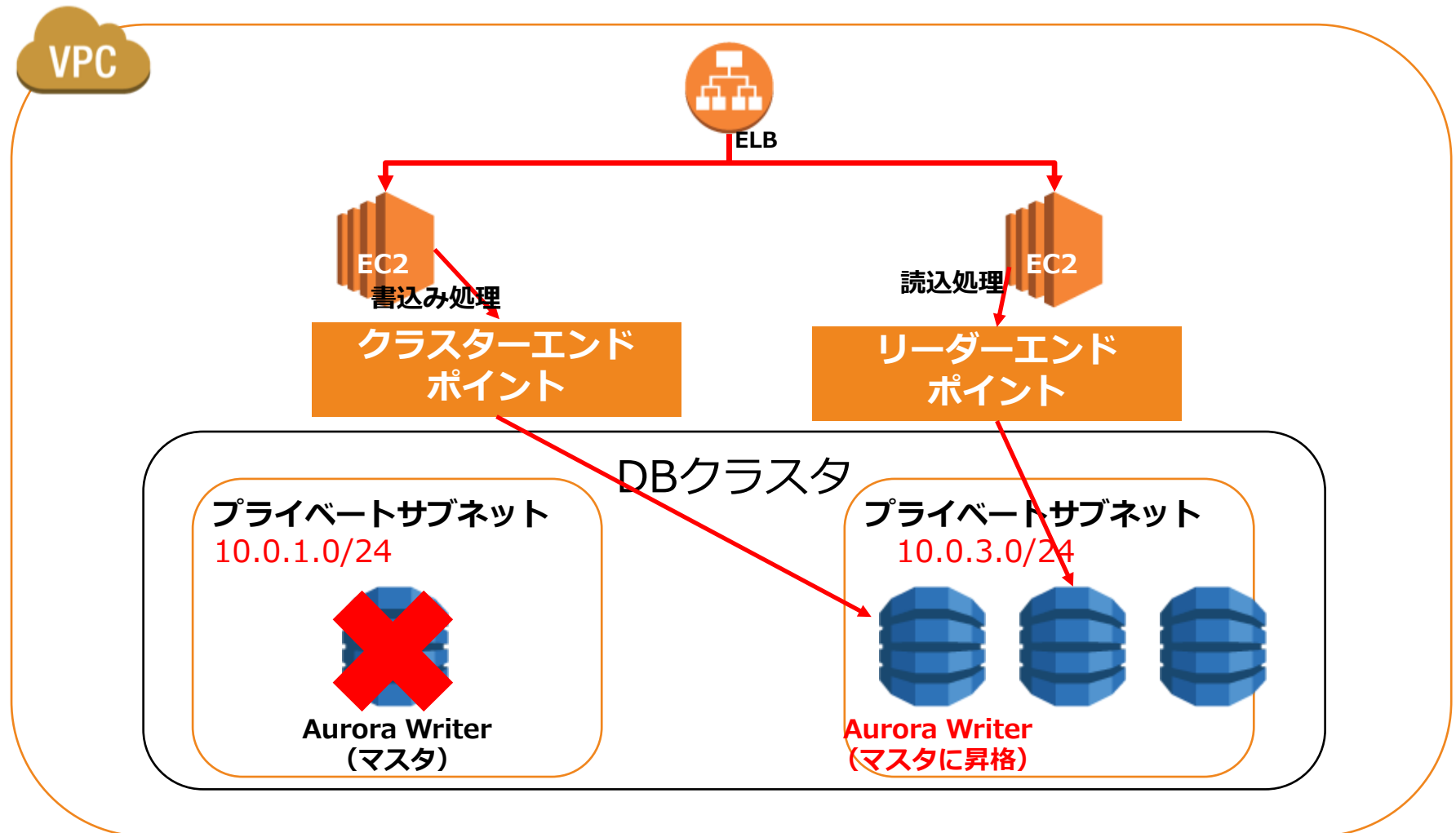
フェールオーバーの実行

マスタに障害が発生するとリードレプリカがプロモーションしてフェイルオーバーを行う。



フェールオーバーの実行

マスタに障害が発生するとリードレプリカがプロモーションしてフェールオーバーを行う。



マイグレーション

MySQLとPostgreSQLのスナップショットからAuroraへとマイグレーションが可能

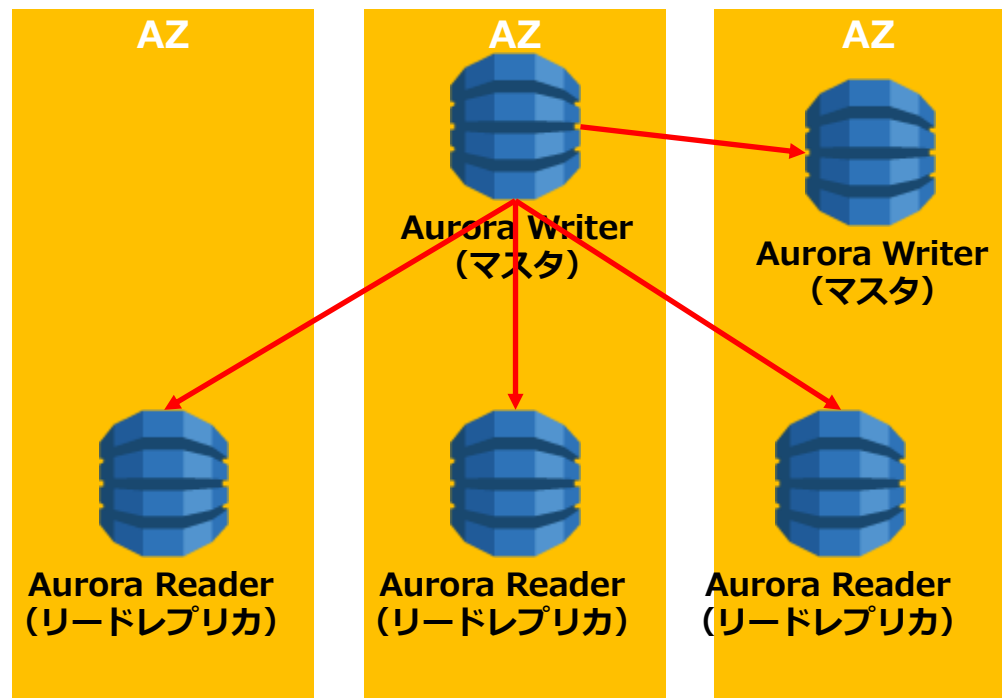


Auroraマルチマスター

マスターデータベースを複数構築してWrite性能もスケラブルに構築可能

VPC

マルチマスター構成

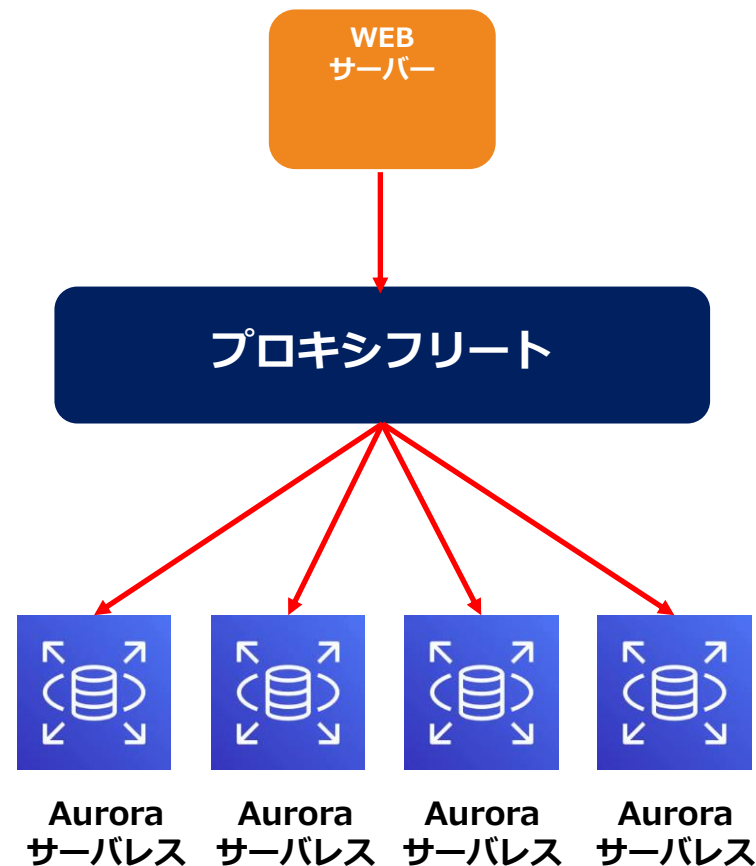


- ❑ どのノードが落ちてもダウンタイムがゼロに
- ❑ どのAZが落ちてもダウンタイムがゼロに
- ❑ Write性能のスケーリング

Auroraサーバレス

予測困難なアプリケーションワークロードに対応したAuroraのオンデマンド自動スケーリング構成

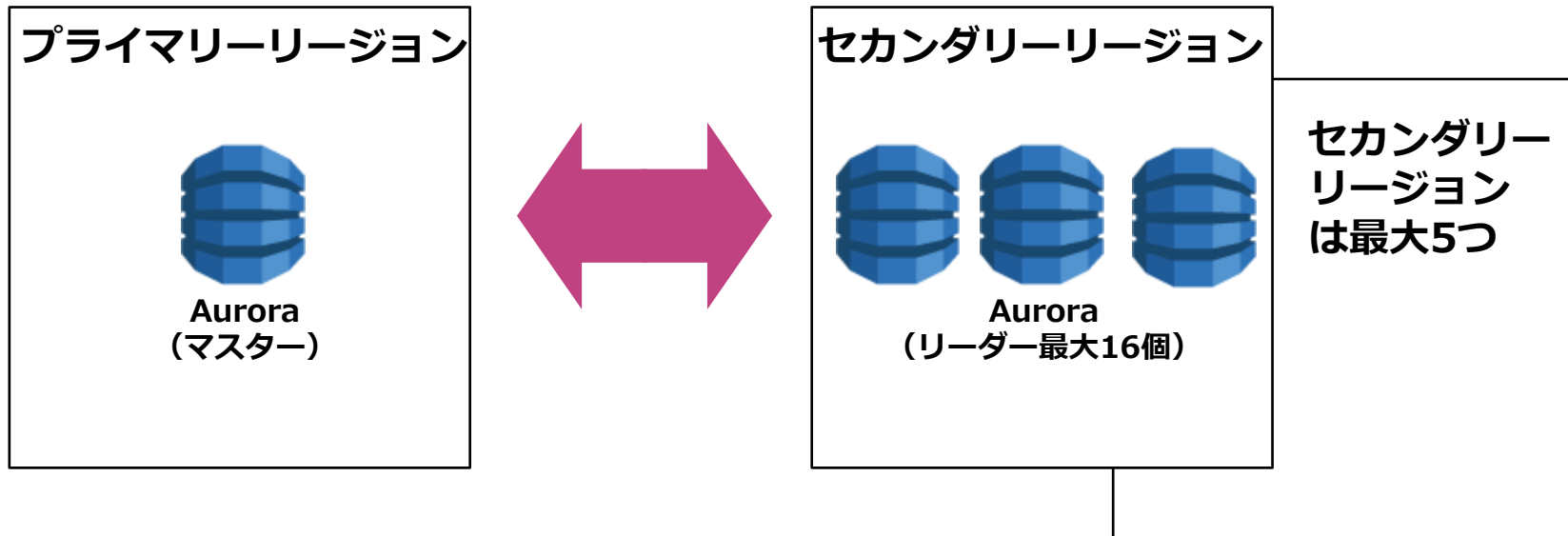
- アプリケーションのニーズに基づいて実行される
 - 自動的に起動／シャットダウン
 - 自動でスケールアップ／スケールダウン
- 断続的な予測不能なワークロードに向いている。
- 秒単位の支払い



AuroraグローバルDB

他リージョンに対する高性能なリードレプリカ作成機能

- ❑ 最大5つのセカンダリリージョンを構成できる。各リージョンでは最大15個のリードレプリカを構成
- ❑ ログ転送ではなく、ストレージレベルのレプリケーション機能を利用してレプリケーションを実施
- ❑ 概ね1秒以下／最大でも5秒でレプリケーションを実行する低レイテンシーレプリケーションを実現
- ❑ ディザスタリカバリ時のRTOは1分未満



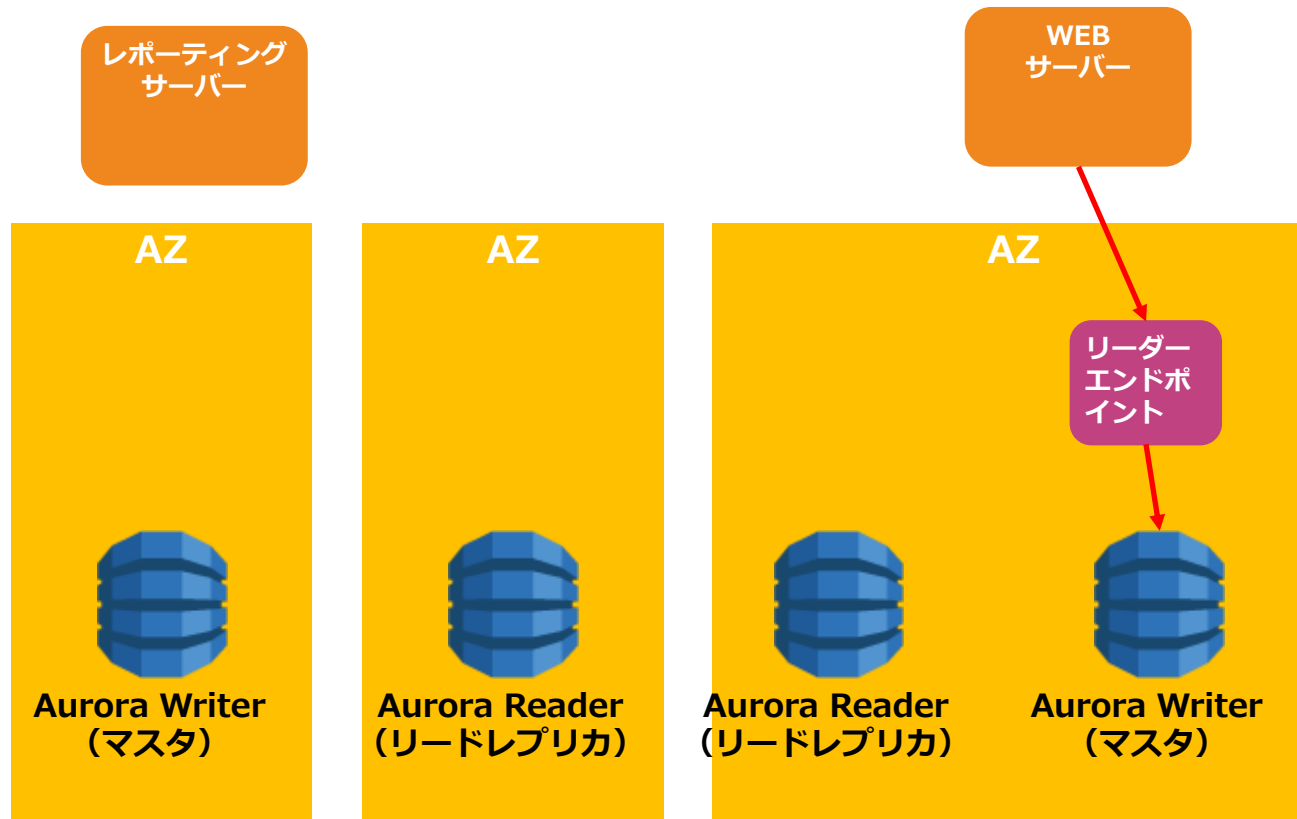
エンドポイントの選択

利用したいインスタンスタイプに応じてエンドポイントを選択

クラスター (書き込み) エンドポイント	❑ Auroraクラスターの書き込み処理に利用できるエンドポイント ❑ Writer専用
リーダー (読み込み) エンドポイント	❑ リードレプリカへのアクセスに利用するReaderエンドポイント ❑ リードレプリカ専用
インスタンス エンドポイント	❑ 特定のインスタンスへアクセスするエンドポイント ❑ 個別のインスタンスに設定
カスタム エンドポイント	❑ 特定のリーダーインスタンスを、組み合わせを変更できるカスタムエンドポイント。 ❑ 高負荷な分析処理などの処理ごとにインスタンスを分けたい場合に利用する。

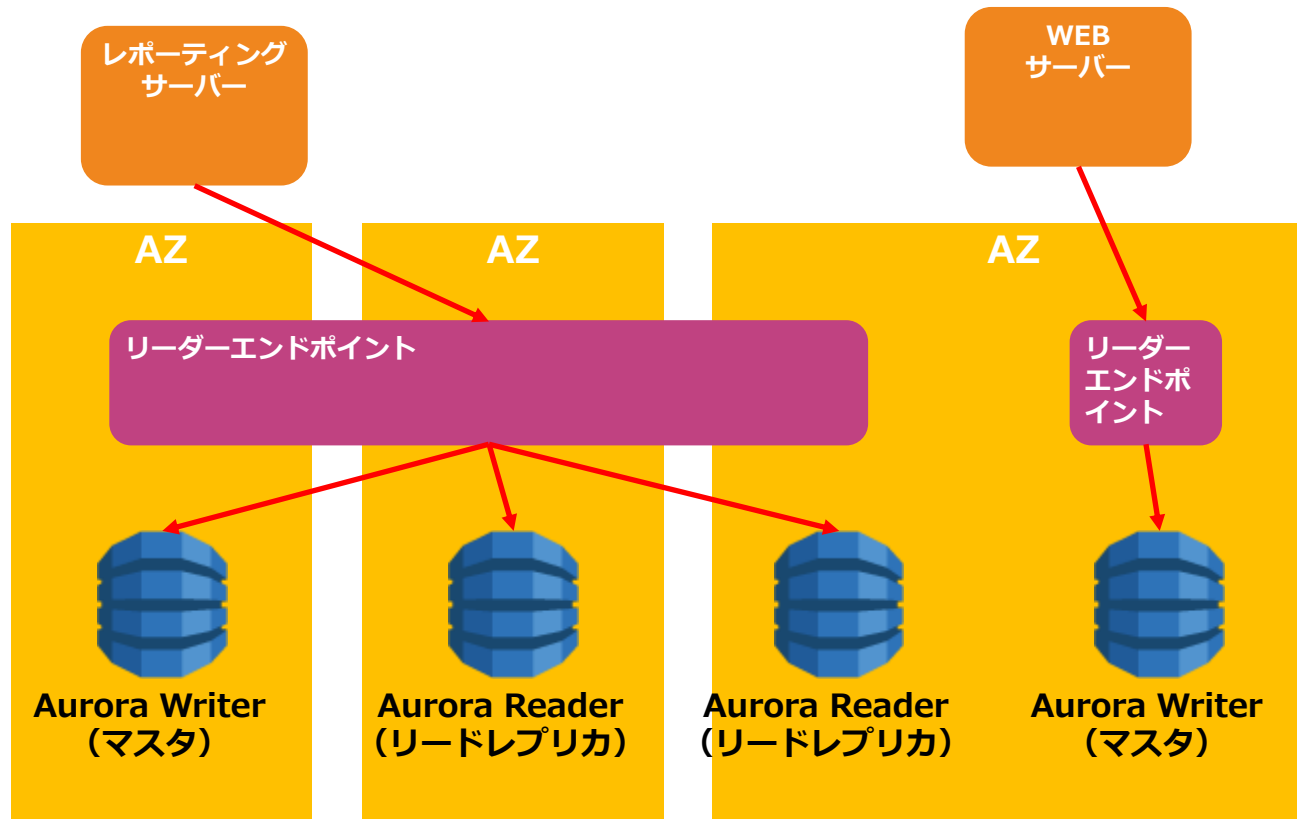
エンドポイントの選択

利用したいインスタンスタイプに応じてエンドポイントを選択



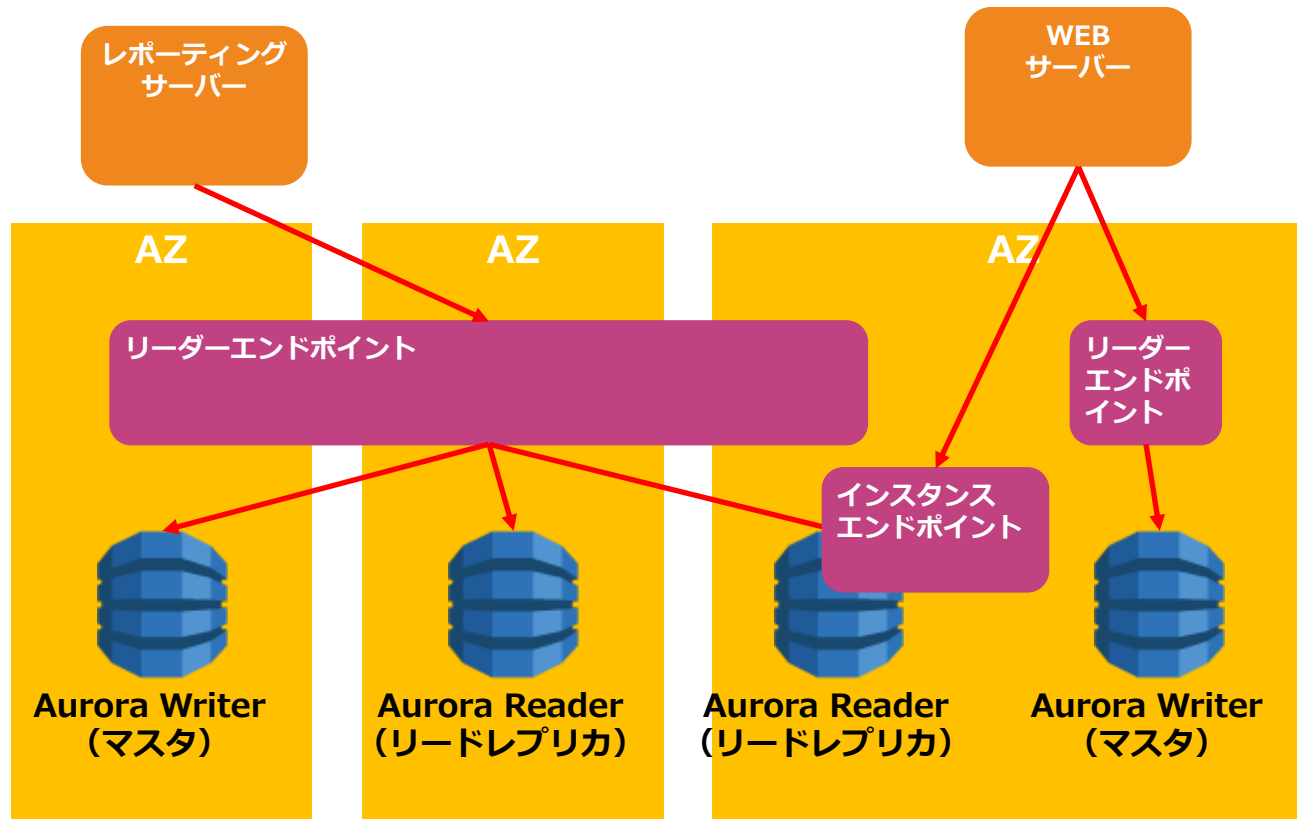
エンドポイントの選択

利用したいインスタンスタイプに応じてエンドポイントを選択



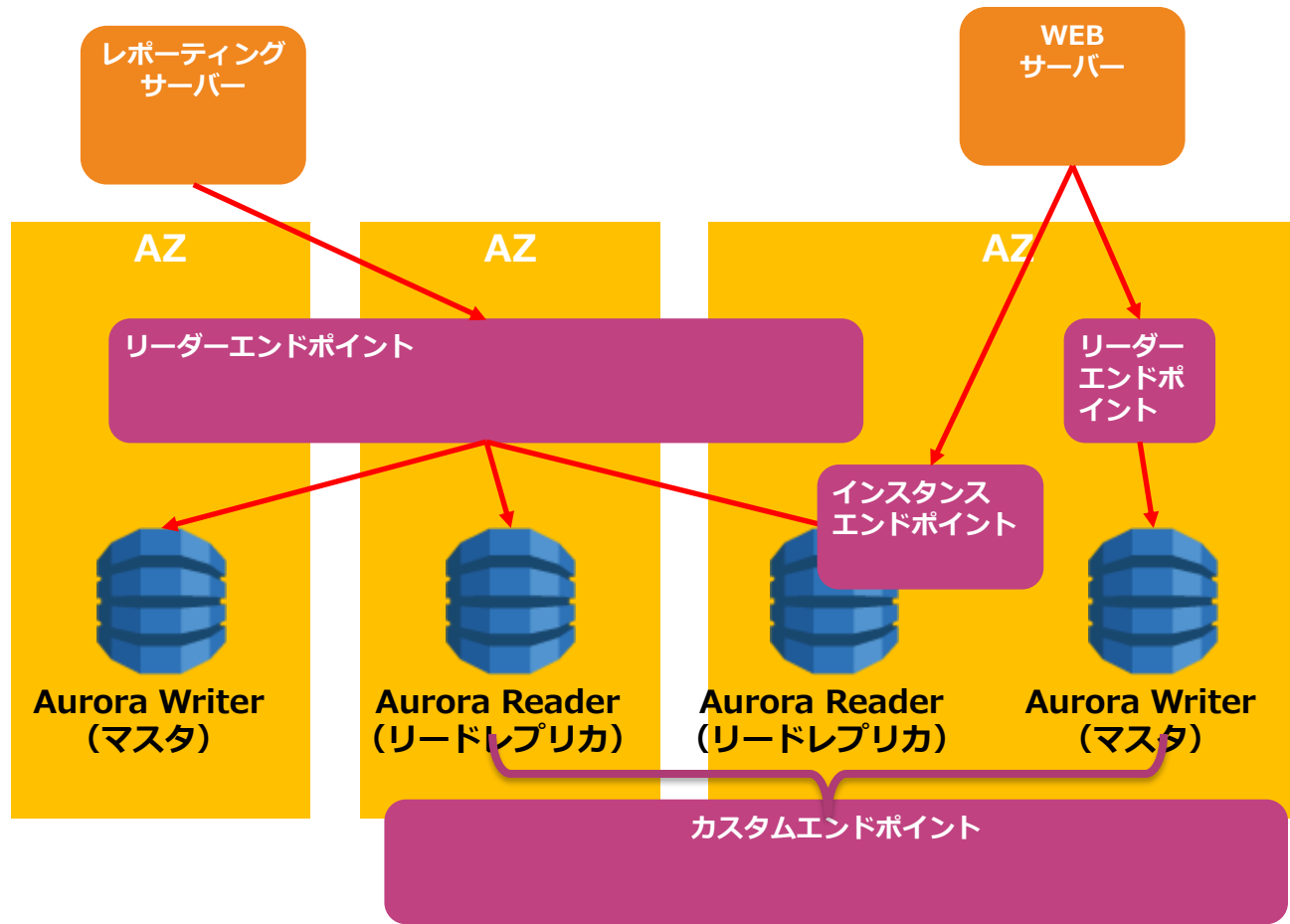
エンドポイントの選択

利用したいインスタンスタイプに応じてエンドポイントを選択



エンドポイントの選択

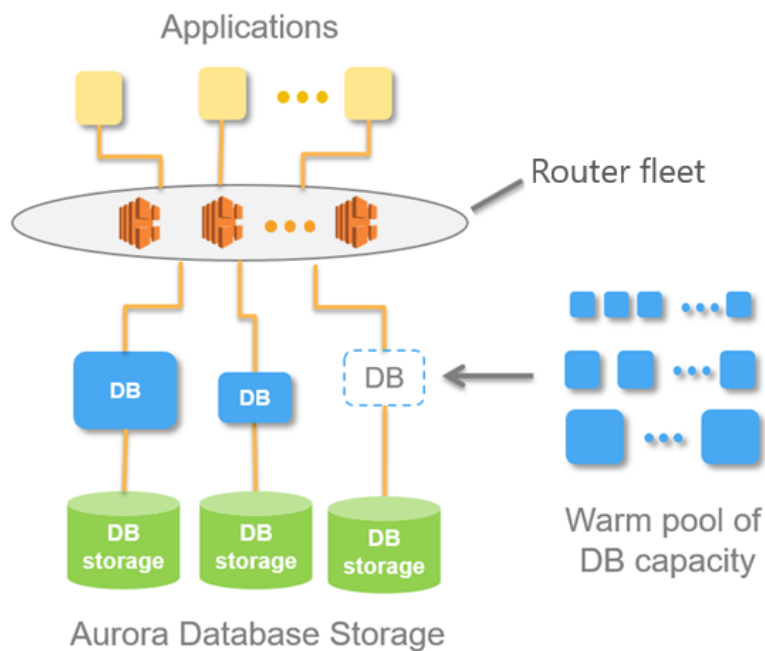
利用したいインスタンスタイプに応じてエンドポイントを選択



Auroraサーバレスの設定

Auroraサーバレス

予測困難なアプリケーションワークロードに対応したAuroraのオンデマンド自動スケーリング構成



アプリケーションのニーズに基づいて実行される

- ・自動的に起動／シャットダウン
- ・自動でスケールアップ／スケールダウン

サーバレスアプリケーション向けに発生ベースで利用可能な柔軟なリレーショナルデータベースとして利用可能

インスタンスサイズを決めない

EC2やRDSを設定する際に必ずあったインスタンスクラス
の選択がない。

DB インスタンスクラス

DB インスタンスクラス [情報](#)

- ☒ メモリ最適化クラス (r クラスを含む)
- ☐ バースト可能クラス (t クラスを含む)

db.r5.large

2 vCPUs 16 GiB RAM ネットワーク: 4,7

☐ 以前の世代のクラスを含める

最小・最大キャパシティを設定

代わりにACUというAuroraサーバレスが起動するキャパシティ範囲を最小値と最大値で設定する。

キャパシティの設定

この請求予測額は、公開されている料金に基づきます。[詳細はこちら](#) 

最小 Aurora 容量ユニット [情報](#)

1 ACU
2 GiB RAM



最大 Aurora 容量ユニット [情報](#)

64 ACU
122 GiB RAM



▶ スケーリングの追加設定

最小・最大キャパシティを設定

代わりにACUというAuroraサーバレスが起動するキャパシティ範囲を最小値と最大値で設定する。

キャパシティの設定

この請求予測額は、公開されている料金に基づきます。[詳細はこちら](#)

最小 Aurora 容量ユニット 情報	最大 Aurora 容量ユニット 情報
1 ACU 2 GiB RAM	64 ACU 122 GiB RAM

▶ スケーリングの追加設定

両方のACU値の間で自動でキャパシティを増減する。

最小・最大キャパシティを設定

スケーリングがタイムアウトした場合やアイドル状態となった場合の処理設定も必要

▼ スケーリングの追加設定

オートスケーリングのタイムアウトとアクション [情報](#)

タイムアウトアクションの前に Aurora がスケーリングポイントを検索するまでの時間を指定します。

00:05:00

最大: 10 分。最小: 1 分。

スケーリングポイントが見つかる前にタイムアウトが期限切れになった場合は、次の操作を行います。

☒ 容量の変更をロールバック

Aurora Serverless クラスターの容量は変更されません。現在の容量のままです。

☐ 容量の変更を強制

Aurora Serverless クラスターの容量は、スケーリングポイントなしで変更されます。これにより、処理中のトランザクションが中断され、再送信が必要になります。

非アクティブ後に一時停止 [情報](#)

☐ クラスターがアイドル状態のときに容量を 0 ACU にスケールする

このオプションの設定により、Aurora Serverless クラスターは非アクティブ時に容量を 0 ACU にスケーリングできます。データベーストラフィックが再開すると、Aurora Serverless クラスターは処理能力を再開し、トラフィックを処理するようにスケーリングします。

DATA API

エンドポイントを使用して、接続を管理せずにAuroraサーバレスに対してSQL ステートメントを実行する。

CLIでSQLコマンドを実行

```
aws rds-data execute-statement --resource-arn "arn:aws:rds:us-east-1:123456789012:cluster:mydb" --database "mydb" --secret-arn "arn:aws:secretsmanager:us-east-1:123456789012:secret:mysecret" --sql "select * from mytable"
```



結果を抽出

```
{
  "numberOfRecordsUpdated": 0,
  "records": [
    [
      {
        "longValue": 1
      },
      {
        "stringValue": "ValueOne"
      }
    ],
    [
      {
        "longValue": 2
      },
      {
        "stringValue": "ValueTwo"
      }
    ],
    [
      {
        "longValue": 3
      },
      {
        "stringValue": "ValueThree"
      }
    ]
  ]
}
```

コストを抑える

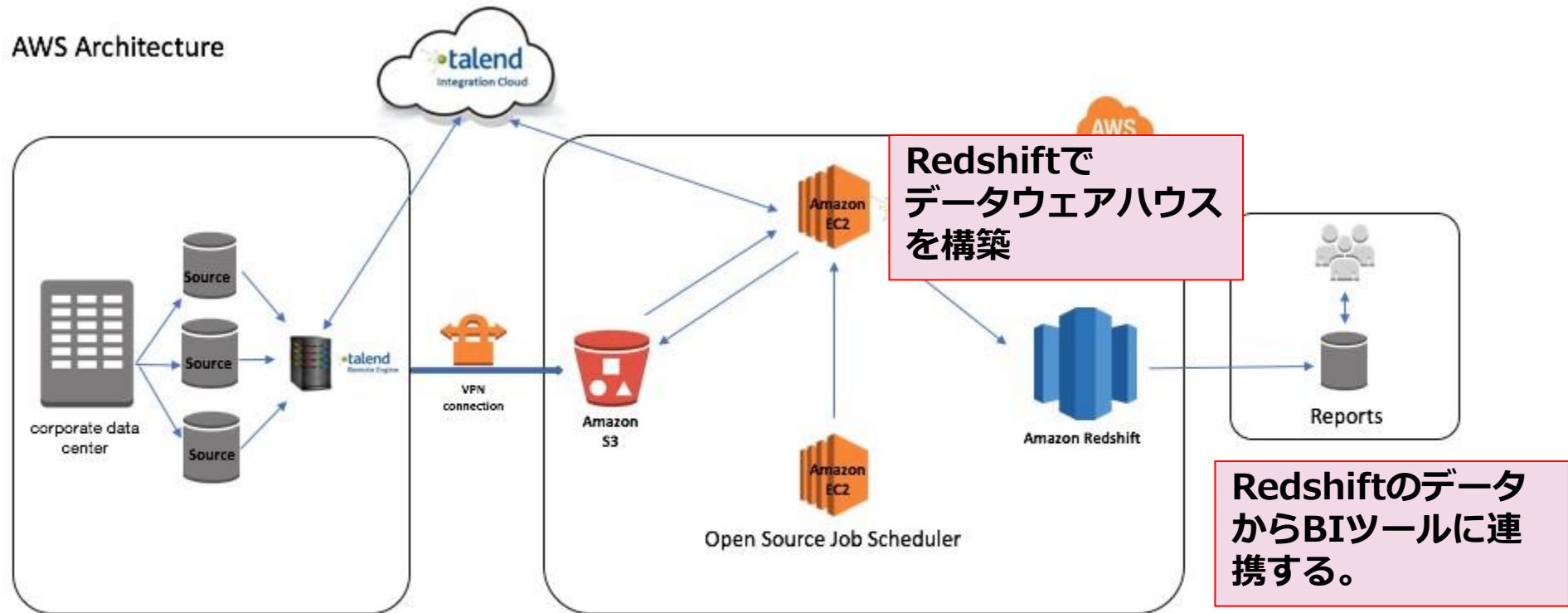
Auroraサーバレスを利用する大きなメリットはユースケースに応じてコストを最も抑えることができること。

	RDS	Aurora	Auroraサーバレス
処理性能	標準的な性能を発揮	AuroraがRDSの5倍近い性能を発揮するため最も性能が高い。	Auroraと同じくらいの性能を発揮するものの、スケーリング処理でAuroraに劣る。
可用性	マルチAZ配置にリードレプリカを複数リージョンに展開可能だが、Auroraに劣る。	マルチマスター/15台のレプリカなど最も可用性が高い。 高速フェールオーバーが可能	Auroraに比較すると冗長化度合いが低くなる。
運用	運用はマネージド型サービスで展開	運用はマネージド型サービスで展開 + AutoScaling	運用はマネージド型サービスで展開 + AutoScaling
コスト	実働時間が予測不能で変動が激しいケースではコスト効率が悪い。	実働時間が予測不能で変動が激しいケースではコスト効率が悪い。	実働時間が予測不能で変動が激しいケースでは コストが最も安くなる。

Redshiftの概要

Redshiftとは何か？

AWS上にデータウェアハウスを構築することができるマネージド型サービス



Reference: <https://aws.amazon.com/jp/blogs/database/using-amazon-redshift-for-fast-analytical-reports/>

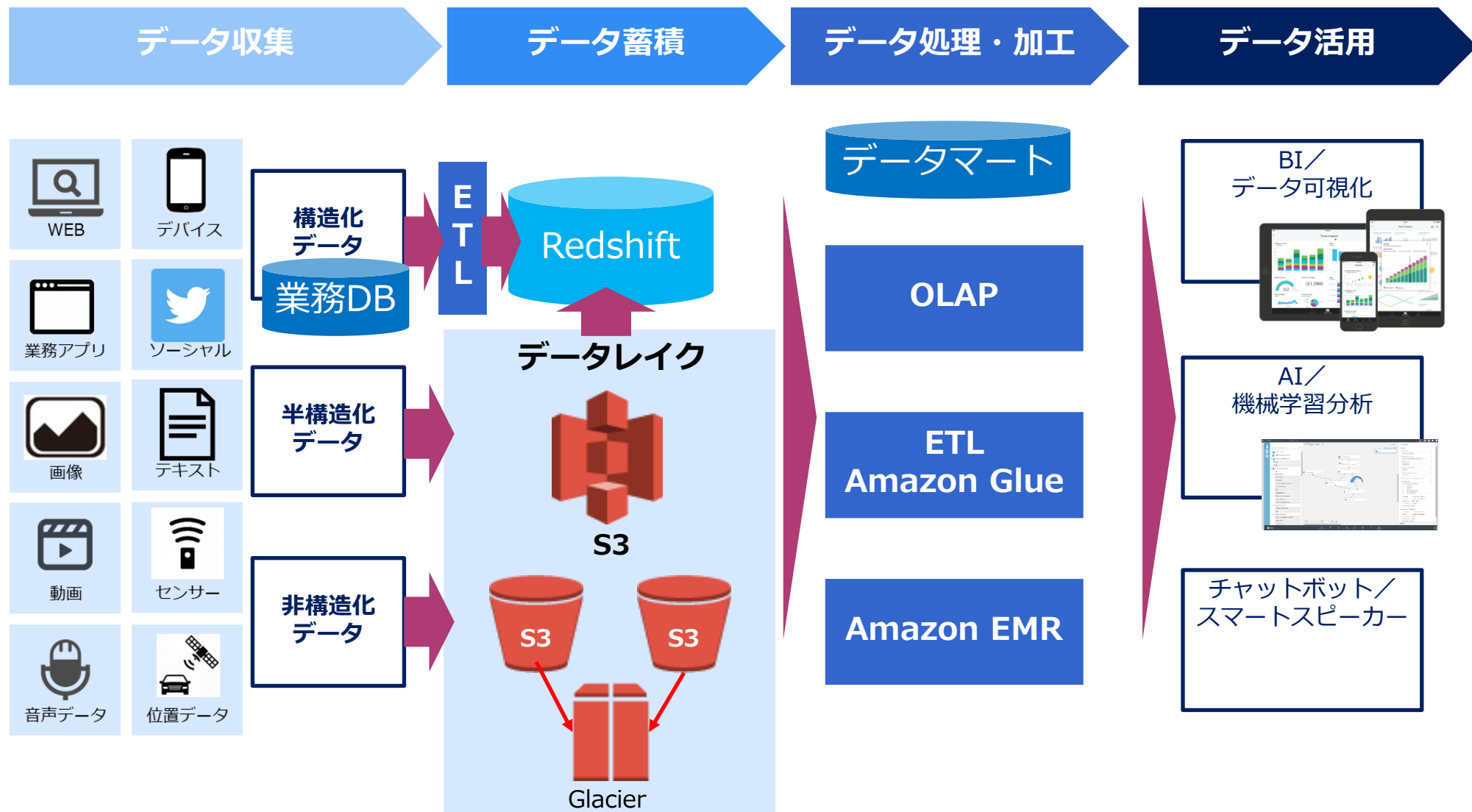
Redshift

高速でスケラブルな費用対効果の高いマネージド型のDWH

- データ分析用のリレーショナルデータベースであり、OLTPには利用不可
- 数百ギガバイトのデータから開始して、ペタバイト以上まで拡張
- 1 テラバイトあたり年間 1,000 USD 以下で利用可能
- 自動ワークロード管理など自動テーブルメンテナンスなど多くのメンテナンスタスクやデータ配置が自動化されているフルマネージド型
- PostgreSQL互換の列指向データモデルであり、列ベースでデータを保存
- 複数ノードをまとめたクラスター構成。マルチAZ構成も可能になった。
- 大規模並列クエリ実行によって高パフォーマンスを達成
- AQUAによる分散キャッシュで、Redshift が他のクラウドデータウェアハウスに比べて最大 10 倍の速度で動作
- RA3インスタンスで最大で他のクラウドデータウェアハウスの 3 倍に達するパフォーマンス

データレイク

S3はデータレイクとしてデータ活用のハブとして利用できる



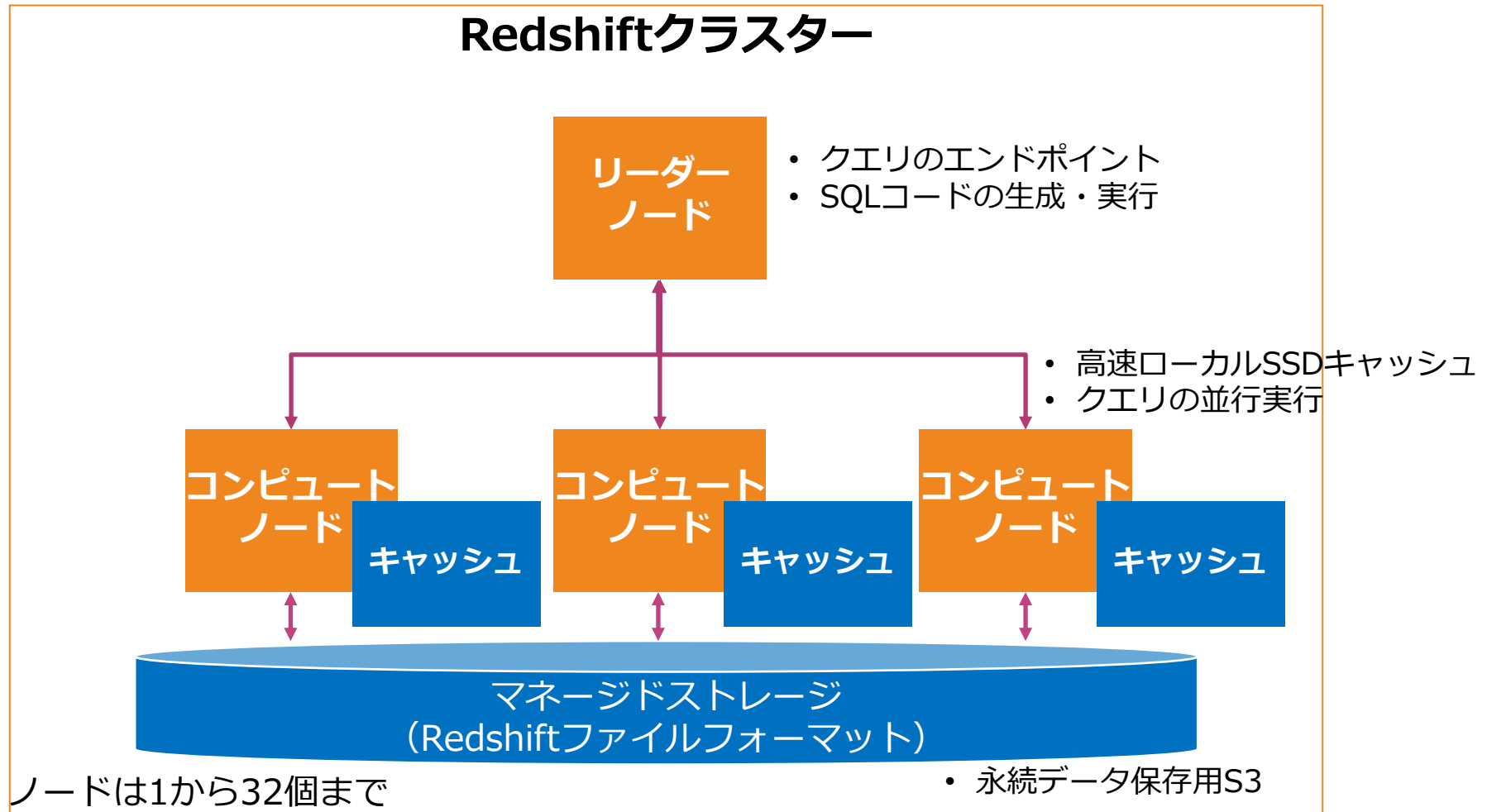
ノードタイプ

利用するデータサイズやケースに応じてノードタイプを選択

RA3	Dense Storage (DS2) 高密度ストレージ	Dense Compute (DC) 高密度コンピューティング
■ 高性能コンピューティングとスケーリングされるストレージを利用するDWH ■ コンピューティングをストレージから別にスケーリングできる柔軟性 ■ データ量の増大が予想される場合に推奨 ■ ホットデータには高性能のSSDを使用し、コールドデータにはS3を使用	■ 固定ローカル SSD ストレージを使用した低コストDWH ■ 3年間のリザーブドインスタンスを購入するとHDDベースの低コストで利用可能 ■ 未圧縮で1TB未満のデータセットではDC2 ノードタイプの利用を推奨	■ 高いスループットと低いレイテンシを必要とするDWHワークロード用 ■ 高速 CPU、大容量 RAM、SSD (Solid-State Disk) を使用 ■ データが500GBを超える場合に最適

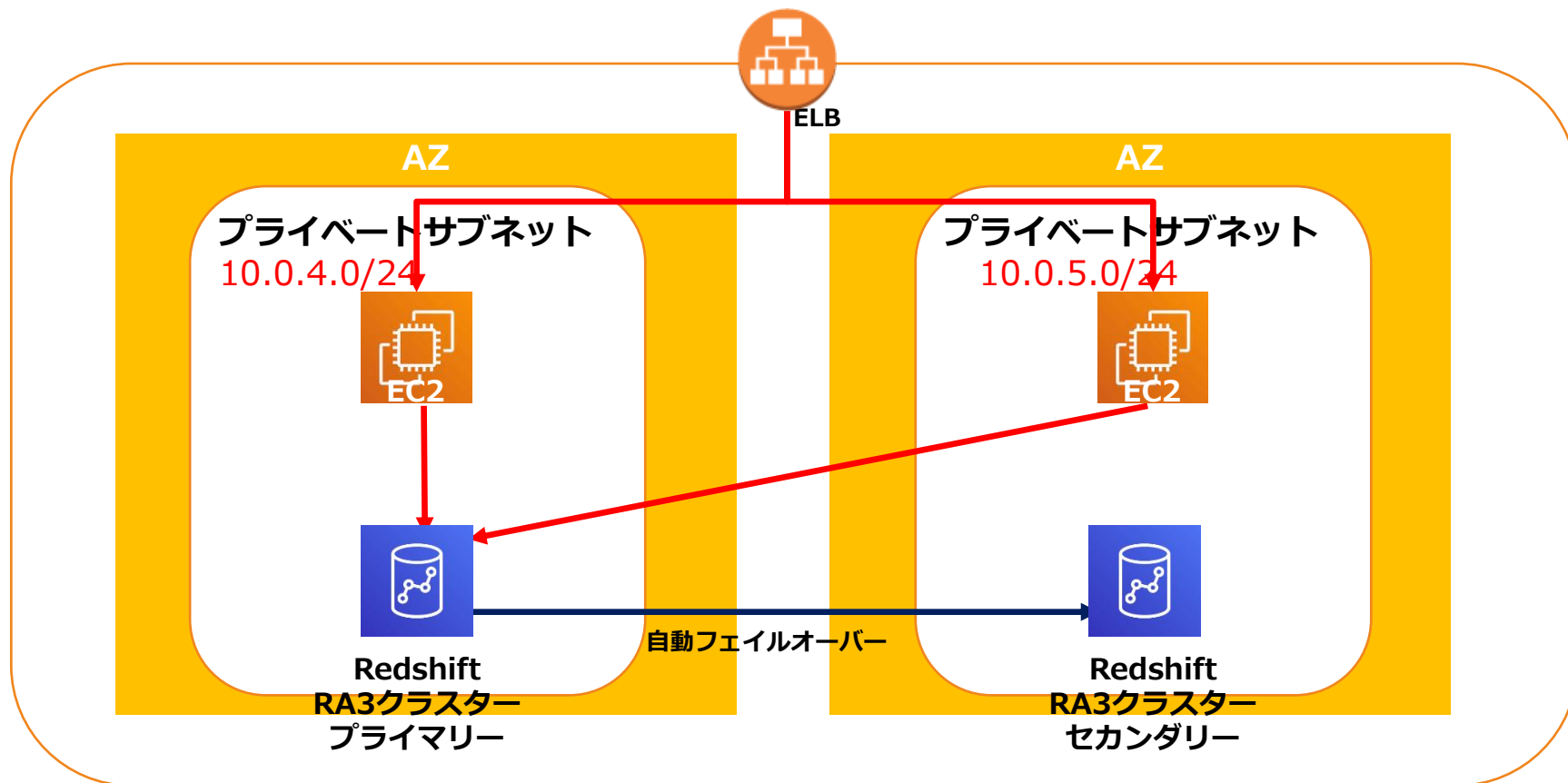
Redshiftの構成

クラスターというグループ単位で、複数ノードによってデータ処理を実行する構成



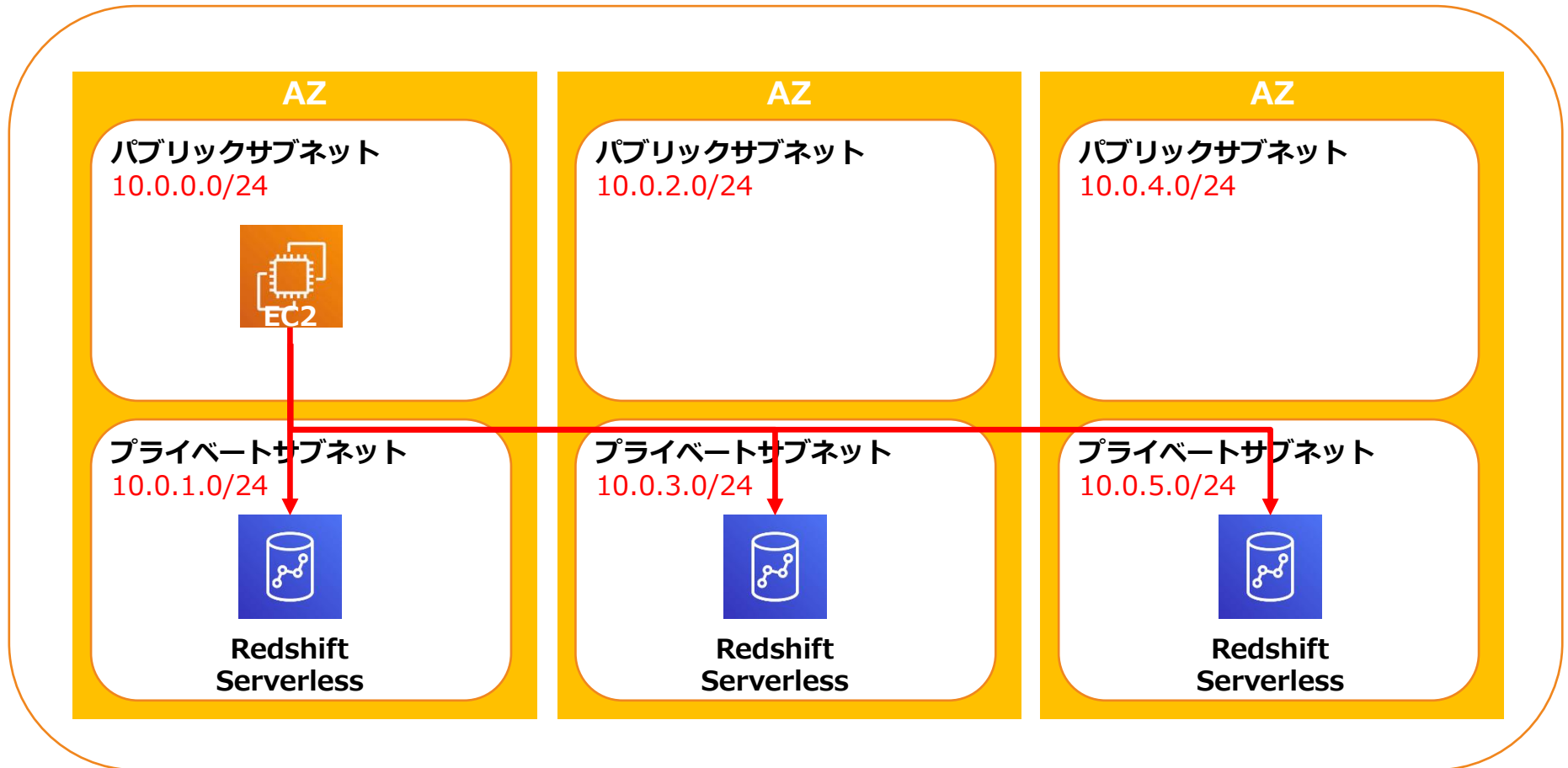
マルチAZ構成

Redshift（プロビジョンドクラスター）でもマルチAZのフェールオーバー構成ができるようになりました。



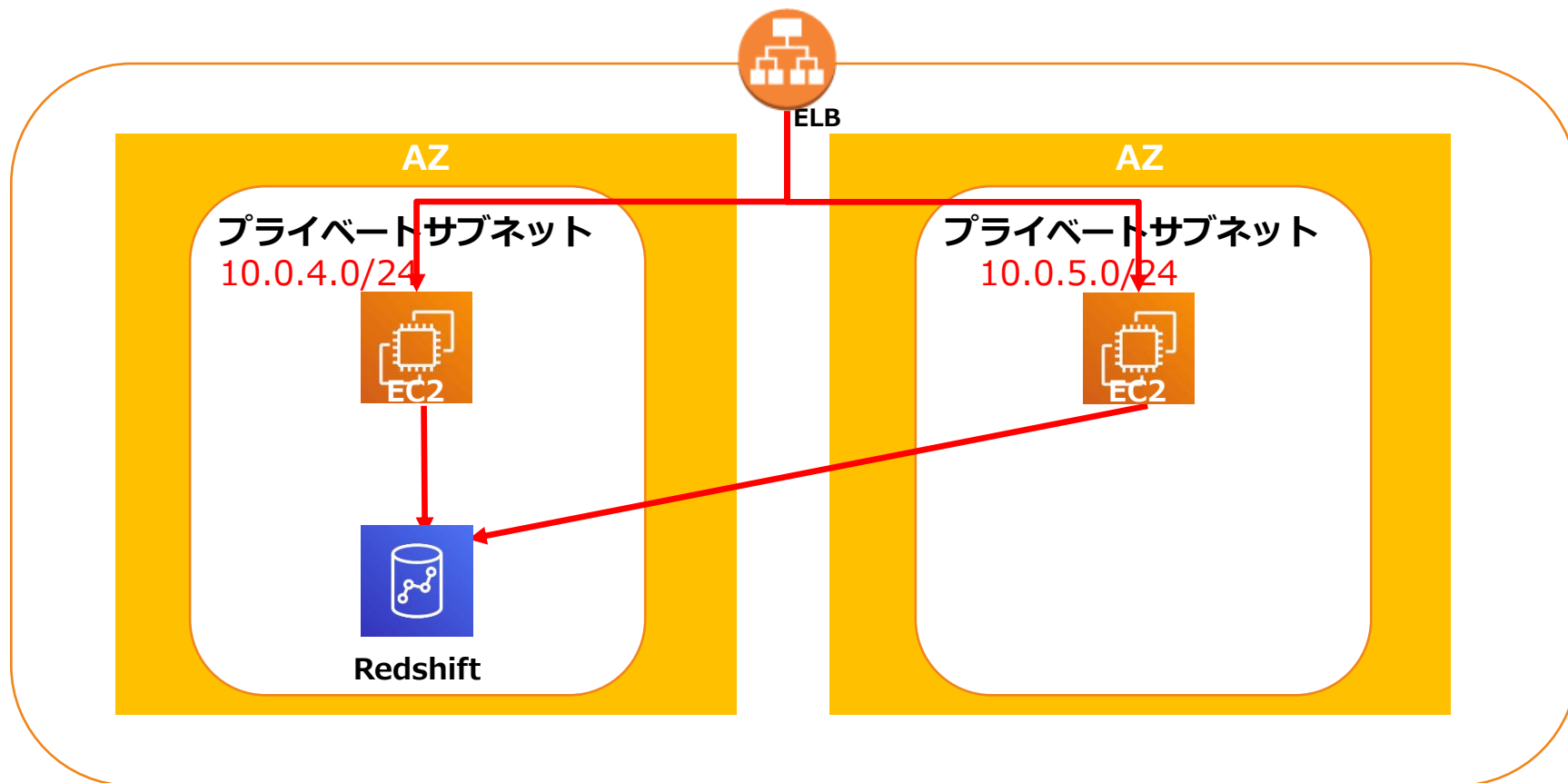
Redshift Serverless

クラスターをセットアップ、調整、管理することなく、データにアクセスして分析が可能なRedshiftの構成



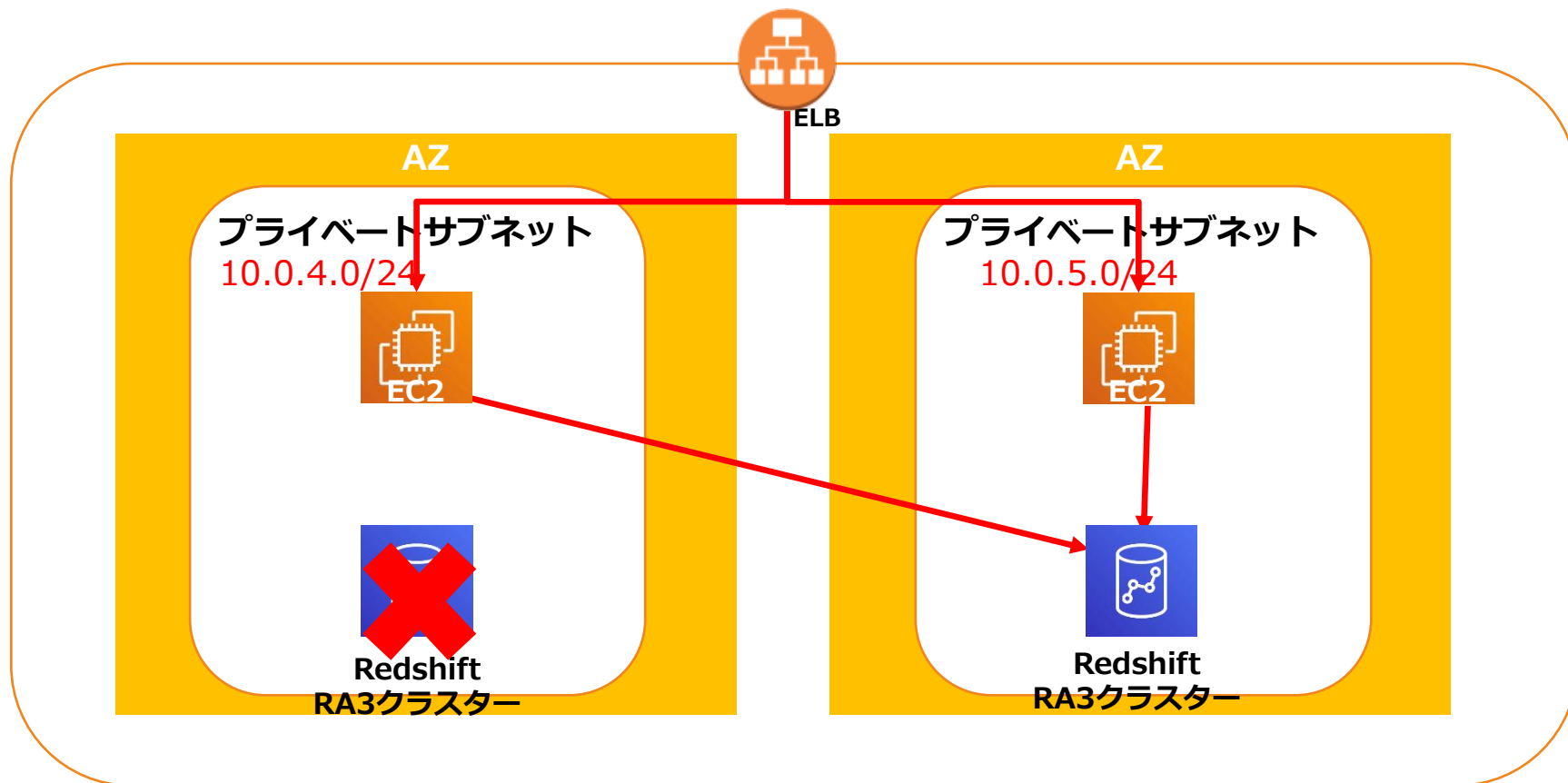
クロスAZクラスターリカバリー

Redshift障害を自動的に検知して、別AZにクラスターを復旧



クロスAZクラスターリカバリー

Redshift障害を自動的に検知して、別AZにクラスターを復旧



運用の自動化

自動的なメンテナンス機能と詳細のモニタリングによる簡易な運用が可能

CloudWatchとの連動	❑ 初期設定でCloudWatchメトリクス取得が自動で実施され、Redshiftコンソール内で確認可能
バックアップ	❑ 実行時間を設定して自動でバックアップを定期取得する ❑ スナップショットを手動で取得することも可能
自動メンテナンス	❑ パッチ適用も自動で実施 ❑ メンテナンスウィンドウでパッチ適用時間を指定可能

機械学習によるクエリ効率化

機械学習によってクエリ実行を調整し、効率的な自動実行を補助してくれる。

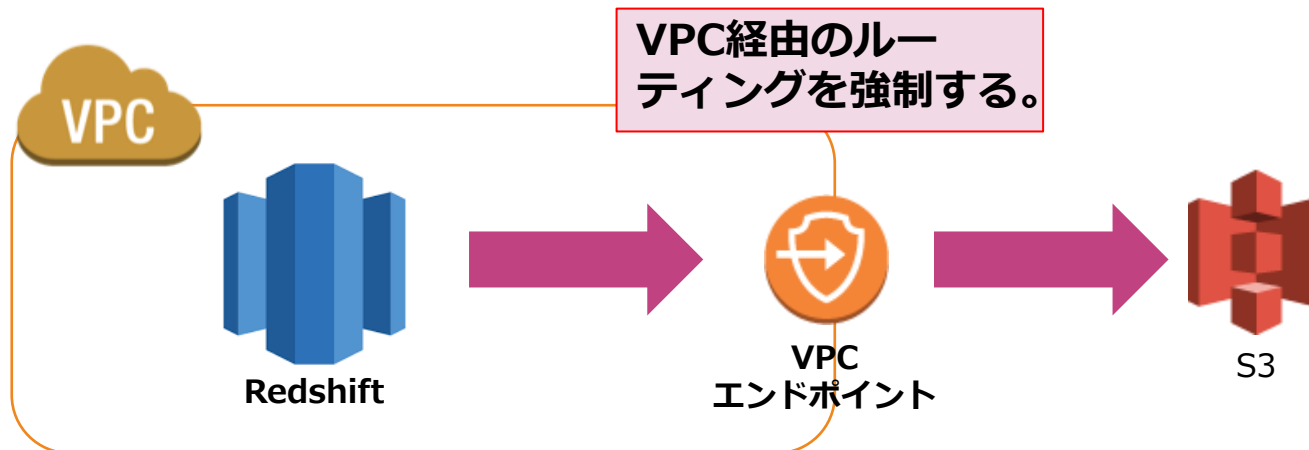
テーブルメンテナンス の自動化	□ テーブルの分散スタイルの自動最適化 □ 統計情報の自動更新 □ データの再編成の自動実行
自動ワークロード管理	□ 複数クエリの実行をワークロード管理で設定する際に、機械学習でクエリ実行の優先順位決めを自動化する
ショートアクセル レーション	□ 機械学習アルゴリズムを使用して対象のクエリを 1 つ 1 つ分析し、クエリの実行時間を予測し、実行時間が短いクエリを、実行時間が長いクエリよりも優先して実行 □ WLMキューを削減可能
設定の レコメンデーション	□ 自動でクラスターパフォーマンスなどを分析し、最適化やコスト削減に対するレコメンデーションを実施

トラフィック制御

拡張VPCルーティングによってVPCにトラフィックを強制しつつ、モニタリングが可能

拡張VPCルーティング

- ❑ Amazon Redshift はクラスターとデータリポジトリ間のすべての COPY と UNLOAD トラフィックが Amazon VPC を経由するよう強制する。
- ❑ VPC フローログを使って COPY と UNLOAD トラフィックを監視する
- ❑ 有効にすることでVPCにて設定しているルートテーブルに従った通信を行う



暗号化の実施

RedshiftにおいてもKMSやACMを利用した暗号化を実施可能

保存データの暗号化

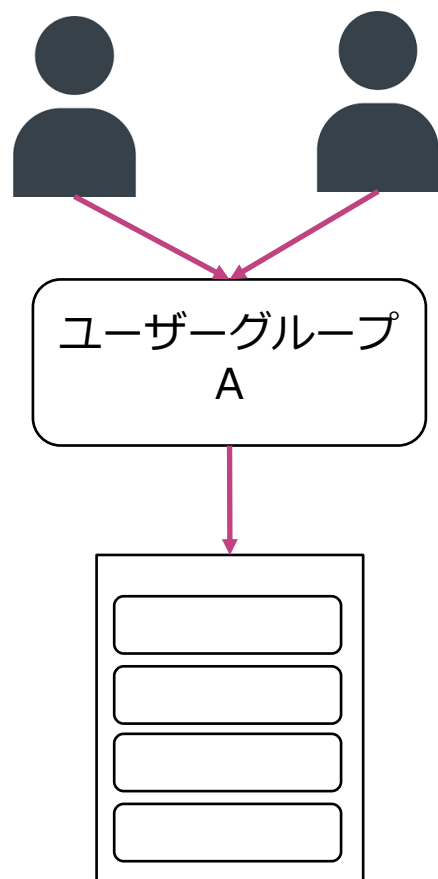
- ❑ 他のデータベースと同様にAWS KMSを利用して暗号化
- ❑ 暗号化対象はディスク内に保存されているデータとスナップショット

通信の暗号化

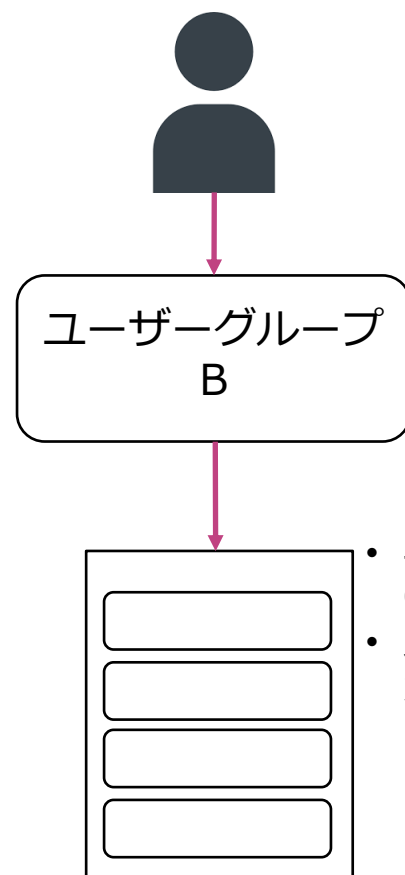
- ❑ Amazon Redshift クラスターと SQL クライアントの間の送信時のデータを JDBC/ODBC 経由で暗号化する
- ❑ ACMによる証明書に基づいてSSL通信を実施

ワークロード管理（WLM）の活用

ワークロードに応じて複数のキューを設定し、クエリ割り当てルールに基づいてキューを設定して、優先順位を設定可能



ロングクエリ用キュー



ショートクエリ用キュー

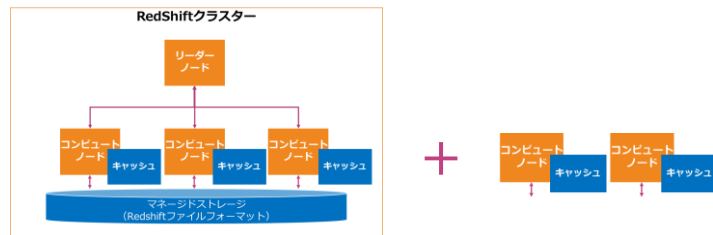
- キューにスロットを設定し、CPUとメモリの割り当てる
- スロットを増やすと並列度が向上するが、割り当てメモリが減少

スケーリング

RedShiftはノードのタイプ変更・追加とクラスターの追加によってスケーリング可能

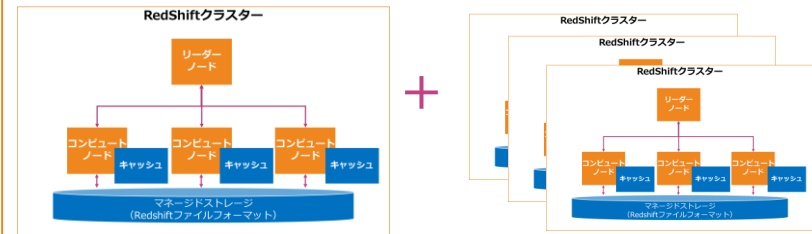
ノードの追加

コンピューティングノードを追加することでパフォーマンスを向上



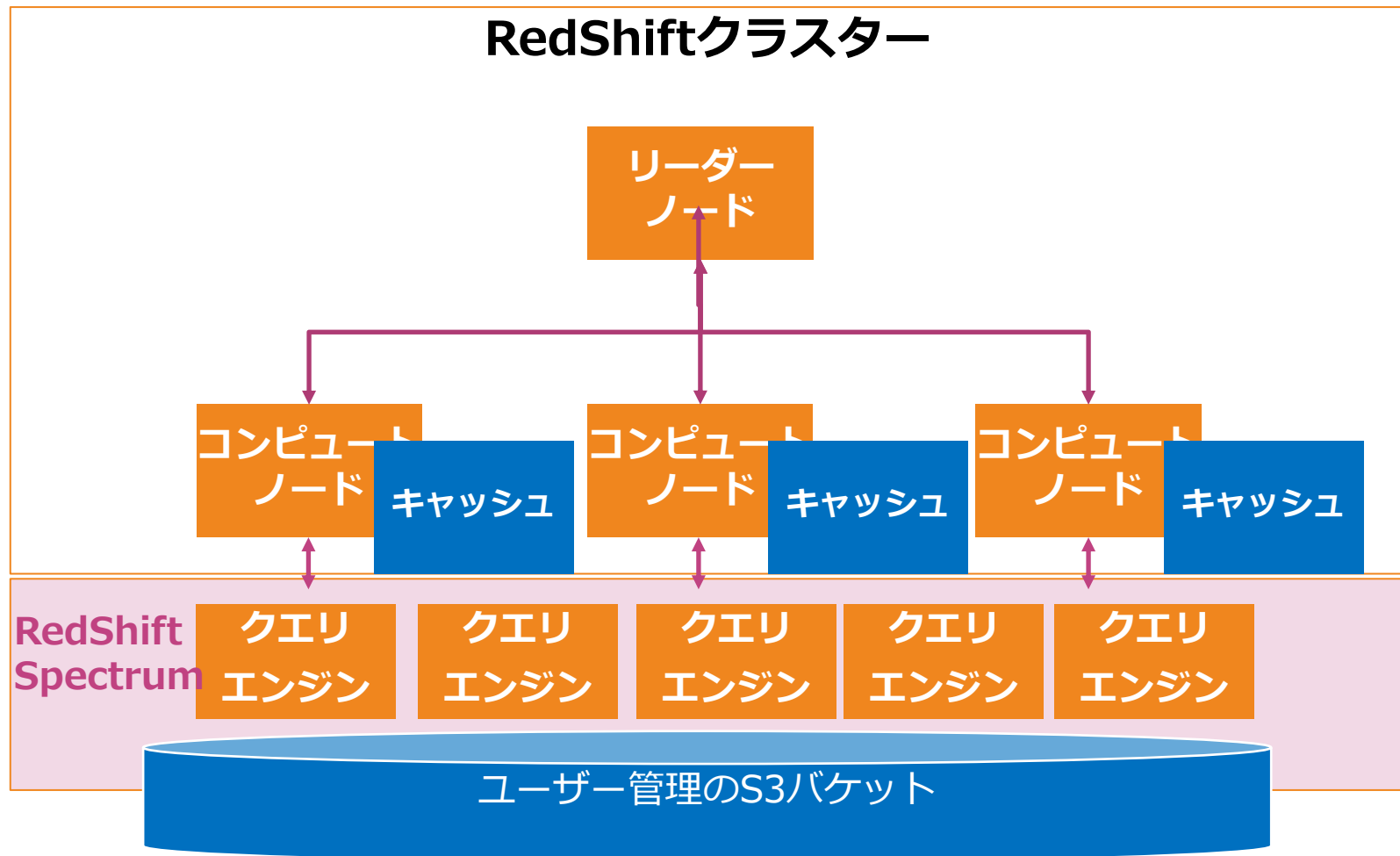
クラスターの追加

Concurrency Scalingにより急な同時実行リクエストに対応するために 一時的なクラスタを自動的に数秒で追加し、一貫して高速なパフォーマンスを発揮
(追加クラスターは1~10)



RedShift Spectrum

RedShift Spectrumにより、ユーザーが管理するS3バケットに対して直接データ解析を実行可能



データ連携 (To Redshift)

Redshiftへとデータを移動させることで、DWHとしての解析基盤を集約化することが重要

S3	❑ S3からデータを取得してRedshiftで解析。 ❑ 自動アップロード機能によりRedshiftにデータロード ❑ S3内部のデータ解析を直接実行することも可能
Kinesis	❑ Kinesis data Firehoseを利用してストリーミングデータの格納先としてRedshiftを指定してデータを保存して、解析に利用することが可能
RDS	❑ RDSとの直接接続はないが、AWS Data PipelineやDMSを利用してデータ移行を実施可能 ❑ Federated QueryによりRDSとAuroraに直接クエリを実行
DynamoDB	❑ DynamoDBからRedshiftにデータコピーを実行可能
Amazon EMR	❑ EMRからRedshiftにデータコピーを実行可能

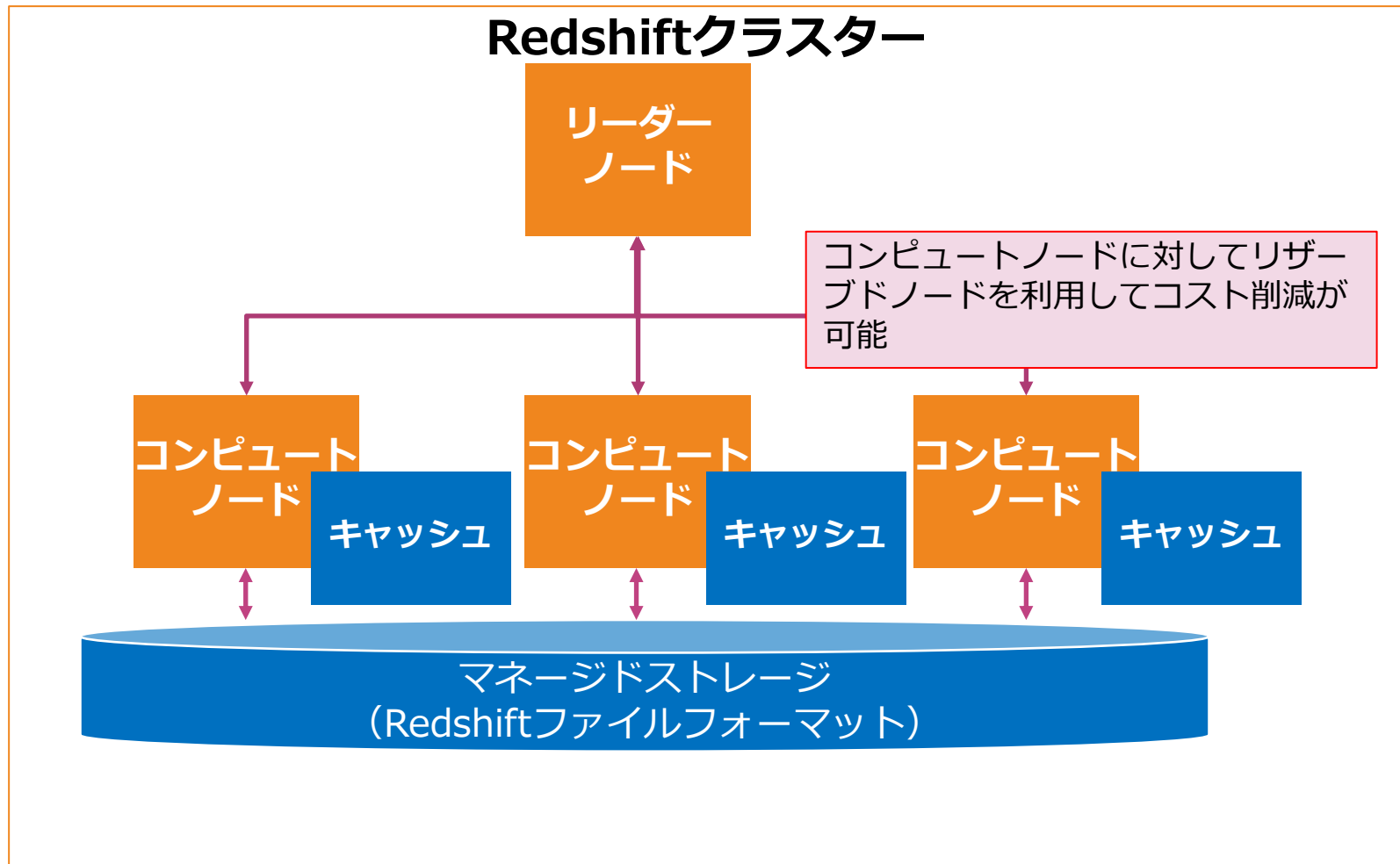
データ連携 (From Redshift)

RedshiftからはQuickSightを利用したデータ可視化に加え、S3へとデータ抽出も可能

Amazon QuickSight	❑ Redshiftに接続して、データの可視化を実施可能
S3	❑ UNLOADコマンドを実行することで、RedshiftからS3へとデータを抽出することが可能
Amazon Machine Learning	❑ RedShiftを機械学習の学習データとして設定して利用可能
RDS	❑ 直接に連携はできないが、PostgreSQLの機能を利用してデータをRedShiftからRDSと連携可能

リザーブドノードの利用

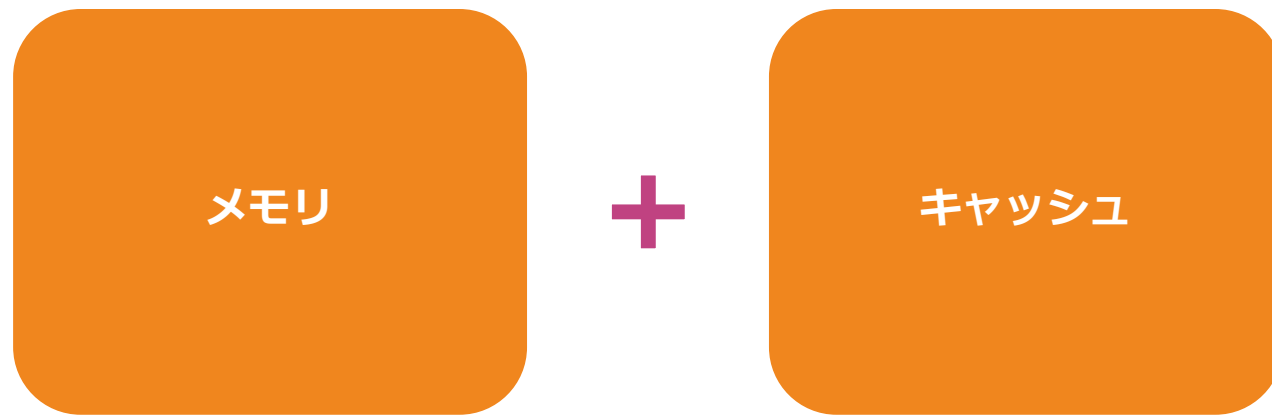
クラスターというグループ単位で、複数ノードによってデータ処理を実行する構成



ElastiCacheの概要

インメモリキャッシュ

ElastiCacheはメモリ+キャッシュというデータベース



データを保持するHW

PCなどの機器でデータを保存するためのHWはメモリとHDDなどのディスクがある



メモリ

HDD

メモリ型DB

データをメモリ上で動作をさせるとディスク上で動作させる場合と比較して大幅に高速に処理が可能

メモリ型DB

データ

データ

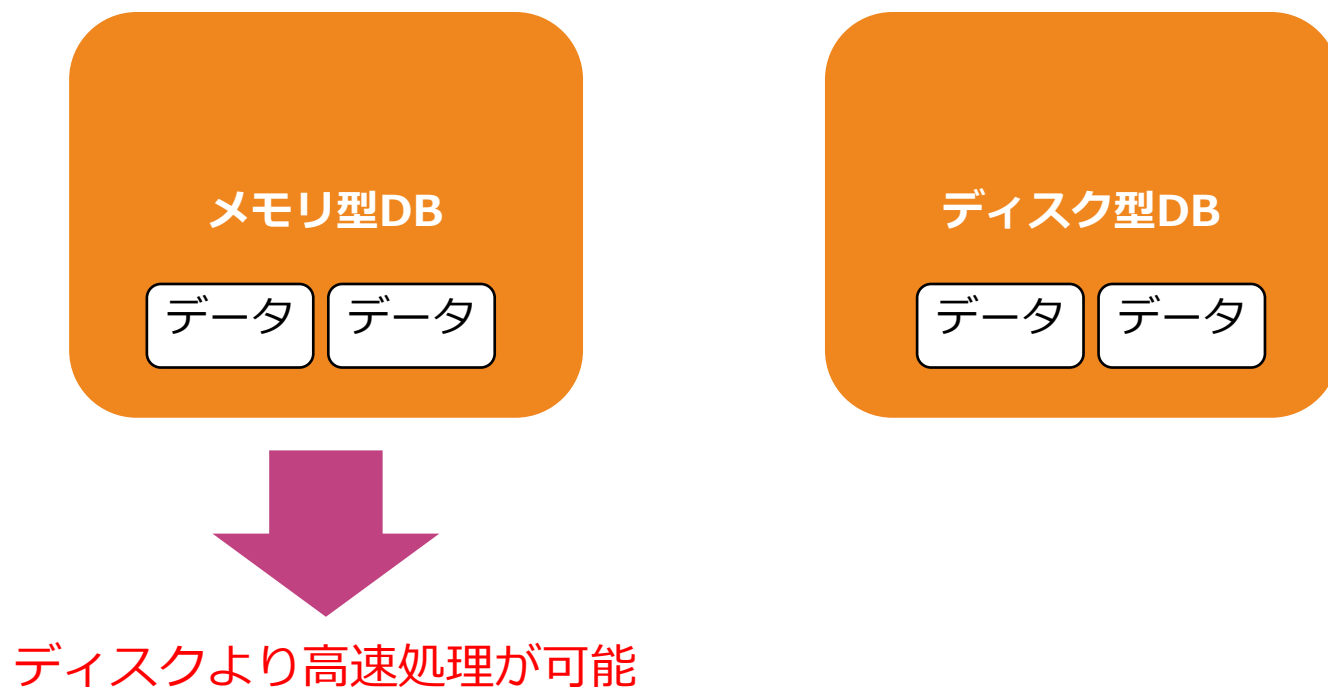
ディスク型DB

データ

データ

メモリ型DB

データをメモリ上で動作をさせるとディスク上で動作させる場合と比較して大幅に高速に処理が可能



メモリ型DB

つまり、メモリDBをうまく利用することでデータの高速処理を実現できる



ディスクより高速処理が可能

キャッシュとは

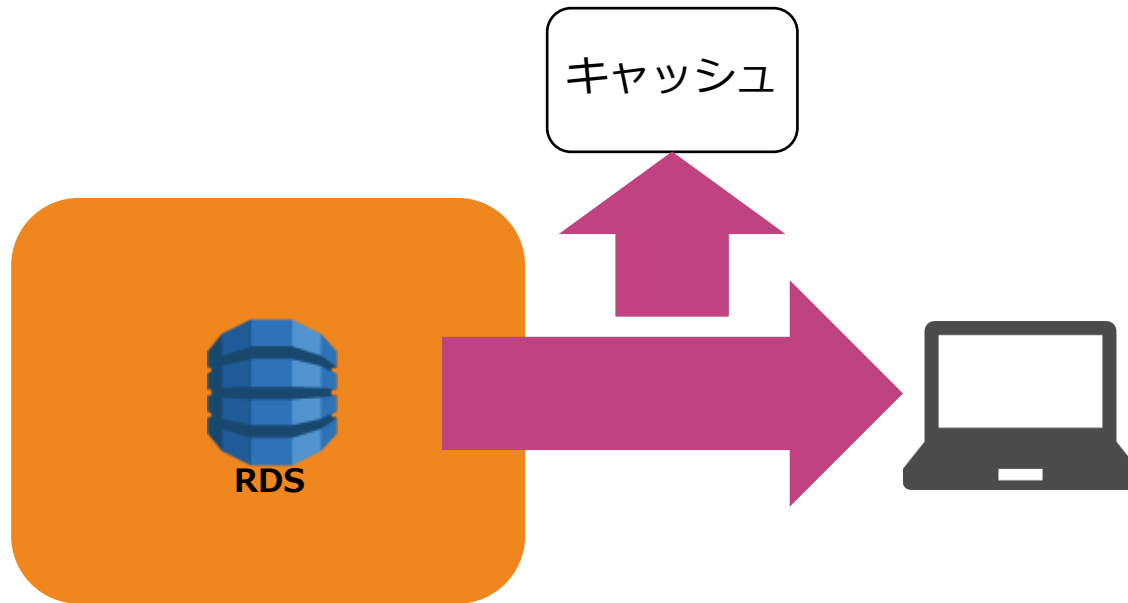
一度アクセスしたデータを保存して次回アクセス時に高速にアクセスできるようにする仕組み



データベースから画像などのデータを取得するにはそれなりに時間がかかる

キャッシュとは

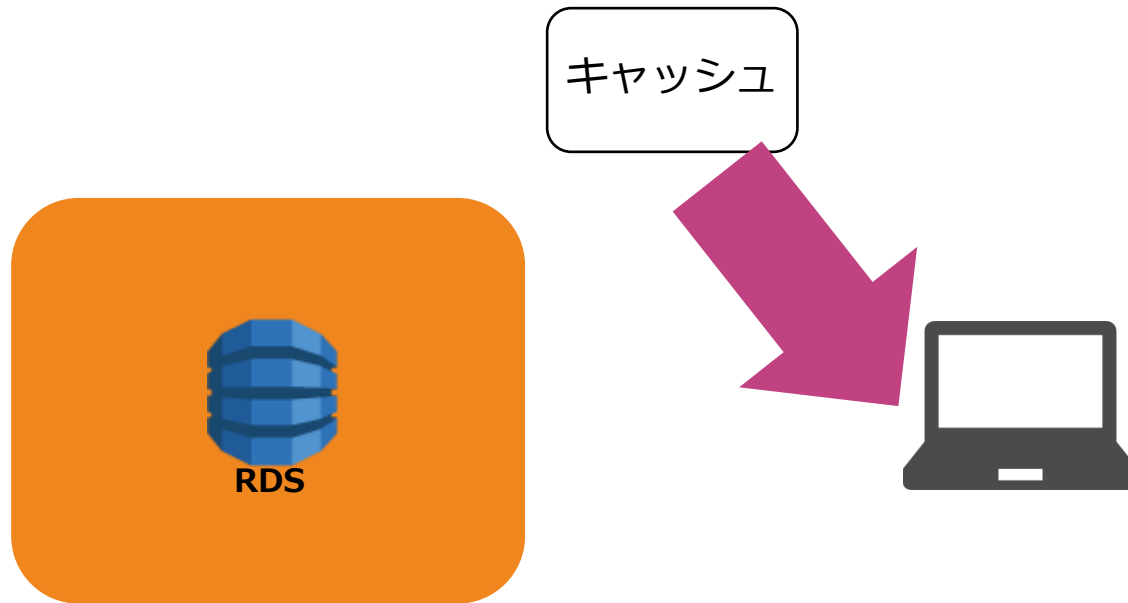
一度アクセスしたデータを保存して次回アクセス時に高速にアクセスできるようにする仕組み



データベースから画像などのデータを取
得するのにはそれなりに時間がかかる

キャッシュとは

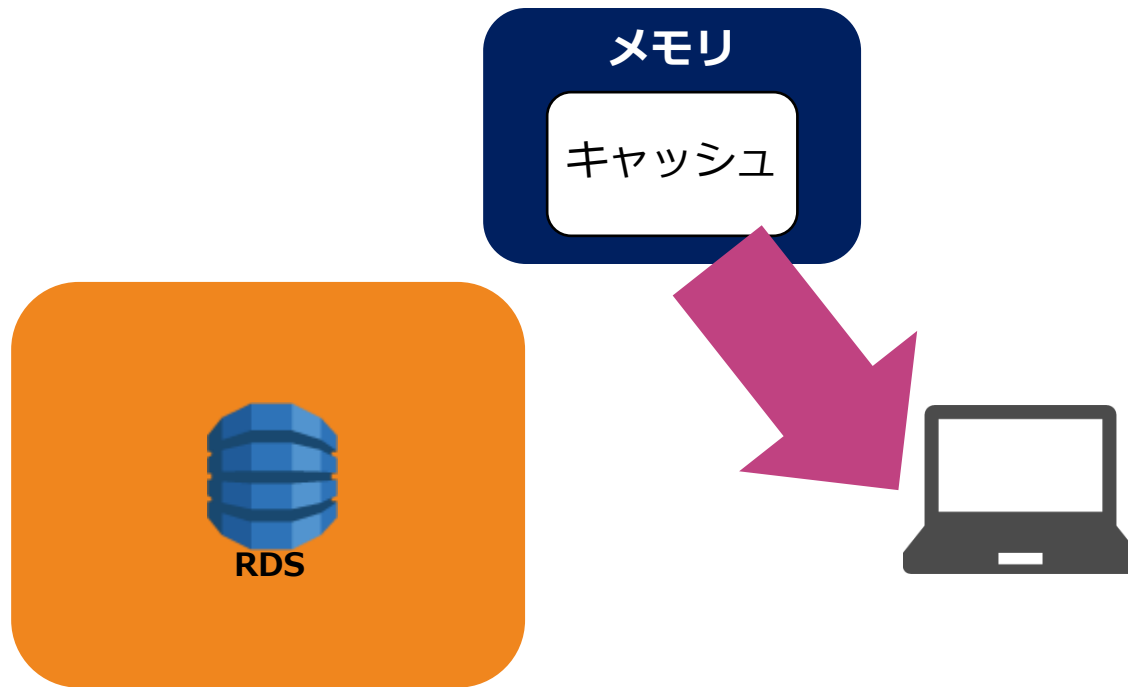
一度アクセスしたデータを保存して次回アクセス時に高速にアクセスできるようにする仕組み



次にアクセスした際はキャッシュから
データを取得する

インメモリキャッシュとは

メモリを活用して高速にキャッシュへのアクセスを可能にしたデータベースの仕組み



次にアクセスした際はキャッシュから
データを取得する

メモリ型DB

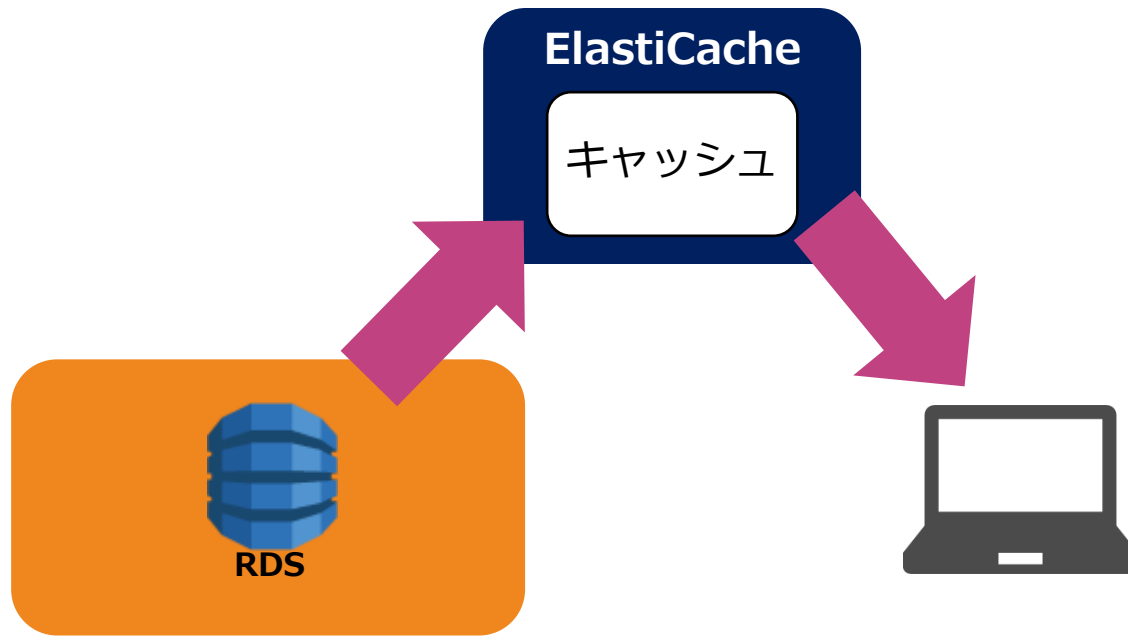
メモリDBをキャッシュを保持してデータの高速処理を実現する



ディスクより高速処理が可能

ElastiCacheとは何か？

ElastiCacheはメモリにキャッシュを保持して、ミリ秒以下の高速処理を実施するインメモリデータベース



次にアクセスした際はキャッシュからデータを取得する

ElastiCache

分散インメモリキャッシュサービスの構築・管理及びスケーリングを容易に実施することができるサービス

- キャッシュクラスタを数クリックで起動
- フルマネージド型でモニタリング、自動障害検出、復旧、拡張、パッチ適用、バックアップに対応し高可用性を実現
- 広く利用されている2種類のエンジンmemcached, /redisから選択可能

ユースケース

データアクセスを高速にしたいケースがあればキャッシュの活用を検討する

【ユースケース】

- セッション管理
- IOT処理とストリーム分析
- メタデータ蓄積
- ソーシャルメディアのデータ処理／分析
- Pub/Sub処理
- DBキャッシュ処理

ユースケース

アプリケーションでデータの即時反映が必要なケースなどに活用する

【ユースケース】

- ユーザーのマッチング処理
- レコメンデーションの結果処理
- 画像データの高速表示
- ゲームイベント終了時のランキング表示

ElastiCache

オープンソースのRedisとMemcachedを利用可能で汎用性あり

Redis

- ❑ 高速に値をRead/Writeできるインメモリキャッシュ型DB
- ❑ シングルスレッドで動作するインメモリキャッシュDBで全てのデータ操作は排他的
- ❑ スナップショット機能がある
- ❑ データを永続化できる

Memcached

- ❑ 高速に値をRead/Writeできるインメモリキャッシュ型DB
- ❑ マルチスレッドで動作するインメモリキャッシュDB
- ❑ スナップショット機能がない
- ❑ データを永続化できない
- ❑ フェイルオーバーや復元ができない。

ElastiCache

シンプルに利用する場合はMemcachedを利用するが、それ以外はRedisを利用する場合が多い

Redis

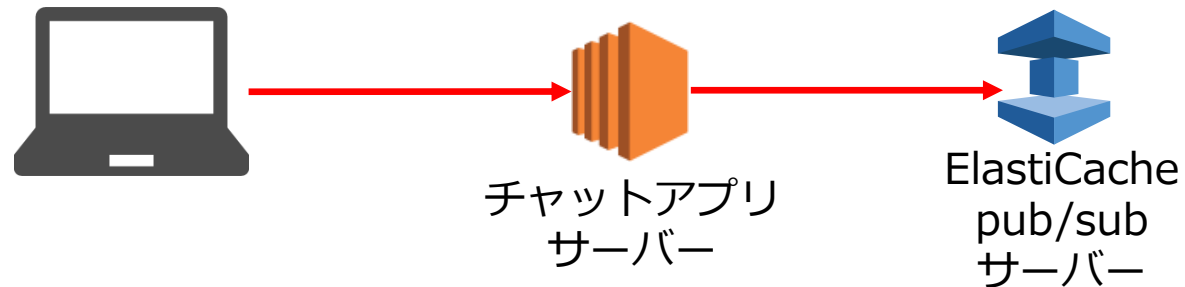
- ❑ 複雑なデータ型が必要である。
- ❑ インメモリデータセットをソートまたはランク付けする必要がある。
- ❑ 読込処理の負荷に対してAuto Scalingが利用可能になった（2021年）
- ❑ SSDキャッシュを利用して低コストで大容量のキャッシュ運用が可能
- ❑ pub/sub機能が必要
- ❑ 自動的なフェイルオーバーが必要である
- ❑ キーストアの永続性が必要である。
- ❑ バックアップと復元の機能が必要である。
- ❑ 複数のデータベースをサポートする必要がある。
- ❑ データ転送時にTLS暗号化が可能

Memcached

- ❑ シンプルなデータ型が必要である
- ❑ 複数のコアまたはスレッドを持つ大きなノードを実行する必要がある。
- ❑ システムでの需要の増減に応じてノードを追加または削除するスケールアウトおよびスケールイン機能が必要である。
- ❑ データベースなどのオブジェクトをキャッシュする必要がある。
- ❑ キーストアの永続性は必要ない
- ❑ バックアップと復元の機能が必要でない
- ❑ 複数のデータベースを利用できない
- ❑ データ転送時にTLS暗号化が可能に（2022年）

ユースケース（チャットアプリ）

ElastiCacheのpub/sub機能を活用したチャットアプリ



ElastiCache with Redis

その他に位置情報クエリ／Luaスクリプトによる操作や
pub/subモデルを活用可能

Luaスクリプト

- ❑ 移植性が高く、高速な実行速度などの特徴を持っているスクリプト言語

位置情報クエリ

- ❑ 経度・緯度などの位置情報をクエリ処理することが可能
- ❑ 検索距離や検索範囲の指定可能

pub/subモデル の利用

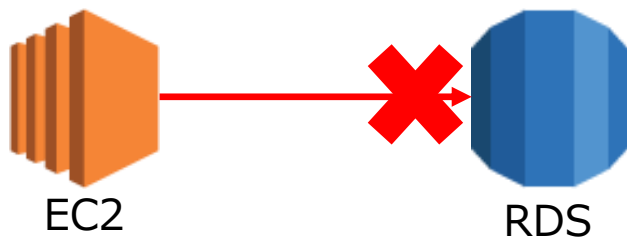
- ❑ 「イベントを起こす側」と「イベント処理を行う側」を分離するのがpub/subモデル
- ❑ メッセージ処理やイベント処理で活用

ElastiCacheの構成

キャッシュすべきデータを特定して、他のDBと合わせて利用するのが標準的な構成方法

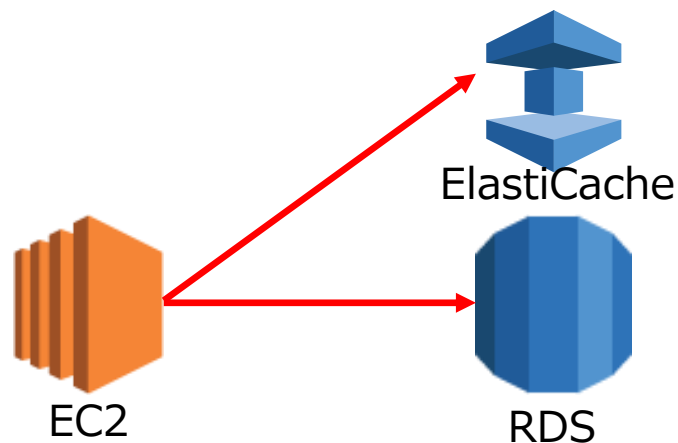
キャッシュ未使用パターン

DBアクセス負荷が増大すると処理能力が低下し可用性が低下する



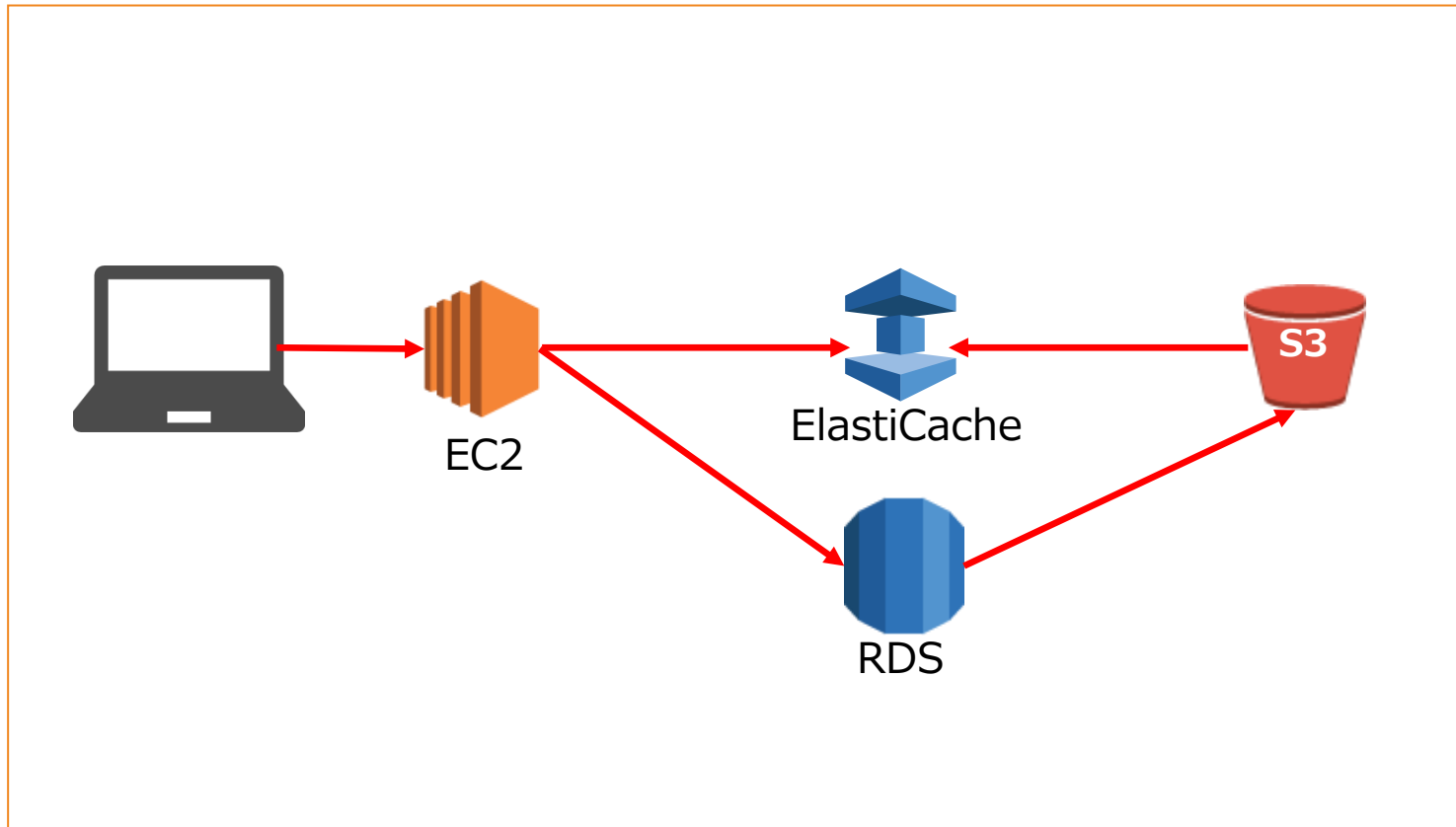
インメモリキャッシュ 利用パターン

アクセス頻度の高いクエリ結果をキャッシュに配置して可用性を高める



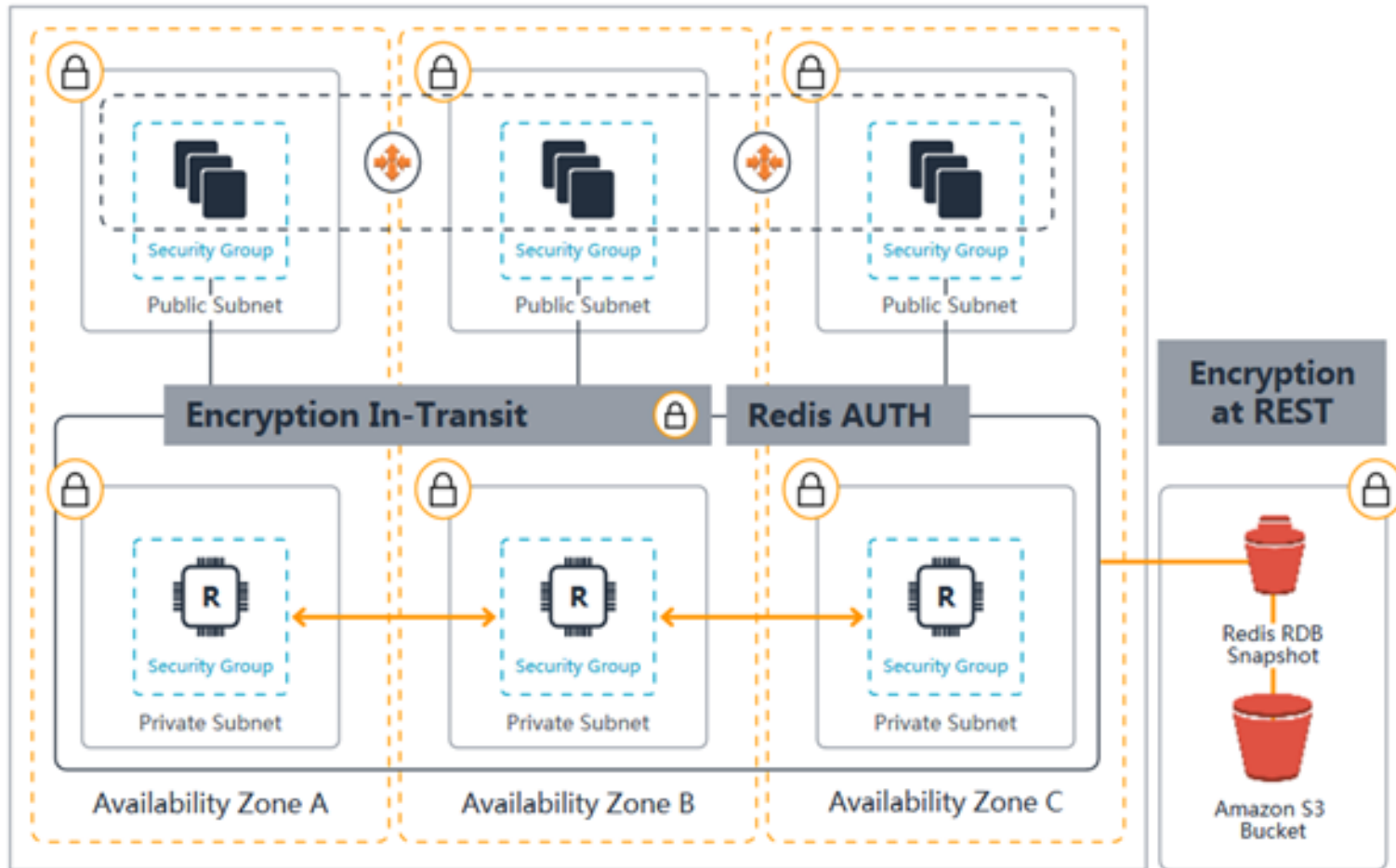
ElastiCacheの構成

ElastiCacheを活用したシンプルなアーキテクチャパターン



セキュリティ対応

ElastiCache Redisはデータ転送時の暗号化、データ保管時の暗号化およびRedis Authによる認証を実施することが可能



セキュリティ対応

ElastiCache Redisはデータ転送時の暗号化、データ保管時の暗号化およびRedis Authによる認証を実施することが可能

保管時の暗号化

- ❑ AWS KMSにより、データ同期やバックアップオペレーションの実行中にオンディスクデータが暗号化される。
- ❑ 対象は同期・バックアップ・スワップ操作中のディスクとAmazonS3に保存されているバックアップデータ。

通信の暗号化

- ❑ RedisとMemcachedでTLS暗号化を実施できる。
- ❑ データ転送時 (クラスターのノード間、クラスターとアプリケーション間など) に暗号化を実施

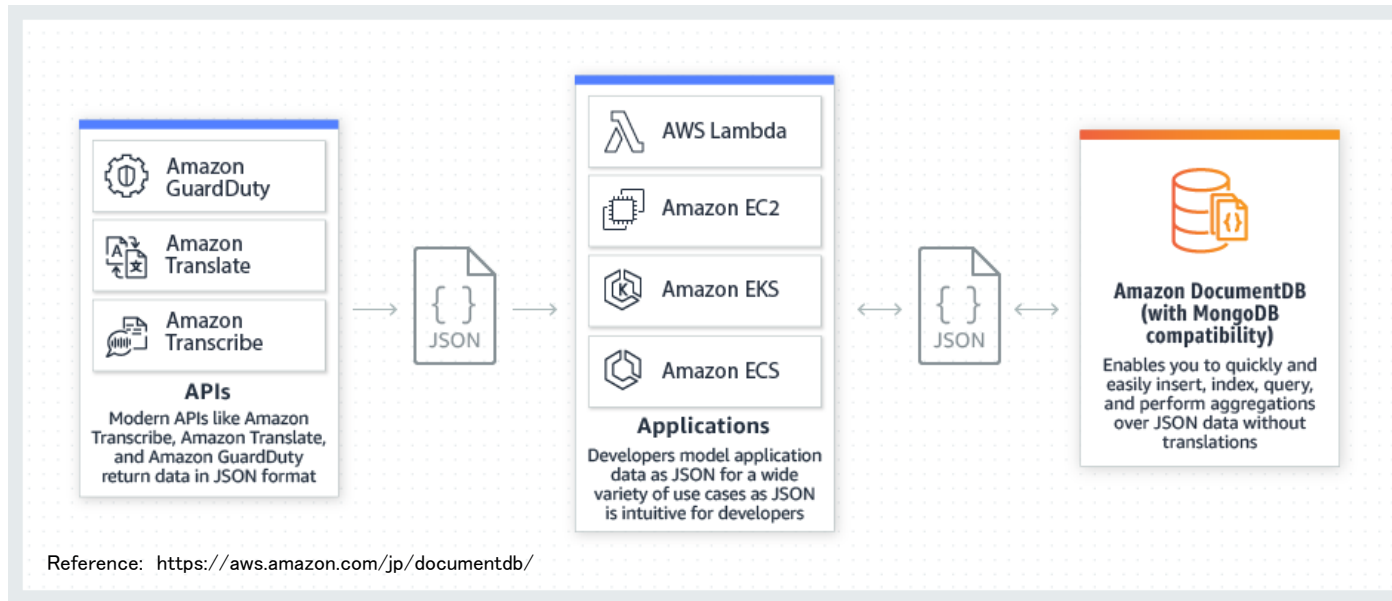
Redis Auth

- ❑ Redis認証トークンを使用して、クライアントがコマンド実行前にトークン(パスワード)を要求して認証を実施する。

その他の
データベース

Amazon DocumentDB

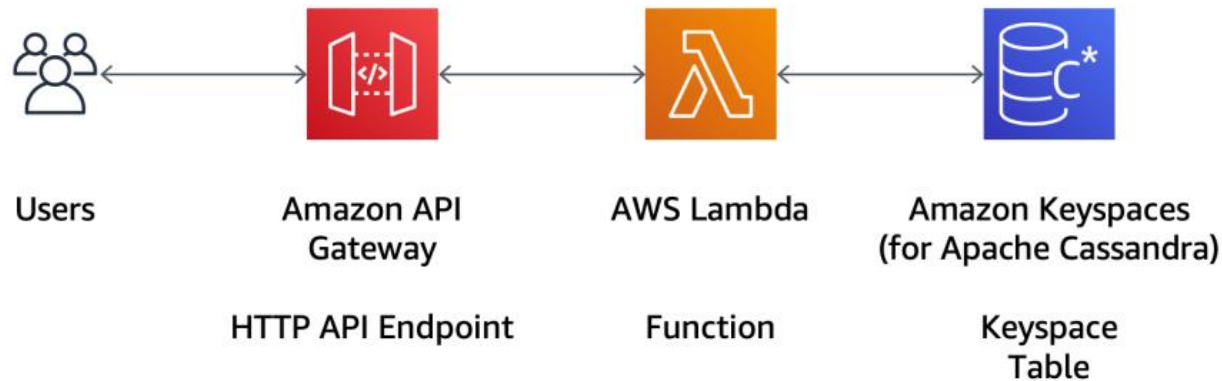
ミッションクリティカルなMongoDB のワークロードを運用するためのJSONドキュメントデータベース



- アプリケーションからJSON 形式で、データを格納して、クエリ、インデックス作成、および集計することが可能
- ペタバイトのストレージを使用して毎秒数百万規模の高速ワークロードに対応可能できるように自動スケーリング
- MongoDB API、ドライバー、ツールを使用できる。
- コンテンツ管理システム (CMS) に保存されているレビューやコンテンツへの高速アクセス
- 何百万ものユーザーのプロファイルと設定を管理

Amazon Keyspaces (for Apache Cassandra)

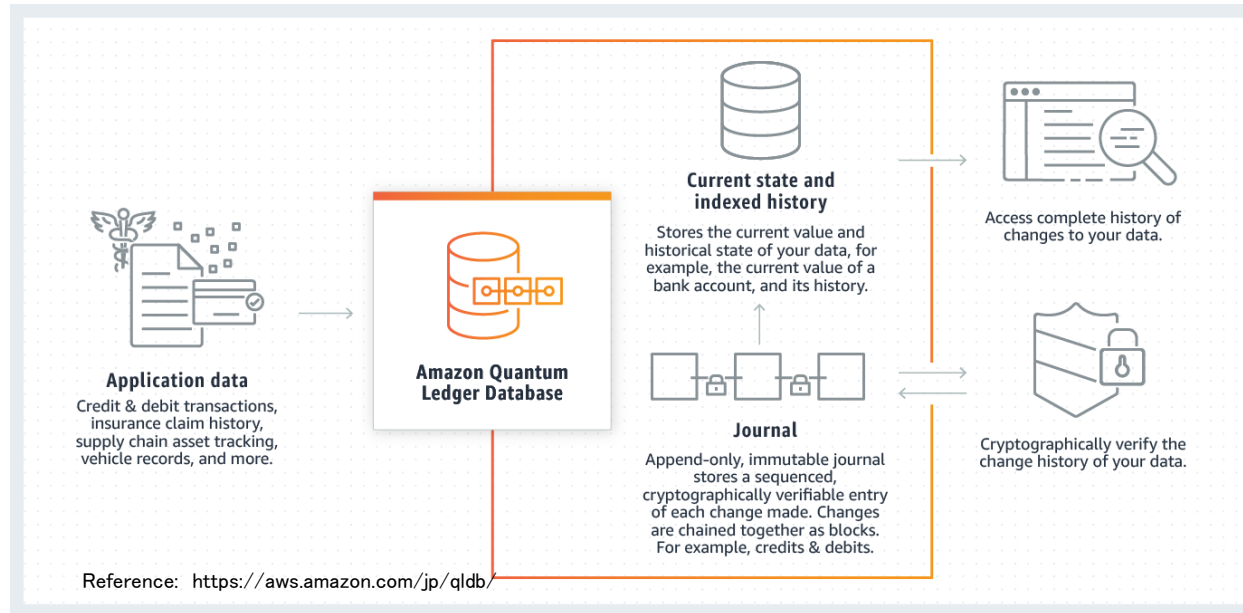
Apache Cassandra向けのフルマネージド型のNoSQL型データベースサービス



- AWS クラウドで Cassandra テーブルをセットアップ、セキュリティ保護、およびスケーリングが可能
- Cassandra アプリケーションコードとCassandra クエリ言語 (CQL) API コード、Cassandra ドライバーなどのツールを使用可能
- 数ミリ秒単位の処理時間／毎秒数千のリクエストに対応するアプリケーションを構築可能
- 産業用機器のメンテナンス、取引監視、フリート管理、ルート最適化など、1 桁ミリ秒のレイテンシが必要なアプリケーションのデータを高速処理
- マルチAZ構成により99.99% の可用性
- 直前の35日間のポイントタイムリカバリが可能

Amazon Quantum Ledger Database (QLDB)

フルマネージド型のブロックチェーン技術を利用した分散台帳データベース



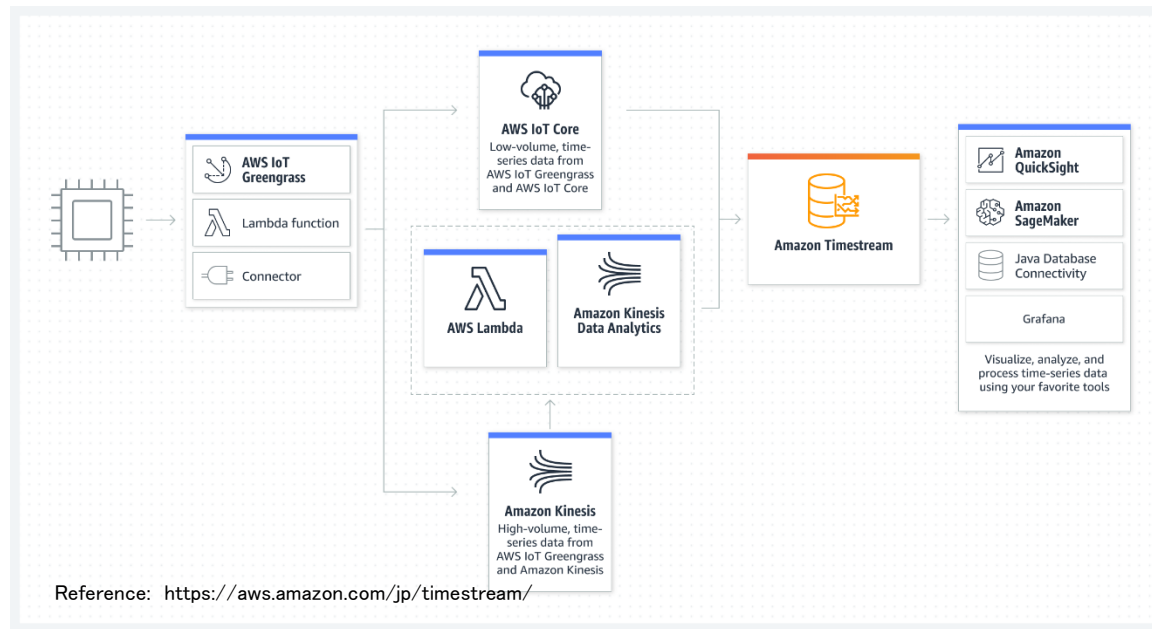
- トランザクション処理を変更不可のジャーナルに、暗号化技術を利用して各トランザクションログが保存される。変更はブロックとして引き継ぐ。
- データの全変更がすべて追跡され、変更履歴が長期間維持される。
- 透過的かつイミュータブルに検証可能なトランザクションログの保存向き。

【ユースケース】

- 台帳形式のアプリケーション開発を支援
- クレジット取引などの金融取引の完全で正確なレコードを処理する
- 各取引の履歴を記録し、施設から店舗までの製造、出荷、保存、販売の各バッチの詳細を提供する。

Amazon Timestream

IoT および運用アプリケーションに適した、高速かつスケール
ブルなサーバーレス時系列データベースサービス

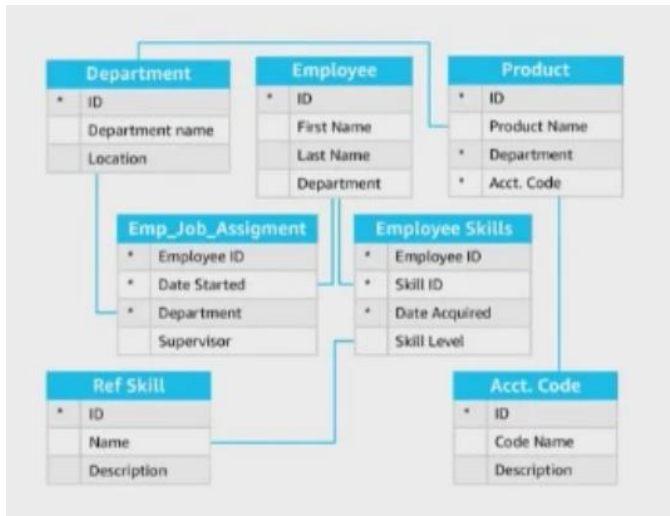


- 1日あたり数兆件規模のイベントを最大 1,000 倍の速度でより簡単に保存および分析する。
- 毎日数百万のクエリを処理し、必要に応じて自動的にスケーリング
- 直近のデータをメモリに保存して高速処理し、過去データはコスト最適なストレージに移行する。
- IoT アプリケーションによって生成された時系列データを迅速に分析する。

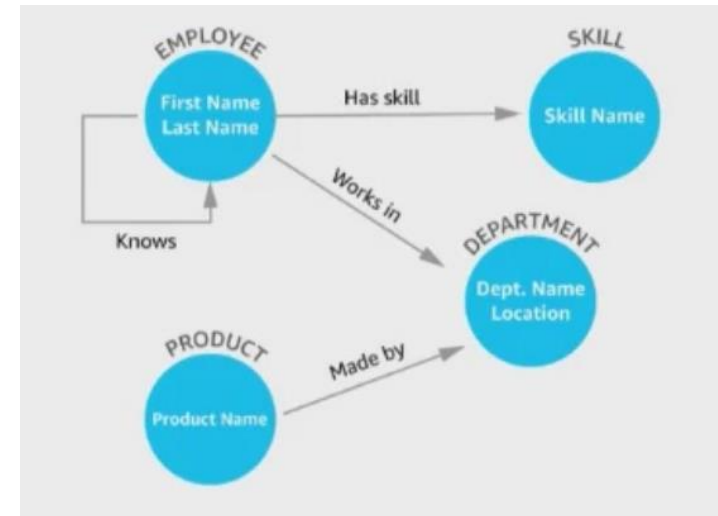
Amazon Neptune

ID グラフ、ナレッジグラフ、不正検出などのグラフのユースケースを強力にサポートする高速のグラフデータベース

ER図



グラフ



Reference: 2017 AWS Re invent資料

- データ間の関連性をグラフ形式で関連付けて、複雑な処理を実現する。
- ER図では困難な関係性を 直感的にわかりやすいグラフ形式として構成し, 検索クエリーも速度に加えて, コード量も減らすことができる。
- 毎秒 100,000 以上のクエリ実行が可能
- クラスタあたり最大 128Tib のストレージスケーリング、最大15 レプリカのリードスケーリングが可能
- ソーシャルグラフや顧客の 360度ビューなどが実現可能

AWS Backup

AWSのデータベースとストレージのバックアップを自動化する一元管理ソリューション

- Amazon EC2
- Amazon S3 データ
- Amazon EBS
- Amazon DynamoDB
- Amazon RDS
- Amazon Aurora
- Amazon EFS
- Amazon FSx
- AWS Storage Gateway (ボリュームゲートウェイ)
- Amazon DocumentDB
- Amazon Neptune
- Amazon Redshift
- Amazon Timestream

